Dissertation presented to the Instituto Tecnológico de Aeronáutica, in partial fulfillment of the requirements for the degree of Master of Science in the Graduate Program of Electronic Engineering and Computer Science, Field of Systems and Control.

Leandro Ventricci

DESIGN AND EXPERIMENTAL VALIDATION OF A MODULAR BALLBOT PLATFORM FOR BALANCE CONTROL STUDIES

Dissertation approved in its final version by signatories below:

Prof. Dr. Cairo Lúcio Nascimento Júnior

Advisor

Campo Montenegro
São José dos Campos, SP - Brazil
2025

Cataloging-in Publication Data
Documentation and Information Division

Ventricci, Leandro
Design and Experimental Validation of a Modular Ballbot Platform for Balance Control Studies / Leandro Ventricci.
São José dos Campos, 2025.
116f.

Dissertation of Master of Science – Course of Electronic Engineering and Computer Science. Area of Systems and Control – Instituto Tecnológico de Aeronáutica, 2025. Advisor: Prof. Dr. Cairo Lúcio Nascimento Júnior.

1. Automatic Control. 2. Robots. 3. Control Systems. I. Instituto Tecnológico de Aeronáutica. II. Title.

BIBLIOGRAPHIC REFERENCE

VENTRICCI, Leandro. **Design and Experimental Validation of a Modular Ballbot Platform for Balance Control Studies**. 2025. 116f. Dissertation of Master of Science – Instituto Tecnológico de Aeronáutica, São José dos Campos.

CESSION OF RIGHTS

AUTHOR'S NAME: Leandro Ventricci
PUBLICATION TITLE: Design and Experimental Validation of a Modular Ballbot Platform for Balance Control Studies.
PUBLICATION KIND/YEAR: Dissertation / 2025

Leandro Ventricci
Praça Marechal Eduardo Gomes, 50 - Vila das Acácias
CEP: 12228-900, São José dos Campos - SP

DESIGN AND EXPERIMENTAL VALIDATION OF A MODULAR BALLBOT PLATFORM FOR BALANCE CONTROL STUDIES

Leandro Ventricci

Thesis Committee Composition:

Prof. Dr.	Eduardo Lenz Cesar	Chairperson	-	ITA
Prof. Dr.	Cairo Lúcio Nascimento Júnior	Advisor	-	ITA
Prof. Dr.	Jacques Waldmann	Internal Member	-	ITA
Prof. Dr.	Sérgio Ronaldo Barros dos Santos	External Member	-	UNIFESP-SJC

ITA

Acknowledgments

I express my sincere gratitude to all those who have supported me throughout this academic journey. Without their encouragement, guidance, and collaboration, this thesis would not have been possible.

First, I thank my family for their unconditional love and support. To my parents and brother, your constant faith in me has been a driving force throughout this entire process, and I am deeply grateful for your unwavering support.

I would also like to extend a special thanks to my supervisor, Prof. Cairo Lúcio Nascimento Júnior, for his invaluable guidance, expertise, and encouragement throughout this work. His support has been essential in shaping the direction of my research, and I am thankful for the opportunity to work under his mentorship.

A heartfelt thank you goes to my friends at the Laboratory of Intelligent Machines (LMI), Luiz Eugênio Santos Araújo Filho and Juan Ramón Belchior de França Silva. Your thoughtful advice, inspiring conversations, and shared moments of laughter have made this experience both enjoyable and meaningful. I am truly grateful for your friendship and for the invaluable contributions you made to this work. I would also like to express my gratitude to all my colleagues at the Aeronautics Institute of Technology (ITA) for their collaboration and insights, which have enriched this thesis and helped me grow as a researcher.

Finally, I would like to acknowledge the financial support from Coordination for the Improvement of Higher Education Personnel (CAPES), which made this research possible.

Thank you all for your support and for being a part of this journey.

Resumo

Este trabalho apresenta o projeto, a implementação e a validação experimental de uma aquitetura modular para um robô omnidirectional do tipo Ballbot desenvolvido para estudos de controle de equilíbrio. O sistema emprega uma arquitetura em camadas, na qual um *loop* de controle de alta frequência é executado em um microcontrolador ESP32, enquanto as tarefas de comunicação e monitoramento são realizadas por um microcomputador Raspberry Pi 4 utilizando *Robot Operating System* (ROS). Esta arquitetura garante que o *loop* de controle seja executado de forma confiável na frequência necessária para operação em tempo real, permite o monitoramento contínuo de todos os estados do robô e preserva a flexibilidade do sistema. O design modular também possibilita fácil integração de novos sensores, permitindo que estes sejam rapidamente substituídos e testados, facilitando a avaliação de diferentes métodos de estimação de pose, como o *Perspective-n-Point* (PnP) utilizando câmeras externas e o *Laser Scan Matching* utilizando sensores do tipo *Light Detection and Ranging* (LiDAR). Para controlar o movimento do robô, foram implementados e testados um Regulador Linear Quadrático (LQR) e um controlador Proporcional-Derivativo (PD). O controlador LQR foi empregado para controlar o equilíbrio e a posição do robô nos planos X–Z e Y–Z, enquanto o controlador PD foi empregado para controlar a direção de proa do robô no plano X–Y. Resultados experimentais demonstram que o controlador mantém o ballbot equilibrado em torno da posição vertical dentro de uma região de $\pm 5^{\circ}$ tanto nos ângulos de rolagem quanto de arfagem. Além disso, o controlador regula com sucesso a posição do robô, alcançando desvios inferiores a 20 cm para ambos os sensores avaliados, apesar das diferenças na frequência de atualização e nas características de ruído, dado que a utilização do LiDAR resultou na introdução de vibrações adicionais, as quais afetaram as leituras da Unidade de Medição Inercial (IMU). De modo geral, a arquitetura proposta oferece uma plataforma flexível e adaptável, adequada para pesquisa e atividades de ensino contínuas. Ela possibilita a experimentação rápida com novos sensores e estratégias de controle, proporcionando conhecimentos valiosos sobre os desafios práticos envolvidos na implementação do controle de equilíbrio em robôs dinamicamente instáveis. Todo o material do projeto, incluindo o código-fonte, os arquivos da estrutura do robô e os vídeos dos experimentos, está disponível no repositório do GitHub: `https://github.com/Intelligent-Machines-Lab/Ballbot_LMI_V2`.

Abstract

This work presents the design, implementation, and experimental validation of a modular Ballbot framework developed for balance control studies. The system employs a layered architecture in which a high-frequency control loop runs on an ESP32 microcontroller, while communication and monitoring tasks are handled by a Raspberry Pi 4 through the Robot Operating System (ROS) framework. This architecture ensures that the control loops execute reliably at the required frequency for real-time operation, enables continuous monitoring of the robot's complete state, and preserves overall system flexibility. The modular design further enables easy hardware integration, allowing sensors to be quickly swapped and tested, facilitating the evaluation of different pose estimation methods, such as the Perspective-n-Point (PnP) using external cameras, and Laser Scan Matching using Light Detection and Ranging (LiDAR) sensors. To regulate the robot's motion, both a Linear Quadratic Regulator (LQR) and a Proportional-Derivative (PD) controller were implemented and tested. The LQR was employed to control the robot's balance and position in the XZ and YZ planes, while the PD controller regulated the robot's heading. Experimental results demonstrate that the controller maintains the ballbot balanced around the upright position within a region of $\pm 5^{\circ}$ in both roll and pitch angles. Additionally, the controller successfully regulates the robot's position, achieving deviations below 20 cm for both evaluated sensors, despite differences in update frequency and noise characteristics, with the LiDAR introducing additional vibrations that affected the Inertial Measurement Unit (IMU) readings. Overall, the proposed architecture provided a scalable and adaptable platform for continued research and education. It enables rapid experimentation with new sensors and control strategies, while offering valuable insights into the practical challenges of implementing balance control on dynamically unstable robots. The complete source code, structure design files, and videos of the experimental results are available in the project's GitHub repository: `https://github.com/Intelligent-Machines-Lab/Ballbot_LMI_V2`.

List of Figures

List of Tables

List of Abbreviations and Acronyms

2D	Two-dimensional
3D	Three-dimensional
CAD	Computer Aided Design
CMU	Carnegie Mellon University
DOF	Degrees of Freedom
FIR	Finite Impulse Response
ICP	Iterative Closest Point
IMU	Inertial Measurement Unit
LiDAR	Light Detection and Ranging
LQR	Linear Quadratic Regulador
LTI	Linear Time Invariant
MEMS	Micro-Electro-Mechanical System
PD	Proportional-Derivative
PID	Proportional-Integral-Derivative
PLA	Polylactic acid
PnP	Perspective-n-Point
PURE	Personal Unique Rolling Experience
ROS	Robot Operating System
SISO	Single-Input, Single-Output

Contents

1 Introduction

1.1 Motivation

Balancing robots represent a fundamental class of underactuated and nonlinear systems that have attracted significant attention in control theory, robotics, and embedded systems research. Their inherent instability and dynamic coupling make them ideal platforms for evaluating advanced control algorithms, state estimation techniques, and sensor fusion strategies. Among these systems, the Ballbot, a robot that balances dynamically on a single spherical wheel, stands out due to its full omnidirectional mobility and compact footprint. These characteristics make it particularly suited for operation in human-centered environments, where agile and stable locomotion is required within confined spaces.

Since its first development at Carnegie Mellon University in 2005, the Ballbot concept has inspired extensive research in areas such as dynamic modeling, mechanical design, and real-time control. Despite the maturity of theoretical studies and simulation-based developments, the transition from simulation to a reliable physical prototype remains a considerable challenge. This difficulty stems from the strong coupling between mechanical, electronic, and software subsystems, as well as the system's sensitivity to noise, time delays, and unmodeled nonlinearities. Consequently, constructing and experimentally validating a Ballbot platform not only facilitates practical evaluation of control strategies but also contributes to a broader understanding of real-time control of unstable robotic systems.

Beyond serving as a research platform, the Ballbot also offers remarkable educational potential. As a compact, low-cost, and yet complex dynamic system, it provides an excellent hands-on learning tool for undergraduate courses in control and robotics. Its modular design and open-source software implementation could enable students to explore key topics such as sensor integration, real-time control, and state estimation, effectively bridging the gap between theoretical study and practical experimentation.

1.2 Objectives

The primary objective of this work is to design, model, and experimentally validate a low-cost Ballbot platform capable of achieving stable balance and controlled motion. The system is intended to serve both as a research platform for control studies and as an educational tool for laboratory-based learning.

To achieve this goal, the following specific objectives are established:

- Develop a fully functional Ballbot prototype integrating mechanical, electronic, and software subsystems.
- Design and implement state-feedback control strategies, including Linear Quadratic Regulator (LQR) and Proportional-Derivative (PD) controllers, for stabilization and motion control.
- Integrate real-time sensing, communication, and control using a dual-layer embedded architecture based on the ESP32 microcontroller and the Raspberry Pi platform, enabling real-time monitoring and interaction with the system.
- Evaluate the performance of the proposed system through experimental testing under Laser Detection and Ranging (LiDAR) and camera-based pose estimation.

1.3 Contributions

The main contributions of this work can be summarized as follows:

- Development of a complete open-source Ballbot platform featuring a modular, low-cost mechanical design fabricated using 3D-printed components;
- Implementation of a dual-layer embedded control architecture integrating ESP32 and Raspberry Pi hardware for real-time operation with Robot Operating System (ROS) communication;
- Experimental validation of LQR and PD controllers for stabilization and motion control using both LiDAR and camera-based pose estimation.

1.4 Structure of the Dissertation

This dissertation is structured as follows: Chapter 1 introduces the motivation, objectives, and contributions of the research. Chapter 2 presents a review of the literature, including the historical development of Ballbots, the dynamic models for the X–Z and X–Y planes, the design of controllers such as LQR and PD, and pose estimation techniques based on ArUco markers and laser scan matching. Chapter 3 describes the proposed solution, covering 2D simulations, the hardware and software architecture, and the overall control architecture. Chapter 4 details the experimental validation, presenting results from 3D simulations and physical experiments using both camera and LiDAR based pose estimation, and providing a comparative analysis of system performance. Finally, Chapter 5 summarizes the main conclusions and discusses directions for future research.

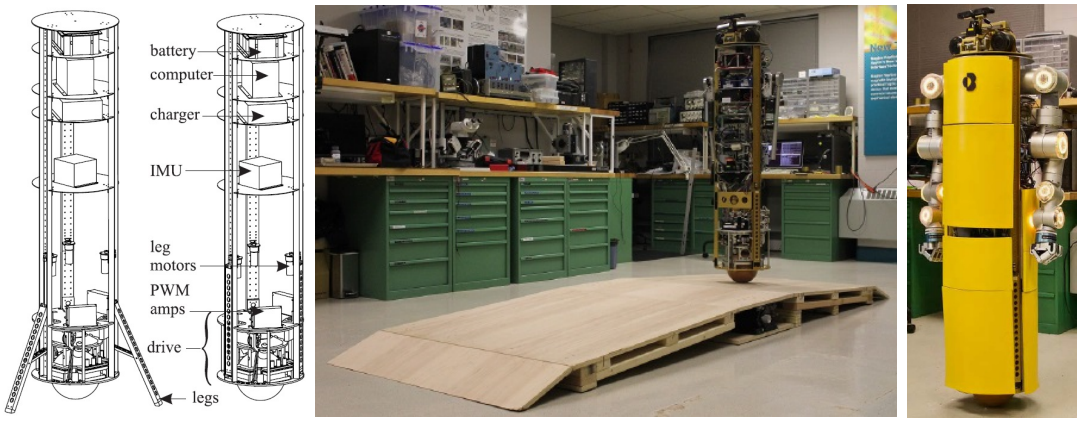
2 Literature Review

2.1 Ballbot History

The concept for the Ballbot was first introduced in 2005 as a human-scale mobile robot designed to maintain dynamic stability while balancing on a single spherical wheel. This unique configuration enables agile, omnidirectional motion and makes the platform particularly suitable for safe physical interaction with humans. The original prototype, developed at Carnegie Mellon University's (CMU) Robotics Institute, featured a 1.5 m-tall cylindrical body with a diameter of 400 mm and a total mass of 45 kg (LAUWERS *et al.*, 2006). The system balanced on a 190.5 mm hollow aluminum sphere coated with 12.7 mm of urethane. Locomotion was achieved through an inverse mouse-ball drive mechanism, in which actuated rollers mounted on the robot's body transmitted motion to the ball, although control of rotation about the vertical axis was not implemented in the initial design. A simplified dynamical model was proposed, modeling the Ballbot as two uncoupled two-dimensional (2D) inverted pendulum systems. Stability was realized using an inner-loop Proportional-Integral (PI) controller to regulate ball velocity and an outer-loop Linear Quadratic Regulator (LQR) to stabilize the body.

Improvements to the CMU Ballbot were subsequently introduced in Nagarajan *et al.* (2012), in which a pair of two-degrees-of-freedom (2-DOF) arms was integrated and a trajectory planning method for the combined system was proposed. In a later work, a four-wheel inverse mouse-ball drive and an independent yaw actuation mechanism were incorporated, and the control framework was revised to support balancing, station-keeping, and yaw regulation tasks (NAGARAJAN *et al.*, 2014). In further research, the Ballbot's ability to assist humans in standing from a chair was demonstrated, with lean angles greater than 15° being reached and more than 120 N of supportive force being provided during the motion (SHOMIN *et al.*, 2015). In Vaidya *et al.* (2015), equations of motion were derived for operation on sloped surfaces and with center-of-mass offsets, and a compensation strategy was implemented to maintain stability under such conditions. Semi-autonomous manipulation was later achieved in Sonnleitner *et al.* (2019) through an algorithm that allowed a Ballbot equipped with arms to detect, navigate to, lift, and transport heavy objects of unknown mass up to 15 kg. This was followed by the development of a 14-DOF dual-manipulation system comprising two 7-DOF arms with anthropomorphic kinematics, capable of handling 10 kg payloads at full extension, as described in Shut e Hollis (2019).

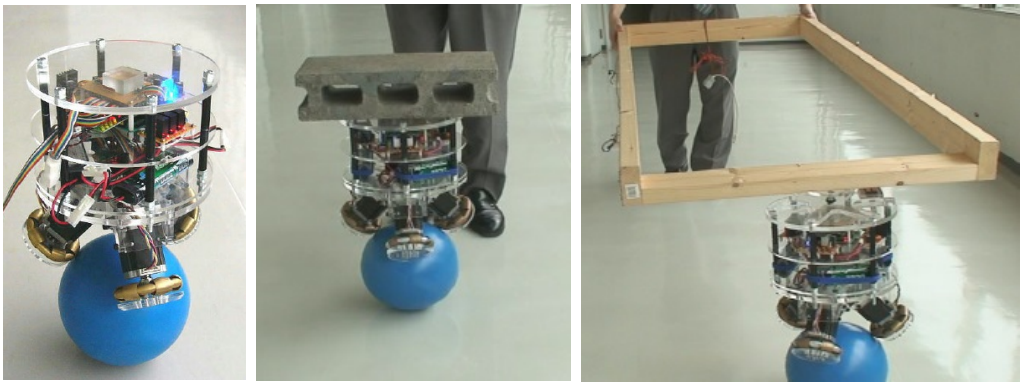
Following CMU's initial work, several research groups developed their own Ballbot variants. An alternative drive concept employing three omnidirectional wheels powered



(a) (b) (c)

FIGURE 2.1 – Evolution of the CMU Ballbot: (a) original prototype with inverse mouse-ball drive mechanism (LAUWERS *et al.*, 2006); (b) demonstration of operation on sloped surfaces (VAIDYA *et al.*, 2015); and (c) dual 7-DOF manipulation system capable of handling 10 kg payloads (SHUT; HOLLIS, 2019).

by stepping motors was introduced in Kumagai e Ochiai (2008), inherently enabling yaw control without the need for a separate mechanism. The first prototype was approximately 500 mm tall, weighed 11 kg, and balanced on a 200 mm rubber-coated bowling ball. Subsequent experiments demonstrated that the robot maintained stability while supporting an additional 10 kg concrete block without any changes to its control gains (KUMAGAI; OCHIAI, 2009).

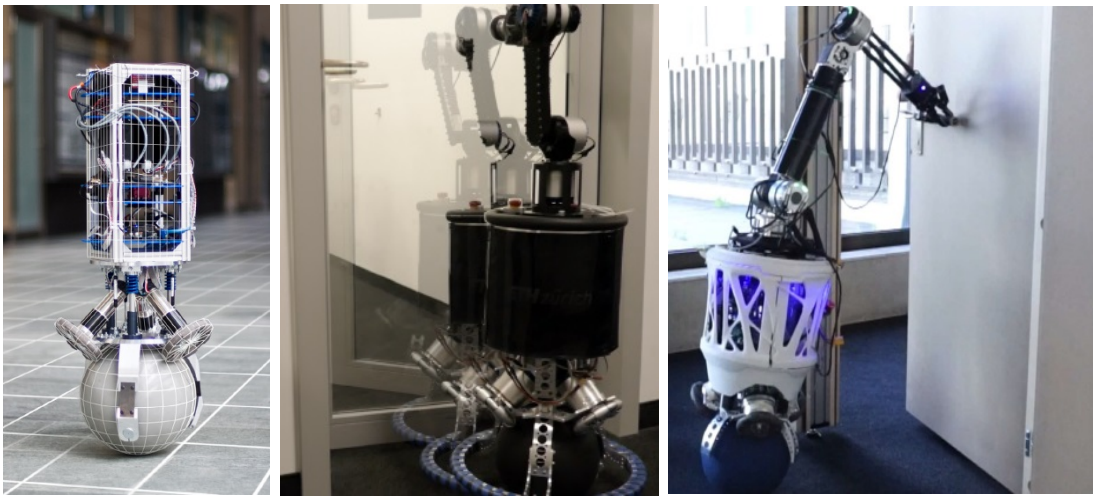


(a) (b) (c)

FIGURE 2.2 – BallIP platform: (a) first prototype with three omnidirectional wheels and stepping motors (KUMAGAI; OCHIAI, 2008); (b) stability demonstration with 10 kg concrete block; and (c) demonstration as an assistant to humans in load handling scenarios (KUMAGAI; OCHIAI, 2009).

Another notable design was the Rezero platform, which was the first to demonstrate high maneuverability and exceptional performance. Velocities of up to 3.5 m/s and tilt angles reaching 17° were reported in Fankhauser e Gwerder (2010), enabled by a novel three-dimensional (3D) dynamic model that explicitly accounted for system couplings and by the use of a gain-scheduled controller. The platform's key specifications included a 9.2 kg body mass, a 0.2 m body diameter, and a 2 mm thick hollow aluminum sphere

weighing 2.29 kg, coated with a 4 mm high-stiction synthetic layer. Subsequent research on the Rezero platform later focused on the development of advanced control strategies for complex tasks. In Farshidian *et al.* (2014), a two-step algorithm was proposed that combines a model-based controller, the iterative Linear Quadratic Gaussian (iLQG), with a model-free reinforcement learning approach, Path Integral Policy Improvement (PI^2), to compensate for model inaccuracies and enhance real-world performance. To enable autonomous mobile manipulation, a whole-body optimal control framework using Model Predictive Control (MPC) was developed to jointly address manipulation, balancing, and interaction within a unified optimization process (MINNITI *et al.*, 2019). The approach was further extended through the incorporation of adaptive control schemes, which permitted interaction with unmodeled environments without requiring re-tuning or pre-modeling, as demonstrated in door-opening and object-lifting tasks (MINNITI *et al.*, 2021).



(a) (b) (c)

FIGURE 2.3 – Rezero Ballbot platform: (a) original high-performance prototype achieving 3.5 m/s velocities (FANKHAUSER; GWERDER, 2010); (b) whole-body MPC for autonomous mobile manipulation (MINNITI *et al.*, 2019); and (c) adaptive control schemes incorporating MPC for interaction with unmodeled environments (MINNITI *et al.*, 2021).

Parallel efforts explored alternative control designs. In Blonk (2014), a 3D dynamic model was developed, and two linear controllers were designed for the ball-balancing robot: a Linear Quadratic Regulator (LQR) and a Single-Input Single-Output (SISO) loop-shaping controller. Simulation results indicated superior balancing performance with the SISO loop-shaping approach, however, excessive gyroscope noise compromised physical implementation. A Finite Impulse Response (FIR) low-pass filter was applied to mitigate the noise, allowing the LQR controller to stabilize the robot. Position control remained unattainable, as the noise limited the controller gains required for larger angular motions, ultimately making the SISO loop-shaping approach impractical in hardware.

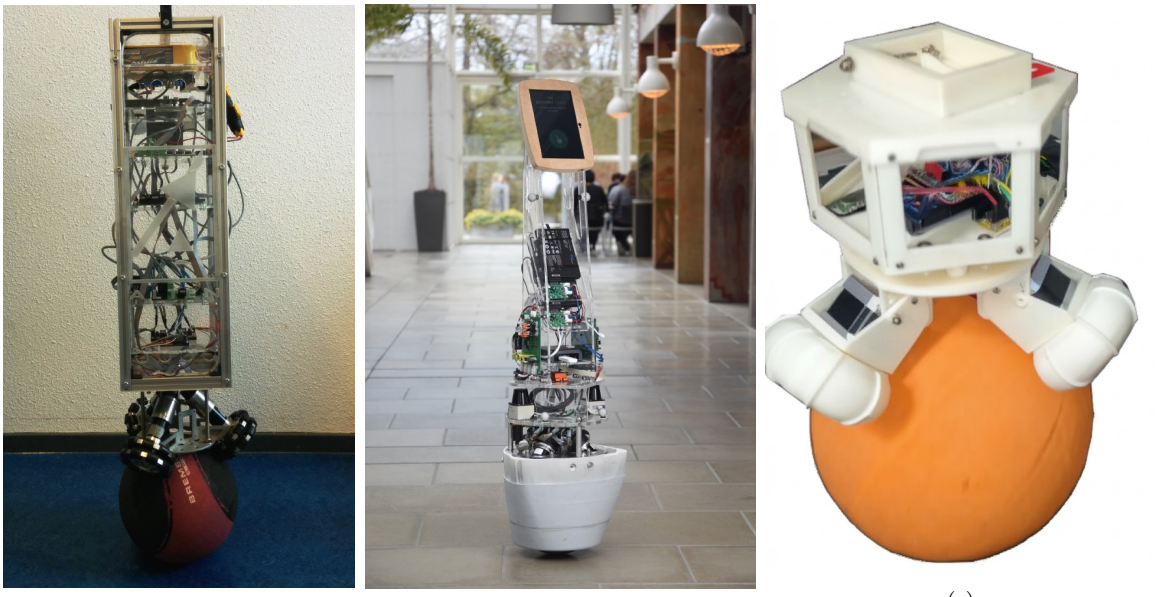
A more sophisticated quaternion-based dynamic model was later formulated for the Kugle V1 Ballbot in Jespersen (2019), a 16 kg Ballbot prototype employing a three-

omniwheel drive mechanism. The model was derived using Lagrangian mechanics and utilized for the design of a sliding mode controller for orientation stabilization, which was subsequently compared against an LQR balance controller. Nearly identical performance was observed between the two approaches. Extended Kalman filters were employed for state estimation, and the sliding mode control architecture was implemented in a cascaded configuration combining an LQR with a MPC controller. Through this design, velocity tracking of up to 1 m/s and 1 rad/s were achieved, with inclination tracking errors below 1°. It was concluded that, although the quaternion-based formulation increases modeling complexity, it provides a viable approach for controlling the Ballbot.

Further work presented a low-cost Ballbot capable of performing both translation and rotation, allowing the robot to execute missions through waypoint-based navigation (JESUS, 2021). The robot's structure and omnidirectional wheels were fabricated using 3D printing technology, and the controllers were successfully implemented on a low-cost hardware platform (Arduino Mega) with inexpensive sensors (MPU6050 Inertial Measurement Unit), demonstrating that the design is both affordable and easily replicable. Several practical challenges were encountered, including high friction in the omnidirectional wheel rollers, which limited angular corrections, and bias errors in the accelerometer readings, which significantly affected balance on the sphere. These findings underscored the importance of initializing the system on a minimally inclined plane to compute accurate bias averages. The LQR controller was effectively implemented based on the planar 2D dynamic model, successfully regulating the pitch and roll angles and limiting oscillatory deviations to within 2°. Additionally, the study demonstrated successful object-tracking functionality in both yaw and translational motion[1].

Recent work explored the development of ballbots designed for human-riding applications, emphasizing the exploitation of their omnidirectional maneuverability and dynamic stability (XIAO, 2022). The study highlighted the lack of design principles for drivetrain configurations capable of supporting practical tasks and safe human interaction. Two drivetrain prototypes were developed: the first employed a conventional design with a cascaded model-based controller, achieving human-walking-level agility in a single translational direction but exhibiting instability in other directions due to omniwheel-spherical wheel slip. To address these limitations, a second prototype was optimized through a design study that evaluated drivetrain variables affecting torque generation and contact stability, resulting in improved stability and agility across all tested translational directions. The final platform, named Personal Unique Rolling Experience (PURE), integrated a chair module capable of measuring torso dynamics and physical interactions. Hands-free control of PURE was enabled via impedance and admittance-based controllers, allowing

[1]The project repository is available at: `https://github.com/pedrophj/ballbot_LMI/tree/main`, containing the experimental results and videos.



(a) (b) (c)

FIGURE 2.4 – Overview of some Ballbot control approaches: (a) Twente University's ball-balancing robot employing LQR and SISO loop-shaping controllers (BLONK, 2014); (b) Kugle V1 utilizing quaternion-based dynamics and sliding mode control (JESPERSEN, 2019); and (c) a low-cost Arduino-based platform featuring a 3D-printed structure and waypoint navigation (JESUS, 2021).

the rider to navigate through torso motions. Moreover, an advanced control scheme incorporating offline gain personalization and online interaction compensation ensured safe and stable operation under parameter uncertainties and external interactions, enabling shared-motion tasks such as idle-keeping and speed-limiting while providing intuitive physical feedback to the rider.

(a) (b) (c)

FIGURE 2.5 – PURE human-riding Ballbot: (a) initial three-omniwheel drivetrain design, (b) optimized drivetrain configuration, and (c) final platform with integrated chair module for hands-free torso-motion control (XIAO, 2022).

2.2 Dynamic Model

Recent Ballbot designs employ the three-omniwheel drive mechanism, which can be modeled as a system consisting of a body, three omnidirectional wheels, and a ball. A total of five degrees of freedom (DOF) is attributed to the full system: three for the robot's attitude (roll ϕ_R, pitch θ_R, and yaw ψ_R) and two for the ball's position (x_B, y_B) (BLONK, 2014).

To obtain a simple yet useful dynamic model, the system is decomposed into three orthogonal planes (X–Z, Y–Z, and X–Y). This simplification results in three uncoupled planar models: the identical X–Z and Y–Z planes are each described by two degrees of freedom, namely (x_B, θ_R) and (y_B, ϕ_R), respectively, with the Y–Z model derived analogously to the X–Z model. The X–Y plane is modeled with a single degree of freedom corresponding to the robot's yaw angle ψ_R. This approach significantly reduces the complexity of the full 3D model while still capturing the fundamental dynamics. An illustration of the planar representations is shown in Figure 2.6. The following sections detail the derivation of the X–Z and X–Y models based on Fankhauser e Gwerder (2010).

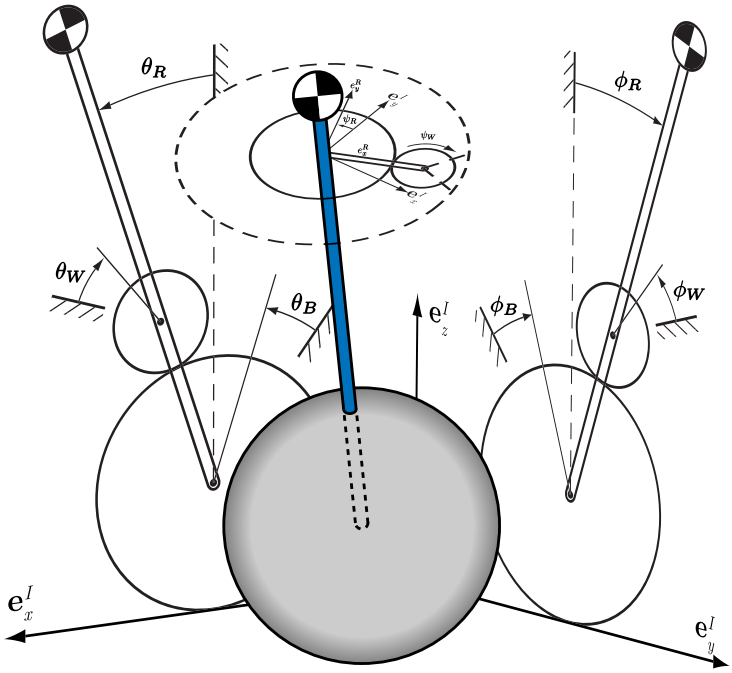


FIGURE 2.6 – Representation of the three planar models corresponding to the full 3D system. Adapted from Fankhauser e Gwerder (2010).

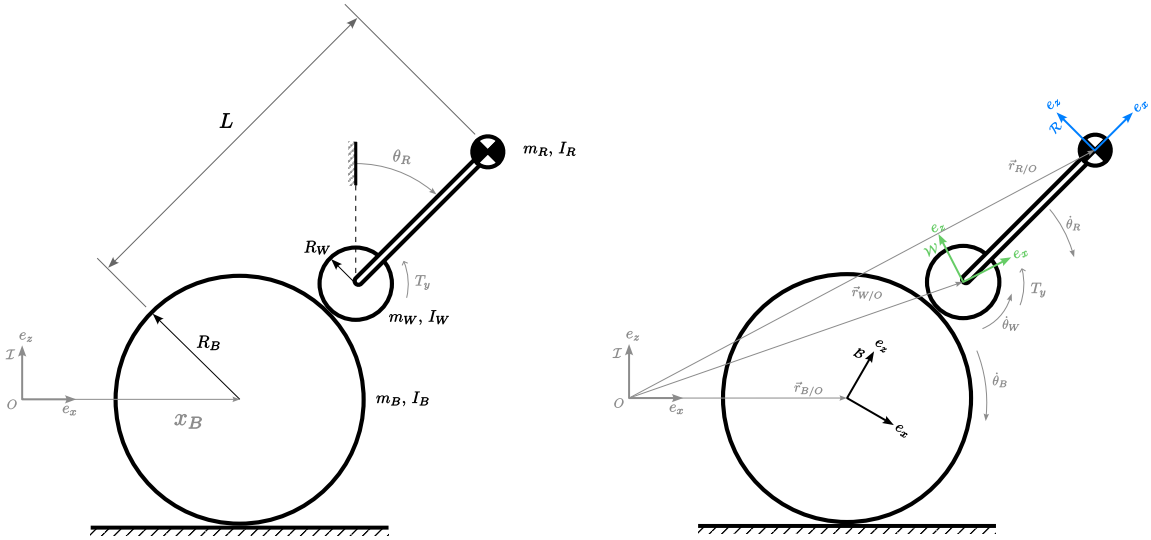
2.2.1 X–Z Model

The X–Z planar model consists of three rigid bodies: the ball, the robot body, and a virtual wheel. The virtual wheel represents the three omnidirectional wheels, and its placement in the planar model is a simplification of the actual drive mechanism. Conse-

quently, conversion relations are required to relate the planar model dynamics to those of the physical robot. The following modeling assumptions are adopted:

- **Rigid bodies:** The ball, the robot's body, and wheel are assumed rigid. No deformation occurs at the contact points between the ball and the ground, or between the ball and the wheel. All contacts are therefore treated as point contacts.
- **No slip:** Perfect rolling without slipping is assumed at both the ball–ground and ball–wheel contacts.
- **Planar constraint:** The X–Z plane is considered flat and leveled. Consequently, the ball does not experience motion along the Z direction.

Figure 2.7a illustrates the planar representation of the system in the X–Z plane, depicting the ball position x_B, the robot pitch attitude angle θ_R, the actuation torque applied by the virtual wheel T_y, and the physical parameters of all bodies. The corresponding parameter values are summarized in Table 2.1.



(a) Physical parameters of the model.

(b) Positions and velocities for all bodies.

FIGURE 2.7 – Planar representation of the system in the X–Z plane.

In order to derive a mathematical model of the system, the Euler–Lagrange equation is employed:

$$\frac{d}{dt}\left(\frac{\partial \mathcal{L}}{\partial \dot{q}_i}\right) - \frac{\partial \mathcal{L}}{\partial q_i} = Q_i, \tag{2.1}$$

In Equation (2.1), q_i denotes the i-th generalized coordinate, $\mathcal{L}$ is the Lagrangian function, and Q_i represents the generalized non-conservative force associated with the

TABLE 2.1 – System parameters for the X–Z planar model.

Parameter	Symbol	Value	Unit
Ball mass	m_B	0.580	kg
Robot mass	m_R	2.18	kg
Wheel mass	m_W	0.039	kg
Ball moment of inertia	I_B	5.56×10^{-3}	kg·m^2
Robot body moment of inertia	I_R	1.93×10^{-2}	kg·m^2
Wheel moment of inertia	I_W	1.96×10^{-5}	kg·m^2
Ball radius	R_B	0.12	m
Wheel radius	R_W	0.035	m
Distance from ball center to robot C.G.	L	0.194	m

i-th coordinate. The equations of motion for the system are derived through the following sequence of steps:

1. Identify the generalized coordinates q_i that minimally and uniquely describe the configuration of the system.
2. Express the position and velocity of each body in terms of the chosen generalized coordinates and their time derivatives.
3. Determine the total kinetic energy T and potential energy V of the system.
4. Formulate the Lagrangian function as $\mathcal{L} = T - V$.
5. Compute the partial derivatives $\dfrac{\partial \mathcal{L}}{\partial q_i}$ and $\dfrac{\partial \mathcal{L}}{\partial \dot{q}_i}$.
6. Evaluate the time derivative $\dfrac{d}{dt}\left(\dfrac{\partial \mathcal{L}}{\partial \dot{q}_i}\right)$.
7. Determine the generalized non-conservative forces Q_i from the virtual work of external inputs, dissipative effects, or other non-potential forces.
8. Substitute all terms into Equation (2.1) to obtain the equations of motion of the system.
9. Arrange the resulting equations into the standard matrix form

$$\boldsymbol{M}(\boldsymbol{q})\,\ddot{\boldsymbol{q}} + \boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}}) + \boldsymbol{G}(\boldsymbol{q}) = \boldsymbol{Q} \tag{2.2}$$

where $\boldsymbol{M}(\boldsymbol{q})$ is the mass/inertia matrix, $\boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})$ contains Coriolis and centrifugal terms, and $\boldsymbol{G}(\boldsymbol{q})$ represents the generalized forces derived from potential energy.

2.2.1.1 Generalized Coordinates

In the X–Z planar model, the robot's body is fully described by a single generalized coordinate, the pitch angle θ_R, due to its rigid connection to the virtual wheel and the kinematic linkage between the wheel and the ball. Additionally, under the no-slip assumption, the ball is fully described by its position x_B along the X–Z plane. Therefore, the generalized coordinate vector:

$$\mathbf{q}_{xz} = \begin{bmatrix} x_B \\ \theta_R \end{bmatrix} \tag{2.3}$$

completely characterizes the X–Z planar model.

2.2.1.2 Kinetic and Potential Energies

The kinetic and potential energy of each rigid body in the system are determined from their translational and angular velocities, which are obtained by differentiating the positions and orientations of the bodies with respect to time. As illustrated in the X–Z planar model diagram in Fig. 2.7b, the position vectors of the ball ${}^I\boldsymbol{r}_B$, the robot body ${}^I\boldsymbol{r}_R$, and the virtual wheel ${}^I\boldsymbol{r}_W$, expressed in the inertial reference frame $\mathcal{I}$, are:

$${}^I\boldsymbol{r}_B = \begin{bmatrix} x_B \\ 0 \\ 0 \end{bmatrix} \tag{2.4}$$

$${}^I\boldsymbol{r}_R = \begin{bmatrix} x_B + L\sin\theta_R \\ 0 \\ L\cos\theta_R \end{bmatrix} \tag{2.5}$$

$${}^I\boldsymbol{r}_W = \begin{bmatrix} x_B + (R_B + R_W)\sin\theta_R \\ 0 \\ (R_B + R_W)\cos\theta_R \end{bmatrix} \tag{2.6}$$

Differentiating these expressions with respect to time yields the translational velocities of the ball, the robot, and the virtual wheel, denoted by ${}^I\boldsymbol{v}_B$, ${}^I\boldsymbol{v}_R$, and ${}^I\boldsymbol{v}_W$, respectively, expressed in the inertial reference frame.

$$ {}^{I}\boldsymbol{v}_B = {}^{I}\dot{\boldsymbol{r}}_B = \begin{bmatrix} \dot{x}_B \\ 0 \\ 0 \end{bmatrix} \tag{2.7} $$

$$ {}^{I}\boldsymbol{v}_R = {}^{I}\dot{\boldsymbol{r}}_R = \begin{bmatrix} \dot{x}_B + \dot{\theta}_R L \cos\theta_R \\ 0 \\ -\dot{\theta}_R L \sin\theta_R \end{bmatrix} \tag{2.8} $$

$$ {}^{I}\boldsymbol{v}_W = {}^{I}\dot{\boldsymbol{r}}_W = \begin{bmatrix} \dot{x}_B + \dot{\theta}_R (R_B + R_W) \cos\theta_R \\ 0 \\ -\dot{\theta}_R (R_B + R_W) \sin\theta_R \end{bmatrix} \tag{2.9} $$

By applying the no-slip condition, the angular velocities of the robot ${}^{I}\boldsymbol{\Omega}_R$, the ball ${}^{I}\boldsymbol{\Omega}_B$, and the virtual wheel ${}^{I}\boldsymbol{\Omega}_W$, expressed in the inertial reference frame, can be written as functions of the generalized coordinates.

$$ {}^{I}\boldsymbol{\Omega}_B = \begin{bmatrix} 0 \\ \frac{\dot{x}_B}{R_B} \\ 0 \end{bmatrix} \tag{2.10} $$

$$ {}^{I}\boldsymbol{\Omega}_R = \begin{bmatrix} 0 \\ \dot{\theta}_R \\ 0 \end{bmatrix} \tag{2.11} $$

$$ {}^{I}\boldsymbol{\Omega}_W = \begin{bmatrix} 0 \\ -\frac{R_B}{R_W}\left(\frac{\dot{x}_B}{R_B} - \dot{\theta}_R\right) + \dot{\theta}_R \\ 0 \end{bmatrix} \tag{2.12} $$

Once the positions and velocities of all bodies in the system are defined, their kinetic and potential energies are computed. The kinetic energy of the ball T_B is determined from Equations (2.7) and (2.10):

$$ \begin{aligned} T_B &= \frac{1}{2} m_B \boldsymbol{v}_B^T \cdot \boldsymbol{v}_B + \frac{1}{2} I_B \, {}^{I}\boldsymbol{\Omega}_B{}^{T} \cdot {}^{I}\boldsymbol{\Omega}_B \\ T_B &= \frac{1}{2} \dot{x}_B^2 \left(m_B + \frac{I_B}{R_B^2} \right) \end{aligned} \tag{2.13} $$

The potential energy of the ball V_B, assuming motion on the flat X–Z plane and with the inertial frame origin set at the height of the ball's center of mass, is:

$$V_B = 0 \tag{2.14}$$

Based on the linear and angular velocities defined in Equations (2.8) and (2.11), the kinetic energy of the robot body, denoted by T_R, can be expressed as:

$$\begin{aligned} T_R &= \frac{1}{2} m_R \boldsymbol{v}_R^T \cdot \boldsymbol{v}_R + \frac{1}{2} I_R \, {}^I\boldsymbol{\Omega}_R^T \cdot {}^I\boldsymbol{\Omega}_R^T \\ T_R &= \frac{1}{2} \left(m_R \left(\dot{x}_B^2 + 2\dot{x}_B \dot{\theta}_R L \cos\theta_R + \dot{\theta}_R^2 L^2 \right) + \dot{\theta}_R^2 I_R \right) \end{aligned} \tag{2.15}$$

The potential energy of the robot's body V_R is:

$$V_R = m_R g L \cos\theta_R \tag{2.16}$$

Using the expressions for the linear and angular velocities derived in Equations (2.9) and (2.12), the kinetic energy of the virtual wheel, T_W, is obtained as:

$$\begin{aligned} T_W &= \frac{1}{2} m_W \boldsymbol{v}_W^T \cdot \boldsymbol{v}_W + \frac{1}{2} I_W \, {}^I\boldsymbol{\Omega}_W{}^T \cdot {}^I\boldsymbol{\Omega}_W \\ T_W &= \frac{1}{2} m_W \left(\dot{x}_B^2 + \dot{\theta}_R^2 (R_B + R_W)^2 + 2\dot{x}_B \dot{\theta}_R (R_B + R_W) \cos\theta_R \right) \\ &\quad + \frac{1}{2} I_W \left(\frac{\dot{x}_B^2}{R_W^2} + \dot{\theta}_R^2 \left(\frac{R_B^2}{R_W^2} + \frac{2R_B}{R_W} + 1 \right) - 2\dot{x}_B \dot{\theta}_R \left(\frac{R_B}{R_W^2} + \frac{1}{R_W} \right) \right) \end{aligned} \tag{2.17}$$

The potential energy of the virtual wheel V_W is:

$$V_W = m_W g (R_B + R_W) \cos\theta_R \tag{2.18}$$

The total kinetic energy T and potential energy V of the system are obtained by summing the contributions from all bodies:

$$T = T_B + T_R + T_W \tag{2.19}$$

$$V = V_B + V_R + V_W \tag{2.20}$$

2.2.1.3 Non-Conservative Generalized Forces

To account for the non-conservative external forces and moments acting on the system, each must be expressed in terms of the generalized coordinates as generalized forces. In this system, the only external inputs considered are the torque applied to the wheel by the motors and the corresponding counter-torque that acts on the robot's body.

Following the convention shown in Figure 2.7, the torques acting on the virtual wheel, $\boldsymbol{\tau}_W$, and on the robot's body, $\boldsymbol{\tau}_R$, are defined as:

$$\boldsymbol{\tau}_R = \begin{bmatrix} 0 \\ T_y \\ 0 \end{bmatrix}, \quad \boldsymbol{\tau}_W = \begin{bmatrix} 0 \\ -T_y \\ 0 \end{bmatrix} \tag{2.21}$$

The corresponding generalized forces acting on the virtual wheel, $\boldsymbol{Q}_W$, and on the robot's body, $\boldsymbol{Q}_R$, are given by:

$$\boldsymbol{Q}_R = {}^I\boldsymbol{J}_R^T \cdot \boldsymbol{\tau}_R \tag{2.22}$$

$$\boldsymbol{Q}_W = {}^I\boldsymbol{J}_W^T \cdot \boldsymbol{\tau}_W \tag{2.23}$$

where ${}^I\boldsymbol{J}_W$ and ${}^I\boldsymbol{J}_R$ are the rotational geometric Jacobians that map the angular velocities of the virtual wheel and the robot's body, expressed in generalized coordinates, to the inertial reference frame. These Jacobians are obtained from:

$${}^I\boldsymbol{\Omega}_R = \begin{bmatrix} 0 \\ \dot{\theta}_R \\ 0 \end{bmatrix} = \underbrace{\begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}}_{{}^I\boldsymbol{J}_R} \cdot \begin{bmatrix} \dot{x}_B \\ \dot{\theta}_R \end{bmatrix} \tag{2.24}$$

$${}^I\boldsymbol{\Omega}_W = \begin{bmatrix} 0 \\ -\frac{R_B}{R_W}\left(\frac{\dot{x}_B}{R_B} - \dot{\theta}_R\right) + \dot{\theta}_R \\ 0 \end{bmatrix} = \underbrace{\begin{bmatrix} 0 & 0 \\ \frac{-1}{R_W} & \frac{R_B}{R_W} + 1 \\ 0 & 0 \end{bmatrix}}_{{}^I\boldsymbol{J}_W} \cdot \begin{bmatrix} \dot{x}_B \\ \dot{\theta}_R \end{bmatrix} \tag{2.25}$$

Finally, the total generalized forces are obtained as:

$$\boldsymbol{Q} = \boldsymbol{Q}_R + \boldsymbol{Q}_W = \begin{bmatrix} \frac{T_y}{R_W} \\ -T_y \frac{R_B}{R_W} \end{bmatrix} \tag{2.26}$$

2.2.1.4 Equations of Motion

By substituting Equations (2.19), (2.20), and (2.26) into the Euler–Lagrange formulation described in steps 4 through 9, the equations of motion for the X–Z planar model are obtained. These can be expressed in matrix form as

$$\boldsymbol{M} = \begin{bmatrix} m_T + I_T & \gamma \cos\theta_R - \eta\, I_W \\ \gamma \cos\theta_R - \eta\, I_W & \lambda \end{bmatrix} \tag{2.27}$$

$$\boldsymbol{C} = \begin{bmatrix} -\gamma \dot{\theta}_R^2 \sin\theta_R \\ 0 \end{bmatrix} \tag{2.28}$$

$$\boldsymbol{G} = \begin{bmatrix} 0 \\ -g\gamma \sin\theta_R \end{bmatrix} \tag{2.29}$$

where

$$m_T = m_B + m_W + m_R \tag{2.30}$$

$$I_T = \frac{I_W}{R_W^2} + \frac{I_B}{R_B^2} \tag{2.31}$$

$$\gamma = m_W \left(R_B + R_W\right) + m_R L \tag{2.32}$$

$$\eta = \frac{R_B}{R_W^2} + \frac{1}{R_W} \tag{2.33}$$

$$\lambda = m_W \left(R_B + R_W\right)^2 + m_R L^2 + I_R + I_W \left(\frac{R_B^2}{R_W^2} + 2\frac{R_B}{R_W} + 1\right) \tag{2.34}$$

Finally, solving Equation (2.2) for $\ddot{\boldsymbol{q}}_{xz}$ yields the following nonlinear second-order system of differential equations:

$$\begin{cases} \ddot{x}_B = \dfrac{\lambda\, T_y + \gamma\, R_B\, T_y\, \cos\theta_R - \eta\, I_W\, R_B\, T_y + \gamma\, \lambda\, R_W\, \dot{\theta}_R^2\, \sin\theta_R}{\sigma} \\ \qquad + \dfrac{-\gamma^2\, g\, R_W\, \cos\theta_R\, \sin\theta_R + \gamma\, \eta\, g\, I_W\, R_W\, \sin\theta_R}{\sigma} \\ \ddot{\theta}_R = \dfrac{-I_T\, R_B\, T_y + \eta\, I_W\, T_y - T_y\, R_B\, m_T - \gamma\, T_y\, \cos\theta_R - \gamma^2\, R_W\, \dot{\theta}_R^2\, \cos\theta_R\, \sin\theta_R}{\sigma} \\ \qquad + \dfrac{\gamma\, g\, I_T\, R_W\, \sin\theta_R + \gamma\, g\, m_T\, R_W\, \sin\theta_R + \eta\, \gamma\, I_W\, R_W\, \dot{\theta}_R^2\, \sin\theta_R}{\sigma} \end{cases} \tag{2.35}$$

with

$$\sigma = R_W \left(-\eta^2\, {I_W}^2 + 2\eta\, \gamma\, I_W\, \cos\theta_R - \gamma^2 \cos{\theta_R}^2 + \lambda\, I_T + \lambda\, m_T\right) \tag{2.36}$$

2.2.1.5 Linearized State-Space Model

The nonlinear second-order system of differential equations derived in the previous section can be reformulated as a first-order system using a state-space representation. The state vector is chosen as

$$\boldsymbol{x} = \begin{bmatrix} x_B \\ \theta_R \\ \dot{x}_B \\ \dot{\theta}_R \end{bmatrix} \tag{2.37}$$

where the first two components correspond to the generalized coordinates, and the last two to their respective velocities. With this definition, the system dynamics can be expressed in the general nonlinear form

$$\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}, u) \tag{2.38}$$

where u is the control input, the torque applied to the wheel, and $\boldsymbol{f}(\cdot)$ represents the nonlinear mapping defined by the equations of motion.

For the purpose of linear control design, the nonlinear equations are linearized about an equilibrium point that satisfies $\dot{\boldsymbol{x}} = 0$. From Equation (2.35), it can be readily verified that the upright balance operating point corresponds to such an equilibrium, defined as

$$\theta_R = 0, \quad \dot{\theta}_R = 0, \quad \dot{x}_B = 0, \quad T = 0 \tag{2.39}$$

The linearized model is then obtained by computing the Jacobian of $\boldsymbol{f}(\boldsymbol{x}, u)$ with

respect to the state vector and the control input, evaluated at the equilibrium point, resulting in the linear time-invariant state-space representation:

$$\begin{aligned} \dot{\boldsymbol{x}} &= \boldsymbol{A}\boldsymbol{x}(t) + \boldsymbol{B}\boldsymbol{u}(t) \\ \boldsymbol{y} &= \boldsymbol{C}\boldsymbol{x}(t) + \boldsymbol{D}\boldsymbol{u}(t) \end{aligned} \tag{2.40}$$

where $\boldsymbol{A}$ is the state matrix, $\boldsymbol{B}$ the input matrix, $\boldsymbol{C}$ the output matrix, and $\boldsymbol{D}$ the feedforward matrix.

Based on the physical parameters in Table 2.1 and assuming full state measurement, the resulting system matrices for the linearized state-space model are obtained as:

$$\begin{aligned} \boldsymbol{A} &= \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & -12.23 & 0 & 0 \\ 0 & 91.78 & 0 & 0 \end{bmatrix}, \quad \boldsymbol{B} = \begin{bmatrix} 0 \\ 0 \\ 29.95 \\ -157.81 \end{bmatrix}, \\ \boldsymbol{C} &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \qquad \boldsymbol{D} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \end{aligned} \tag{2.41}$$

The analyses of the controllability $\mathcal{C}$ and observability $\mathcal{O}$ matrices show that both have full rank, indicating that the system is fully controllable and observable. The eigenvalues of the state matrix $\boldsymbol{A}$ are given by

$$\lambda = 0;\ 0;\ 9.58;\ -9.58 \tag{2.42}$$

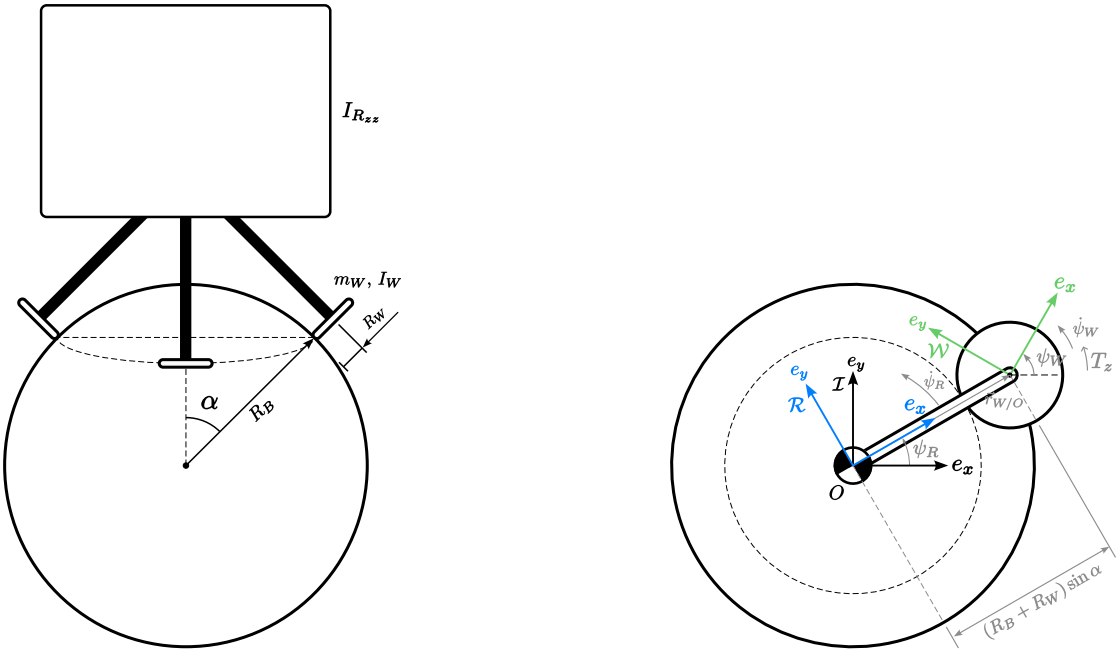
where the presence of an eigenvalue with positive real part confirms that the upright equilibrium is unstable.

2.2.2 X–Y Model

Analogous to the X–Z model, the X–Y plane consists of the same three rigid bodies. The *Rigid Body* and *No Slip* assumptions used for the X–Z model are also applied. Additionally, it is assumed that the torque magnitude applied by the wheel to the ball is limited and smaller than the frictional torque between the ball and the ground. This ensures that the motion in the X–Y plane occurs exclusively through the rotation of the robot body about the ball.

Figure 2.8 shows the planar representation of the system in the X–Y plane, depicting

the wheel mounting angle, body positions, and angular velocities, while the corresponding physical parameters are listed in Table 2.2.



(a) Representation of a particular wheel mounting angle α and its position on the ball.

(b) Illustration of the positions and angular velocities for the X–Y plane.

FIGURE 2.8 – Planar representation of the system in the X–Y plane.

TABLE 2.2 – System parameters for the X–Y planar model.

Parameter	Symbol	Value	Unit
Wheel mass	m_W	0.039	kg
Robot body moment of inertia	$I_{R_{zz}}$	1.14×10^{-2}	kg·m^2
Wheel moment of inertia	I_W	1.96×10^{-5}	kg·m^2
Ball radius	R_B	0.12	m
Wheel radius	R_W	0.035	m
Wheel mounting angle	α	45	deg

Following the same procedure adopted for the X–Z plane, the equations of motion for the X–Y model are obtained.

2.2.2.1 Generalized Coordinates

As the rotation of the robot around the ball is the only motion considered in the X–Y plane, and the *No Slip* and rigid coupling between the wheel and body assumptions are maintained, the robot's yaw angle ψ_R fully describes the system configuration. Therefore, the generalized coordinate for the X–Y plane is defined as:

$$q_{xy} = \psi_R \tag{2.43}$$

2.2.2.2 Kinetic and Potential Energies

For the X–Y plane, only the kinetic energies associated with the robot and the virtual wheel are considered. In order to determine these energies, the position and velocity of each body are first defined. Since the robot's position remains fixed in the X–Y plane, its translational motion is neglected. The position of the virtual wheel ${}^{I}\boldsymbol{r}_{W_{xy}}$, expressed in the inertial reference frame $\mathcal{I}$, is given by:

$${}^{I}\boldsymbol{r}_{W_{xy}} = \begin{bmatrix} (R_B + R_W)\cos\psi_R \sin\alpha \\ (R_B + R_W)\sin\psi_R \sin\alpha \\ 0 \end{bmatrix} \tag{2.44}$$

The velocity of the virtual wheel ${}^{I}\boldsymbol{v}_{W_{xy}}$ is obtained by differentiating Equation (2.44) with respect to time:

$${}^{I}\boldsymbol{v}_{W_{xy}} = \begin{bmatrix} -\dot{\psi}_R\,(R_B + R_W)\sin\psi_R \sin\alpha \\ \dot{\psi}_R\,(R_B + R_W)\cos\psi_R \sin\alpha \\ 0 \end{bmatrix} \tag{2.45}$$

Considering the rigid coupling between the virtual wheel and the robot body, the angular velocities of the robot ${}^{I}\boldsymbol{\Omega}_{R_{xy}}$ and the virtual wheel ${}^{I}\boldsymbol{\Omega}_{W_{xy}}$ are defined as:

$${}^{I}\boldsymbol{\Omega}_{\boldsymbol{R_{xy}}} = \begin{bmatrix} 0 \\ 0 \\ \dot{\psi}_R \end{bmatrix} \tag{2.46}$$

$${}^{I}\boldsymbol{\Omega}_{\boldsymbol{W_{xy}}} = \begin{bmatrix} 0 \\ 0 \\ \frac{R_B}{R_W}\dot{\psi}_R \sin\alpha \end{bmatrix} \tag{2.47}$$

The kinetic energies associated with the robot body $T_{R_{xy}}$ and the virtual wheel $T_{W_{xy}}$ are expressed as:

$$T_{R_{xy}} = \frac{1}{2} I_{R_{zz}} {}^{I}\boldsymbol{\Omega}_{R_{xy}}^{T} \cdot {}^{I}\boldsymbol{\Omega}_{R_{xy}} = \frac{1}{2} I_{R_{zz}} \dot{\psi}_R^2 \tag{2.48}$$

$$T_{W_{xy}} = \frac{1}{2} m_W \boldsymbol{v}_{W_{xy}}^{T} \cdot \boldsymbol{v}_{W_{xy}} + \frac{1}{2} I_W {}^{I}\boldsymbol{\Omega}_{W_{xy}}^{T} \cdot {}^{I}\boldsymbol{\Omega}_{W_{xy}}$$

$$T_{W_{xy}} = \frac{1}{2} \left(m_W \left(R_B + R_W \right)^2 + I_W \left(\frac{R_B}{R_W} \right)^2 \right) \dot{\psi}_R^2 \sin^2 \alpha \tag{2.49}$$

The total kinetic energy T_{xy} is then given by:

$$T_{xy} = T_{R_{xy}} + T_{W_{xy}} \tag{2.50}$$

2.2.2.3 Non-Conservative Generalized Forces

Similar to the X–Z plane, the only forces acting in the X–Y plane are the torque applied by the motor to the wheel $\boldsymbol{\tau}_{W_{xy}}$ and the corresponding counter-torque acting on the robot's body $\boldsymbol{\tau}_{R_{xy}}$, which are defined as:

$$\boldsymbol{\tau}_{R_{xy}} = \begin{bmatrix} 0 \\ 0 \\ T_z \end{bmatrix}, \quad \boldsymbol{\tau}_{W_{xy}} = \begin{bmatrix} 0 \\ 0 \\ -T_z \end{bmatrix} \tag{2.51}$$

The resulting generalized force Q_{xy} is expressed as:

$$Q_{xy} = T_z \left(1 - \frac{R_B}{R_W} \sin \alpha \right) \tag{2.52}$$

2.2.2.4 State-Space Model

The equation of motion for the X–Y plane is derived employing the Euler-Lagrange formulation and is expressed as:

$$\ddot{\psi}_R = \frac{T_Z \left(1 - \frac{R_B \sin(\alpha)}{R_W} \right)}{I_{R_{zz}} + \sin \alpha^2 \left(m_W \left(R_B + R_W \right)^2 + I_W \frac{{R_B}^2}{{R_W}^2} \right)} \tag{2.53}$$

In contrast to the model obtained for the X–Z plane, the X–Y plane yields a linear second-order equation. Defining the state vector as:

$$\boldsymbol{x}_{xy} = \begin{bmatrix} \psi_R \\ \dot{\psi}_R \end{bmatrix} \tag{2.54}$$

a state-space representation in the form of Equation (2.40) is obtained, with matrices $\boldsymbol{A}$, $\boldsymbol{B}$, $\boldsymbol{C}$, and $\boldsymbol{D}$ given by:

$$\boldsymbol{A} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}, \qquad \boldsymbol{B} = \begin{bmatrix} 0 \\ -118.89 \end{bmatrix}, \qquad \boldsymbol{C} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \qquad \boldsymbol{D} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \tag{2.55}$$

The analysis of the controllability $\mathcal{C}$ and observability $\mathcal{O}$ matrices yields results similar to those of the X–Z plane, with both matrices exhibiting full rank, confirming that the system is fully controllable and observable. The eigenvalues of the state matrix $\boldsymbol{A}$ are located at the origin of the complex plane, reflecting marginal stability, which is expected given the purely rotational nature of the X–Y dynamics, as all dissipative and restoring effects have been neglected.

2.2.2.5 Planar Model Conversions

As the positions of the virtual wheels in the planar models differ from those of the actual wheels on the robot, a conversion procedure is required to map the torques T_x, T_y, and T_z obtained from the planar representations to the torques applied by each motor driving the robot's wheels. The adopted formulation, illustrated in Figure 2.9, follows the method proposed in Fankhauser e Gwerder (2010). The figure also shows the angle β, defined as the angle between the robot's x axis and the axis of its first wheel. For the present work, this angle is assumed to be zero, meaning that the axis of the robot's first wheel is aligned with the robot's x axis.

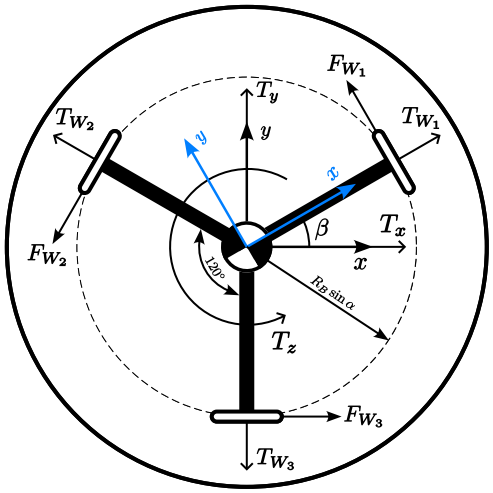


FIGURE 2.9 – Illustration of the torque and force vectors for both the virtual and real wheel configurations. Adapted from Fankhauser e Gwerder (2010).

The positions of the torque application points associated with the virtual wheels are defined as:

$$\boldsymbol{r}_{W_{xz}} = R_B \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \tag{2.56}$$

$$\boldsymbol{r}_{W_{yz}} = R_B \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \tag{2.57}$$

$$\boldsymbol{r}_{W_{xy}} = R_B \begin{bmatrix} \cos\beta \sin\alpha \\ \sin\beta \sin\alpha \\ 0 \end{bmatrix} \tag{2.58}$$

Analogously, the torque application points of the actual robot wheels are defined as:

$$\boldsymbol{r}_{W_1} = R_B \begin{bmatrix} \cos\beta \sin\alpha \\ \sin\beta \sin\alpha \\ \cos\alpha \end{bmatrix} \tag{2.59}$$

$$\boldsymbol{r}_{W_2} = R_B \begin{bmatrix} \cos\left(\beta + \frac{2}{3}\pi\right) \sin\alpha \\ \sin\left(\beta + \frac{2}{3}\pi\right) \sin\alpha \\ \cos\alpha \end{bmatrix} \tag{2.60}$$

$$\boldsymbol{r}_{W_3} = R_B \begin{bmatrix} \cos\left(\beta - \frac{2}{3}\pi\right) \sin\alpha \\ \sin\left(\beta - \frac{2}{3}\pi\right) \sin\alpha \\ \cos\alpha \end{bmatrix} \tag{2.61}$$

The forces applied by the virtual wheels are expressed as:

$$\boldsymbol{F}_{W_{xz}} = \frac{T_y}{R_W} \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \tag{2.62}$$

$$\boldsymbol{F}_{W_{yz}} = \frac{T_x}{R_W} \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \tag{2.63}$$

$$\boldsymbol{F}_{W_{xy}} = \frac{T_z}{R_W} \begin{bmatrix} -\sin\beta \\ \cos\beta \\ 0 \end{bmatrix} \tag{2.64}$$

Similarly, the forces applied by the actual wheels are given by:

$$\boldsymbol{F}_{W_1} = \frac{T_{W_1}}{R_W} \begin{bmatrix} -\sin\beta \\ \cos\beta \\ 0 \end{bmatrix} \tag{2.65}$$

$$\boldsymbol{F}_{W_2} = \frac{T_{W_2}}{R_W} \begin{bmatrix} -\sin\left(\beta + \frac{2}{3}\pi\right) \\ \cos\left(\beta + \frac{2}{3}\pi\right) \\ 0 \end{bmatrix} \tag{2.66}$$

$$\boldsymbol{F}_{W_3} = \frac{T_{W_3}}{R_W} \begin{bmatrix} -\sin\left(\beta - \frac{2}{3}\pi\right) \\ \cos\left(\beta - \frac{2}{3}\pi\right) \\ 0 \end{bmatrix} \tag{2.67}$$

The torques generated by the virtual wheels are calculated as:

$$\boldsymbol{T}_{W_{xz}} = \boldsymbol{r}_{W_{xz}} \times \boldsymbol{F}_{W_{xz}} \tag{2.68}$$

$$\boldsymbol{T}_{W_{yz}} = \boldsymbol{r}_{W_{yz}} \times \boldsymbol{F}_{W_{yz}} \tag{2.69}$$

$$\boldsymbol{T}_{W_{xy}} = \boldsymbol{r}_{W_{xy}} \times \boldsymbol{F}_{W_{xy}} \tag{2.70}$$

Likewise, the torques generated by the actual wheels are:

$$\boldsymbol{T}_{W_1} = \boldsymbol{r}_{W_1} \times \boldsymbol{F}_{W_1} \tag{2.71}$$

$$\boldsymbol{T}_{W_2} = \boldsymbol{r}_{W_2} \times \boldsymbol{F}_{W_2} \tag{2.72}$$

$$\boldsymbol{T}_{W_3} = \boldsymbol{r}_{W_3} \times \boldsymbol{F}_{W_3} \tag{2.73}$$

By imposing that the total torque generated by the virtual wheels equals that produced

by the real wheels, the following relation is established:

$$\boldsymbol{T}_{W_{xz}} + \boldsymbol{T}_{W_{yz}} + \boldsymbol{T}_{W_{xy}} = \boldsymbol{T}_{W_1} + \boldsymbol{T}_{W_2} + \boldsymbol{T}_{W_3} \tag{2.74}$$

Solving Equation (2.74) for the real torques yields:

$$T_{W_1} = \frac{1}{3}\left(T_z + \frac{2}{\cos\alpha}\left(T_x\cos\beta - T_y\sin\beta\right)\right) \tag{2.75}$$

$$T_{W_2} = \frac{1}{3}\left(T_z + \frac{1}{\cos\alpha}\left(\sin\beta\left(-\sqrt{3}\,T_x + T_y\right) - \cos\beta\left(T_x + \sqrt{3}\,T_y\right)\right)\right) \tag{2.76}$$

$$T_{W_3} = \frac{1}{3}\left(T_z + \frac{1}{\cos\alpha}\left(\sin\beta\left(\sqrt{3}\,T_x + T_y\right) + \cos\beta\left(-T_x + \sqrt{3}\,T_y\right)\right)\right) \tag{2.77}$$

2.3 Controllers

To stabilize and control the inherently unstable dynamics of the Ballbot, different control strategies have been proposed in the literature for each planar subsystem. The LQR controller is commonly employed for the X–Z and Y–Z planes, as it can be efficiently implemented once the linearized state-space model is obtained. For the stable X–Y plane, simpler PD controllers are typically used to regulate the robot's yaw attitude. The following sections presents the general formulation of these control strategies.

2.3.1 LQR Controller

The Linear Quadratic Regulator (LQR) is an optimal control approach used to determine the state-feedback control law that minimizes a given performance criterion for a linear system.

For a controllable and observable linear time-invariant (LTI) system described by:

$$\begin{aligned} \dot{\boldsymbol{x}} &= \boldsymbol{A}\boldsymbol{x}(t) + \boldsymbol{B}\boldsymbol{u}(t) \\ \boldsymbol{y} &= \boldsymbol{C}\boldsymbol{x}(t) + \boldsymbol{D}\boldsymbol{u}(t) \end{aligned} \tag{2.78}$$

where $\boldsymbol{x}(t)$ is the state vector and $\boldsymbol{u}(t)$ is the control input. The LQR controller determines the optimal control input that minimizes a quadratic cost function representing the trade-off between state deviations and control effort (KIRK, 1970):

$$J = \tfrac{1}{2}\boldsymbol{x}^T(t_f)\boldsymbol{H}\boldsymbol{x}(t_f) + \tfrac{1}{2}\int_{t_0}^{t_f}\left[\boldsymbol{x}^T(t)\boldsymbol{Q}\boldsymbol{x}(t) + \boldsymbol{u}^T(t)\boldsymbol{R}\boldsymbol{u}(t)\right]dt \tag{2.79}$$

where $\boldsymbol{Q}$ is a positive semi-definite weighting matrix that penalizes deviations of the

state variables from the equilibrium point, and $\boldsymbol{R}$ is a positive definite weighting matrix that penalizes the control effort. By appropriately selecting these matrices, the designer specifies the desired compromise between system performance and energy expenditure.

For the infinite-horizon case, the optimal control law $\boldsymbol{u}^*(t)$ that minimizes the performance index J and asymptotically drives the system states to the origin is given by

$$\boldsymbol{u}^*(t) = -\boldsymbol{K}\boldsymbol{x}(t) \tag{2.80}$$

where the optimal feedback gain matrix $\boldsymbol{K}$ is expressed as

$$\boldsymbol{K} = \boldsymbol{R}^{-1}\boldsymbol{B}^T\boldsymbol{P} \tag{2.81}$$

and $\boldsymbol{P}$ is the solution of the Algebraic Riccati Equation (ARE):

$$\boldsymbol{A}^T\boldsymbol{P} + \boldsymbol{P}\boldsymbol{A} - \boldsymbol{P}\boldsymbol{B}\boldsymbol{R}^{-1}\boldsymbol{B}^T\boldsymbol{P} + \boldsymbol{Q} = \boldsymbol{0} \tag{2.82}$$

The LQR framework provides an effective and systematic method for designing state-feedback controllers, ensuring an optimal trade-off between control performance and effort. Due to its robustness and straightforward implementation, the LQR is widely employed in stabilizing and regulating linearized dynamic systems.

2.3.2 PID Controller

Due to its simple implementation and intuitive tuning principles, the Proportional-Integral-Derivative (PID) controller is one of the most widely used feedback control strategies in engineering applications (ASTROM; MURRAY, 2008).

A block diagram of a general PID controller is shown in Figure 2.10. The control signal $u(t)$ for an ideal PID controller is given by:

$$u(t) = k_p e(t) + k_i \int_0^t e(\tau)\, d\tau + k_d \frac{de(t)}{dt} \tag{2.83}$$

where $e(t)$ is the control error, defined as the difference between the desired and measured outputs, and k_p, k_i, and k_d are the proportional, integral, and derivative gains, respectively. The proportional term provides an immediate correction based on the current error, the integral term eliminates steady-state error by accumulating past errors, and the derivative term predicts future errors by considering the rate of change of the error.

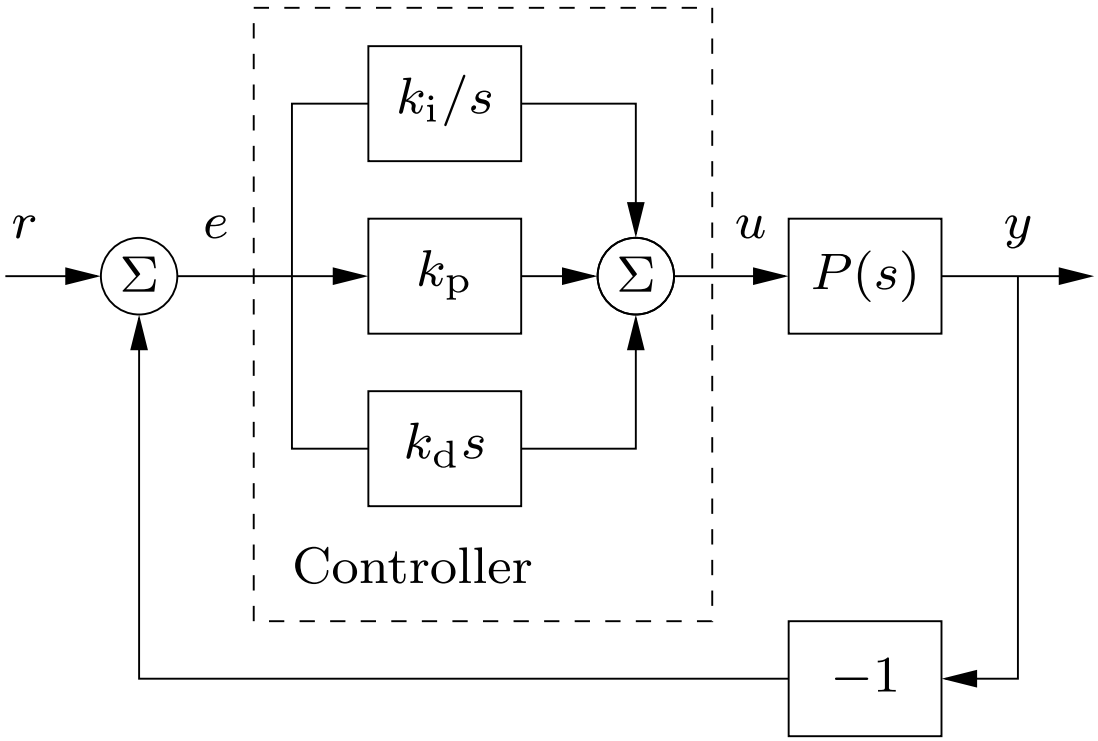


FIGURE 2.10 – Block diagram of a general PID controller for a process P. Source: Astrom e Murray (2008).

2.4 Ballbot Estimation

To obtain the full state of the Ballbot, certain quantities, such as its position, must be estimated. Two complementary approaches are employed for this purpose, and their principles of operation are presented below.

2.4.1 Detection and Pose Estimation using ArUco Markers

ArUco markers are square synthetic patterns widely used in computer vision for robust detection and identification. Each marker consists of a black outer border surrounding a unique binary code that encodes its identification number, as illustrated in Figure 2.11. Detection involves converting the input image to grayscale, applying adaptive thresholding to segment high-contrast regions, and extracting contours from the resulting binary image. Quadrilateral contours with appropriate geometric properties are selected as marker candidates and subsequently validated by decoding their internal binary patterns. The OpenCV library provides a complete implementation of this pipeline, including contour extraction, subpixel corner refinement, and marker identification (OPENCV, 2024).

Once a marker is detected, its spatial pose relative to the camera is estimated by solving a Perspective-n-Point (PnP) problem. The four detected marker corners define a set of two-dimensional image points, which are associated with known three-dimensional

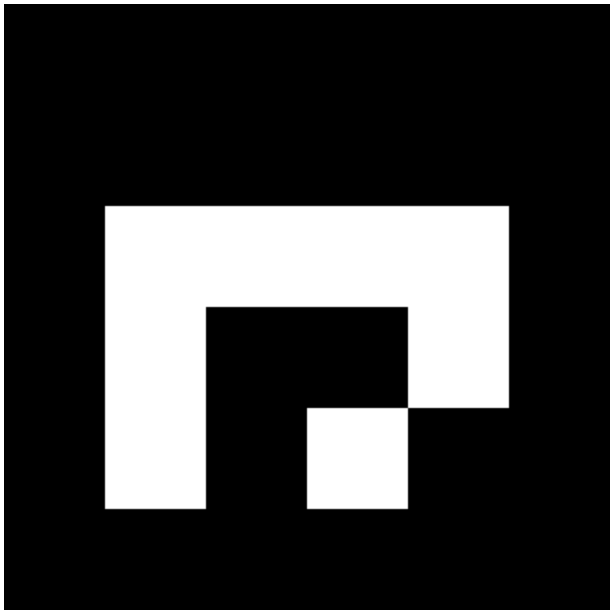


FIGURE 2.11 – Example of a 4×4 ArUco marker, showing the black border and internal binary pattern used for identification.

points defined in the marker reference frame based on its physical dimensions. Given these correspondences, the PnP formulation estimates the rigid-body transformation between the marker frame and the camera frame by minimizing the reprojection error under a pinhole camera model (OPENCV, 2024). The resulting solution yields a rotation vector and a translation vector that fully describe the marker's six-degree-of-freedom pose with respect to the camera coordinate system.

The accuracy of pose estimation is fundamentally constrained by the precision of the camera's intrinsic parameters, specifically the focal lengths, principal point coordinates, and lens distortion coefficients. In this study, these parameters are estimated via an offline calibration procedure using the Kalibr toolbox (FURGALE *et al.*, 2022). While Kalibr leverages the standard OpenCV pinhole model for camera intrinsic parameters calibration, it extends this framework by incorporating rolling shutter temporal parameters. This allows for a more rigorous modeling of image formation, particularly in mitigating the geometric distortions inherent in non-global shutter sensors during high-speed camera or scene motion (OTH *et al.*, 2013).

By combining ArUco marker detection with calibrated camera intrinsics and PnP-based pose estimation, the position and orientation of the Ballbot are estimated in real time relative to the camera frame. This vision-based pose information is then employed as an external measurement source within the overall state estimation and control framework.

2.4.2 Scan Matching for Pose Estimation

Scan matching is a technique used in mobile robotics to estimate a robot's pose by aligning consecutive laser scans. The goal is to compute the rigid-body transformation, translation and rotation, that best aligns the current scan with a reference scan or map, as illustrated in Figure 2.12, allowing incremental localization and odometry refinement even without external positioning systems.

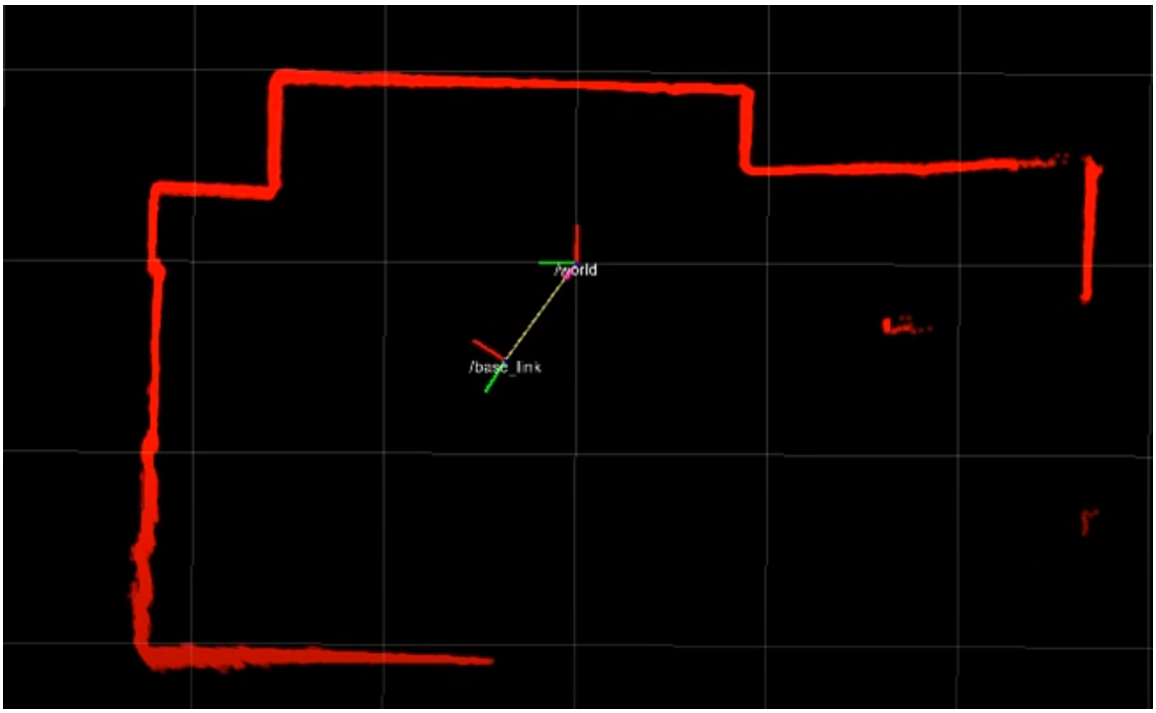


FIGURE 2.12 – Robot pose estimation via scan matching. The figure shows the world-fixed frame and the robot's pose relative to the world frame estimated by the scan-matching algorithm. Source: ROS WIKI (2024).

The `laser_scan_matcher` package (ROS WIKI, 2024) provides a real-time implementation of this technique within the Robot Operating System (ROS) environment. It employs a modified version of the Iterative Closest Point (ICP) algorithm to align consecutive scans by minimizing the distance between corresponding points in successive laser frames. In particular, the package adopts the point-to-line error metric, which enhances the convergence behavior and robustness of the alignment process (CENSI, 2008). By continuously aligning incoming scans with previous ones, the algorithm estimates the robot's incremental motion, which can then be integrated to obtain its pose. This approach is particularly useful in structured environments where reliable geometric features are available, providing accurate short-term localization that complements other state estimation methods.

3 Solution Proposal

This chapter presents the proposed solution for the design and control of the Ballbot system and it is organized into two main sections:

- The first section presents simulations of the 2D dynamic model, which evaluate the controller's performance under different initial conditions of the system. These simulations provide insight into stability and transient response characteristics, and serve as a preliminary step to assess the model's behavior before implementing the controller in a full 3D simulation and, subsequently, on the real physical Ballbot system.
- The second section describes the complete Ballbot system, consisting of its hardware, computational, mechanical, and software components. It presents the selected actuators, drivers, sensors, and power supply, along with the onboard processing units responsible for control and communication. The mechanical design of the structure and wheels is also detailed. Furthermore, this section discusses the software implementation, including communication protocols between embedded components and the interface with external networks through ROS.

Together, the following provide a comprehensive framework for implementing and validating the proposed control strategies on the physical Ballbot platform.

3.1 Simulations - 2D Model

The dynamic behavior of the Ballbot is investigated through simulations of the two-dimensional model implemented in *MATLAB Simulink*. For the nonlinear and inherently unstable X–Z configuration, an LQR controller is designed to stabilize the system, and its performance is evaluated under distinct initial conditions to assess robustness and transient response. For the X–Y model, a PD controller is applied and the performance of the controller is evaluated. The simulation files used for these analyses are available at the project's GitHub repository: `https://github.com/Intelligent-Machines-Lab/Ballbot_LMI_V2/tree/main/Simulations/2D`.

3.1.1 X–Z Model

A schematic representation of the *MATLAB Simulink* simulation is provided in Figure 3.1. The overall *Simulink* model, shown in Figure 3.1a, integrates the reference

input, LQR controller, Ballbot Dynamics, and output feedback. Figure 3.1b shows the detailed implementation of the Ballbot Dynamics block. The `f(u)` blocks implements the nonlinear second-order differential equations defined by Equation (2.35), which are then double-integrated from the initial conditions to obtain the velocities and positions.

Within the controller block, the LQR gains are computed using MATLAB's *lqr(A, B, Q, R)* function, based on the linearized state-space matrices $\boldsymbol{A}$ and $\boldsymbol{B}$ obtained in Equation (2.41). The diagonal weighting matrix $\boldsymbol{Q}$ is tuned to balance performance and stability, assigning a significantly higher weight to the pitch angle state to penalize deviations from the upright equilibrium. This ensures rapid angular correction, given the system's inherently unstable dynamics. In contrast, lower weights are assigned to position-related states to prevent overly aggressive control actions that could increase actuator effort and induce instability. The input weighting matrix $\boldsymbol{R}$ is defined as the identity matrix for simplicity. The resulting weighting matrices $\boldsymbol{Q}$ and $\boldsymbol{R}$, along with the computed LQR gain vector $\boldsymbol{K}$, are presented below:

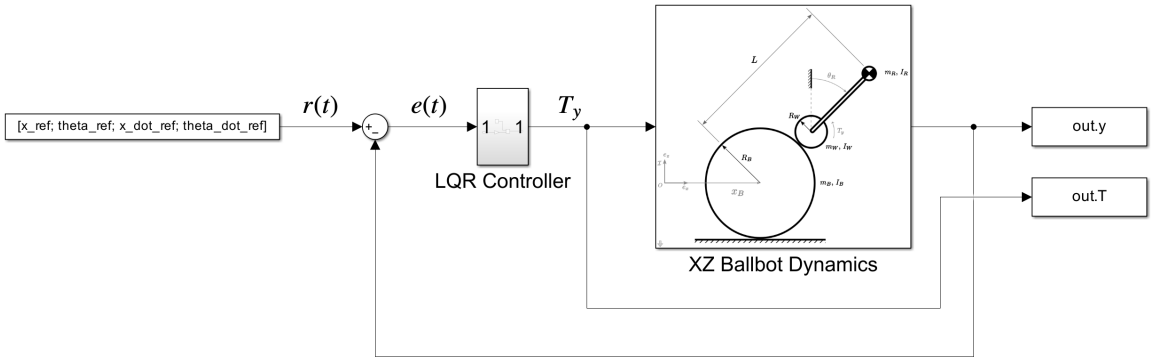
$$\boldsymbol{Q} = \begin{bmatrix} 0.1 & 0 & 0 & 0 \\ 0 & 10 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad \boldsymbol{R} = 1 \tag{3.1}$$

$$\boldsymbol{K} = \begin{bmatrix} -0.32 & -5.76 & -1.28 & -1.29 \end{bmatrix} \tag{3.2}$$

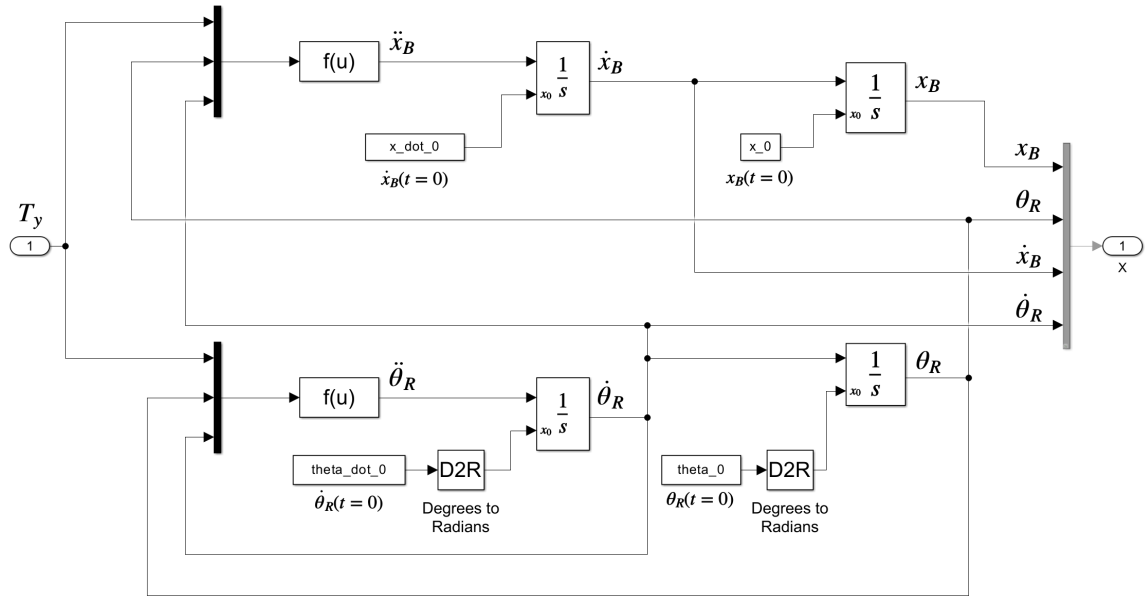
Five simulation scenarios are considered, each corresponding to specific initial conditions applied to the system state:

1. Initial position $x_B = -1$ m.
2. Initial position $x_B = -1$ m and velocity $\dot{x}_B = -0.1$ m/s.
3. Initial pitch angle $\theta_R = 4°$.
4. Initial pitch angle $\theta_R = 4°$ and angular velocity $\dot{\theta}_R = 4°/\text{s}$.
5. Combined perturbations: $x_B = -1$ m, $\dot{x}_B = -0.1$ m/s, $\theta_R = -4°$, and $\dot{\theta}_R = -4°/\text{s}$.

From the first two simulation scenarios shown in Figure 3.2, it can be observed that the controller effectively maintains the Ballbot within a safe operating range of $\pm 5°$ in pitch angle. As shown in Figure 3.2a, the first simulation demonstrates that the controller returns the Ballbot to the origin in under 10 seconds while keeping the pitch angle within the safe limits. In the second scenario, where the robot is subjected to both a position offset and an initial velocity of $\dot{x}_B = -0.1$ m/s, the pitch angle exhibits a larger deviation

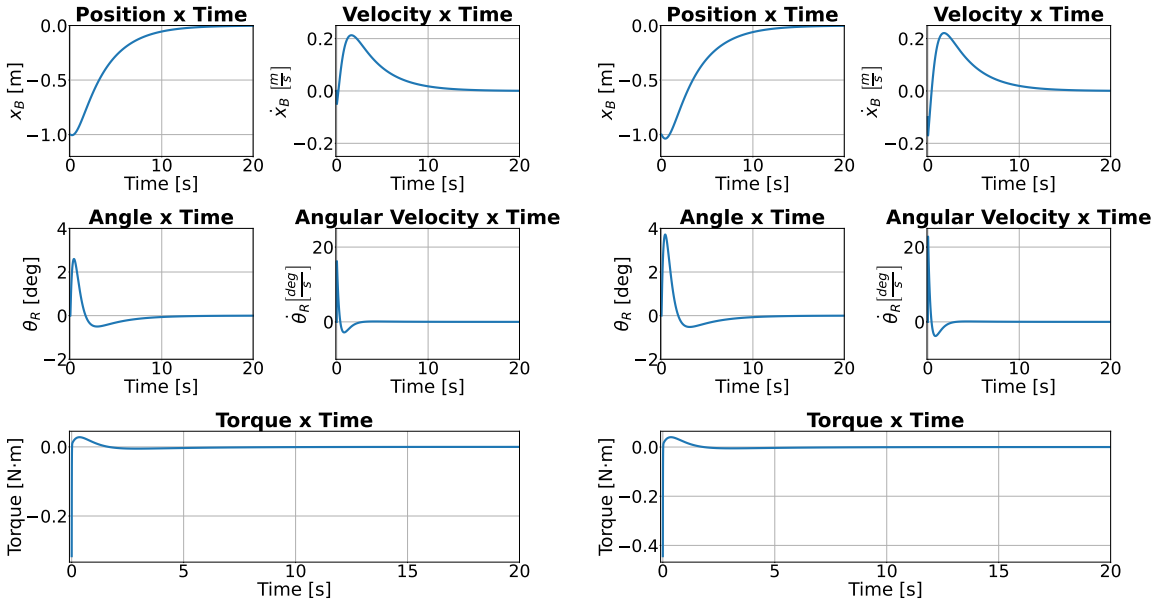


(a) Block diagram of the X–Z model in Simulink.



(b) Implementation of the system dynamics block.

FIGURE 3.1 – Simulink implementation of the Ballbot X–Z model used for the 2D simulations.



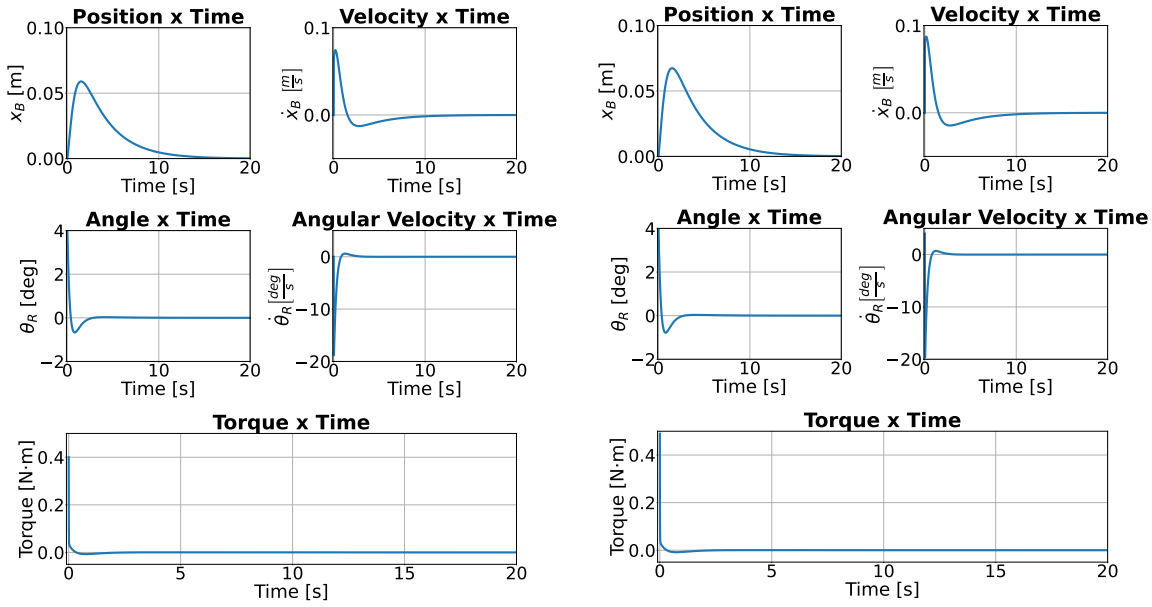
(a) $x_B(0) = -1m$ (b) $x_B(0) = -1m$ and $\dot{x}_B(0) = -0.1m/s$

FIGURE 3.2 – Simulation results for the X–Z planar model under different initial position and velocity conditions: (a) shows the response when the robot starts at $x_B(0) = -1$ m; (b) shows the response when the robot starts at $x_B(0) = -1$ m with an initial velocity of $\dot{x}_B(0) = -0.1$ m/s.

compared to the first case, but it remains within the predefined stability bounds, as illustrated in Figure 3.2b.

Similarly to the first two simulations, the controller keeps the Ballbot within the safe pitch angle range of ±5° when the initial pitch angle is 4°, as shown in Figure 3.3. This initial angular perturbation causes a small translational displacement, which remains within 10 cm of the starting position, as illustrated in Figure 3.3a. In the most challenging scenario, where an additional initial angular velocity of $\dot{\theta}_R = 4°/s$ drives the robot further from the stable equilibrium, the pitch angle still remains within the safe ±5° bounds, as depicted in Figure 3.3b.

When all initial states are simultaneously perturbed, as illustrated in Figure 3.4, the controller effectively stabilizes the Ballbot, returning the system to equilibrium while keeping the pitch angle within a safe range of ±5°, a limit determined concurrently from experiments with the real robot. This scenario also corresponds to the highest actuator demand, highlighting the controller's ability to handle combined initial deviations.



(a) $\theta_R(0) = 4°$

(b) $\theta_R(0) = 4°$ and $\dot{\theta}_R(0) = 4°/s$

FIGURE 3.3 – Simulation results for the X–Z planar model under different initial pitch conditions: (a) shows the response for an initial pitch angle of $\theta_R(0) = 4°$; (b) shows the response for an initial pitch angle of $\theta_R(0) = 4°$ combined with an initial angular velocity of $\dot{\theta}_R(0) = 4°/s$.

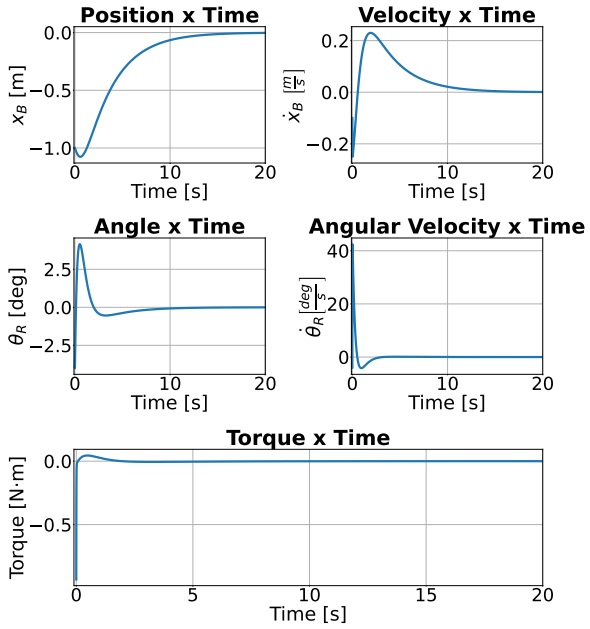


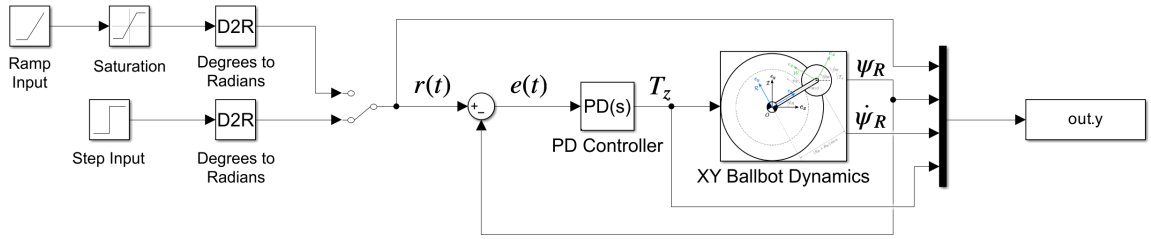
FIGURE 3.4 – Simulation of the X–Z planar model for the most critical scenario with initial conditions: $x_B(0) = -1\,\text{m}$, $\dot{x}_B(0) = -0.1\,\text{m/s}$, $\theta_R(0) = -4°$, and $\dot{\theta}_R(0) = -4°/s$. This scenario illustrates the controller's ability to stabilize the system under large initial deviations in both position and pitch.

In summary, for the 2D X–Z planar model, the proposed LQR controller consistently

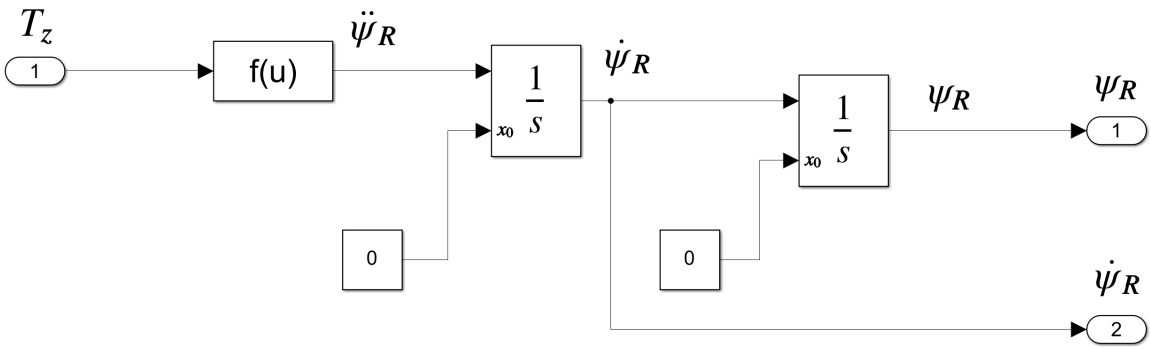
maintains asymptotic stability across all tested initial condition scenarios towards the origin, maintaining the pitch angle within the defined safe limits, and returning the system to its equilibrium.

3.1.2 X–Y Model

The *MATLAB Simulink* model corresponding to the X–Y plane is presented in Figure 3.5. As shown in Figure 3.5a, the model comprises the reference input generation, PD controller and the system dynamics block. A detailed view of the system dynamics implementation is provided in Figure 3.5b, where the `f(u)` block implements the equation of motion given by Equation (2.53).



(a) Block diagram of the X–Y model in Simulink.



(b) Detailed implementation of the system dynamics block.

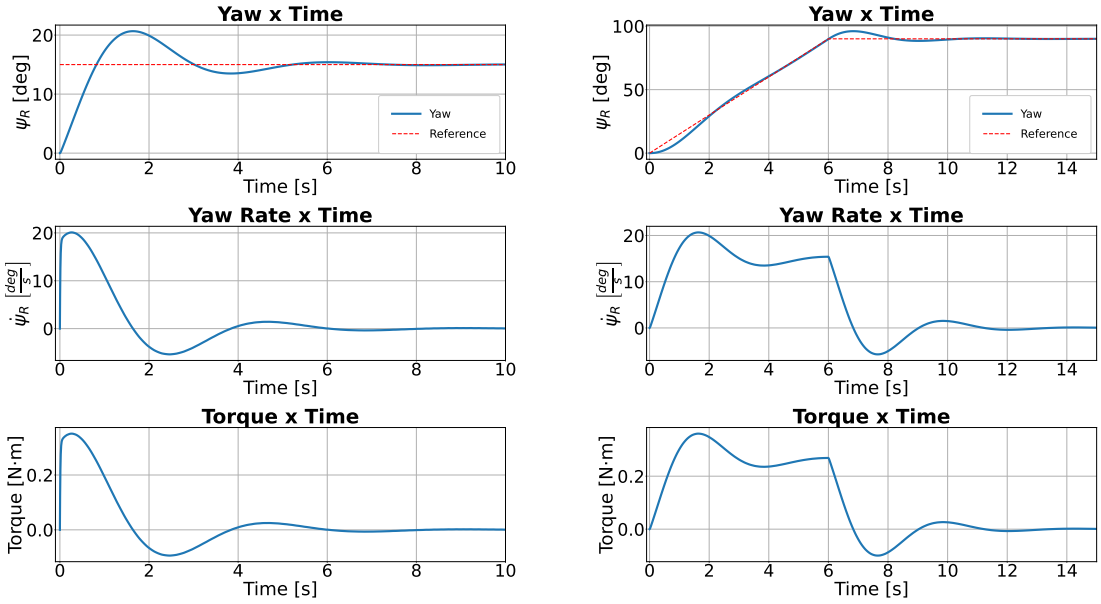
FIGURE 3.5 – Simulink implementation of the Ballbot X–Y model used for the 2D simulations.

Since the X–Y plane dynamics are marginally stable, a simple PD controller is employed to enable the Ballbot to track yaw reference inputs. The controller gains are set to $K_p = -0.02$ and $K_d = -0.01$. These values are kept low to limit the output torque, ensuring that the torque applied by the wheel to the ball remains smaller than the friction between the ball and the ground, so that the ball stays effectively stationary while the robot rotates around its z axis.

Two simulation scenarios are considered to evaluate the controller's performance:

1. A step input of 15° applied to the yaw reference, used to analyze the transient and steady-state response to a sudden change in orientation.

2. A ramp input with a slope of 15°/s, used to assess the tracking capability for a continuously varying yaw reference.



(a) Step input response.

(b) Ramp input response.

FIGURE 3.6 – Simulation results for the X–Y planar model under different yaw reference inputs. Subfigure (a) shows the response to a step input of 15°, while subfigure (b) shows the response to a ramp input of 15°/s.

Figure 3.6 shows that the controller successfully tracks the input reference in both scenarios while maintaining the output torque below 0.4 N·m.

3.2 System Architecture

This section presents the overall architecture of the proposed Ballbot platform, which combines hardware and software components to achieve real-time control, reliable data transmission, and flexible sensor integration while maintaining a low implementation cost. The following subsections describes in detail the hardware and software employed. A general overview of the hardware components and communication interfaces is illustrated in Figure 3.7.

3.2.1 Hardware Architecture

The Ballbot prototype adopts a layered hardware architecture that integrates actuation, sensing, and computation to achieve real-time control and balance. This architecture follows a hierarchical design principle, in which low-level and high-level tasks are

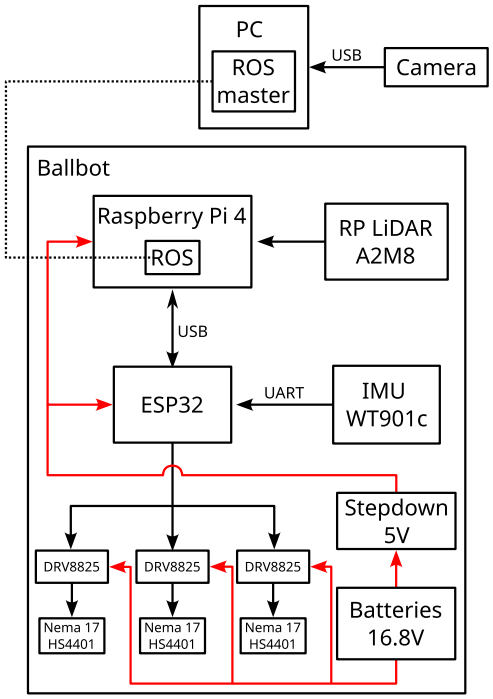


FIGURE 3.7 – Overview of the Ballbot hardware setup, showing the integration among sensors, actuators, and processing units.

distributed across dedicated hardware units. The low-level control layer, implemented on the microcontroller, is responsible for real-time signal acquisition, control loop execution, and actuator commands. Meanwhile, the high-level control layer, executed on the microcomputer, handles more complex algorithms and communications under the ROS environment. The overall system is detailed below.

3.2.1.1 Actuators

The three Ballbot omniwheels are driven by three NEMA 17 stepper motors model 17HS4401, uniformly spaced at 120° around the ball. These actuators provide precise position and velocity control, ensuring stable balance and smooth motion. The motors selected are low-cost, off-the-shelf stepper motors, making the system accessible and easily reproducible. The stepper motor model used is shown in Figure 3.8, and its main specifications are summarized in Table 3.1.

3.2.1.2 Motor Drivers

Each stepper motor is driven by a DRV8825 driver, which supports microstepping for finer control of motor speed and position. A microstep resolution of 1/32 is adopted to achieve smooth and precise motion. The drivers are commanded by the microcontroller through pulse and direction signals, with their maximum current, and consequently output torque, limited by thermal constraints. The driver module used is shown in Figure 3.9,

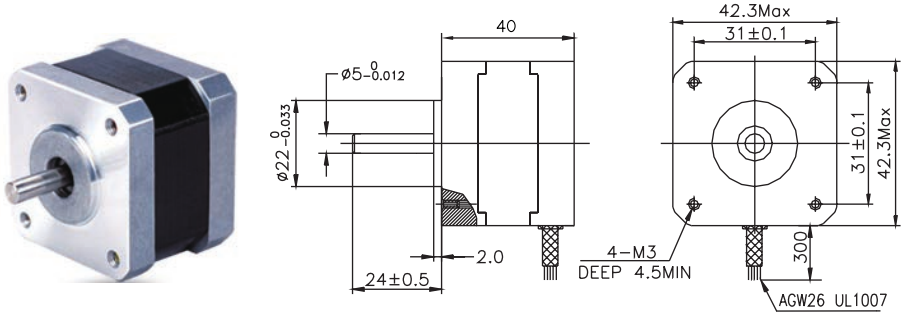


FIGURE 3.8 – Illustration and dimensions of the 17HS4401 stepper motor.

TABLE 3.1 – Specifications for the 17HS4401 stepper motor used in the Ballbot prototype.

Parameter	Value	Unit
Step Angle	1.8	deg
Motor Length	40	mm
Rated Current	1.7	A
Phase Resistance	1.5	Ω
Phase Inductance	2.8	mH
Holding Torque	40	N·cm
Detent Torque	2.2	N·cm
Rotor Inertia	54	g·cm^2
Weight	280	g

and its main specifications are summarized in Table 3.2.

FIGURE 3.9 – DRV8825 stepper motor driver used to control the Ballbot's stepper motors.

3.2.1.3 Sensors

Two sensing modules are used for state estimation and environment perception. The low-cost Micro-Electro-Mechanical Systems (MEMS) IMU, the Witmotion WT901C shown in Figure 3.10, provides angular velocity and linear acceleration measurements, with real-time attitude estimation performed by its onboard firmware. The main specifications of the IMU are summarized in Table 3.3.

Additionally, the RPLIDAR A2M8 sensor, shown in Figure 3.11, performs two-dimensional scanning to support localization and mapping within the ROS framework. By enabling onboard environment perception, it removes the dependency on external position information sources. The main specifications of the LiDAR module are summarized in Table 3.4.

TABLE 3.2 – Specifications for the DRV8825 stepper motor driver.

Parameter	Value	Unit
Minimum operating voltage	8.2	V
Maximum operating voltage	45	V
Continuous current per phase	1.5	A
Maximum current per phase	2.2	A
Minimum logic voltage	2.5	V
Maximum logic voltage	5.25	V
Microstep resolutions	full, 1/2, 1/4, 1/8, 1/16, 1/32	–

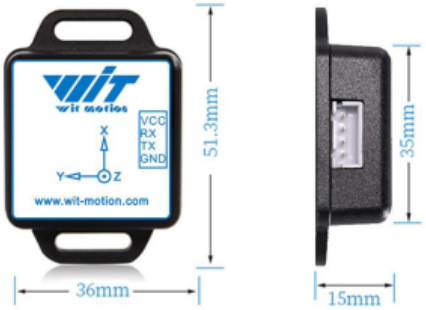


FIGURE 3.10 – Illustration and dimensions of the Witmotion WT901C IMU module.

TABLE 3.3 – Specifications for the WT901C 9-axis IMU module.

Parameter	Value	Unit
Accelerometer range	±16	g
Accelerometer resolution	0.0005	g/LSB
Accelerometer RMS noise	0.75-1	mg (RMS)
Gyroscope range	±2000	deg/s
Gyroscope resolution	0.061	(deg/s)/LSB
Gyroscope RMS noise	0.028-0.07	deg/s (RMS)
Pitch & Roll inclination accuracy	0.2	deg
Heading angle accuracy (9-axis)	1	deg
Maximum output data rate	200	Hz
Operating temperature range	-20 to +60	°C

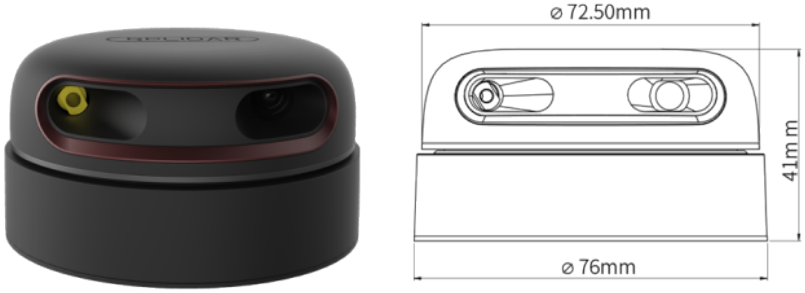


FIGURE 3.11 – Illustration and physical dimensions of the RPLIDAR A2M8 2D scanning sensor.

TABLE 3.4 – Specifications for the RPLIDAR A2M8 360° 2D LiDAR scanner.

Parameter	Specification	Unit
Measuring range	0.2-12	m
Rotational speed	5-15	Hz
Angular resolution	0.45	deg
Sampling frequency	up to 8	kHz
Angular Range	360	deg
System voltage	5	V
Operating temperature range	0-40	°C
Weight	190	g

For configurations where the RPLIDAR is not mounted on the Ballbot, an external Logitech C270 webcam, placed outside the robot and oriented to view the scene from above with its imaging plane parallel to the ground, is connected to an external computer and employed to detect an ArUco marker affixed to the robot's top section. This approach allows the estimation of the Ballbot's global position and heading (yaw angle) within the experimental environment by processing the camera images with computer vision algorithms. The camera module and its main specifications are shown in Figure 3.12 and Table 3.5, respectively.

FIGURE 3.12 – Logitech C270 HD Webcam used for ArUco marker detection and global position estimation.

TABLE 3.5 – Specifications for the Logitech C270 HD Webcam.

Parameter	Specification
Maximum resolution	720p/30 fps
Camera megapixel	0.9 MP
Focus type	Fixed
Diagonal field of view	55°

3.2.1.4 Microcontroller

An ESP32 microcontroller manages the low-level control of the Ballbot. It acquires real-time data from the IMU, executes the LQR and PD control algorithms, and sends

command signals to the DRV8825 drivers. The ESP32 also establishes a serial communication interface with the onboard computer, transmitting sensor data and receiving control parameters and references.

TABLE 3.6 – Specifications for the ESP32 microcontroller.

Parameter	Specification
CPU	Dual-core 32-bit Xtensa LX6, up to 240 MHz
Wi-Fi	802.11 b/g/n, up to 150 Mbps, 2.4 GHz
Bluetooth	v4.2 BR/EDR and BLE, class 1/2/3
General Purpose Input/Output	34 programmable pins
Analog-to-Digital Converter	12-bit, up to 18 channels
Digital-to-Analog Converter	2x 8-bit channels
Pulse Width Modulation	Up to 16 channels, including motor PWM
Communication Interfaces	SPI, I2C, I2S, UART, SDIO
Power Supply	2.3 V-3.6 V DC or 5 V via onboard regulator

3.2.1.5 Onboard Computer

The Raspberry Pi 4 serves as the main computational unit of the Ballbot and runs *ROS Noetic Ninjemys*, handling high-level processes such as laser scan matching for localization. Communication with the ESP32 microcontroller is established through a USB serial interface, enabling data exchange between the two layers. The Raspberry Pi receives the Ballbot's state from the microcontroller and publishes it through ROS topics, while transmitting control parameters and reference signals back to the ESP32. Additionally, it interfaces with the LiDAR sensor and external networks for system supervision and remote teleoperation.

TABLE 3.7 – Specifications for the Raspberry Pi 4.

Parameter	Specification
Processor	Broadcom BCM2711, quad-core Cortex-A72 (ARM v8) 64-bit SoC @ 1.8 GHz.
Memory (RAM)	2 GB LPDDR4.
Wireless	Dual-band 802.11b/g/n/ac (2.4 GHz and 5 GHz), Bluetooth 5.0, BLE.
USB Ports	2 x USB 3.0 and 2 x USB 2.0.
Storage Interface	MicroSD card slot for operating system and data storage.
Power Supply	5 V DC via USB-C (minimum 3 A) or GPIO header.
General Purpose Input/Outpu	40-pin header.

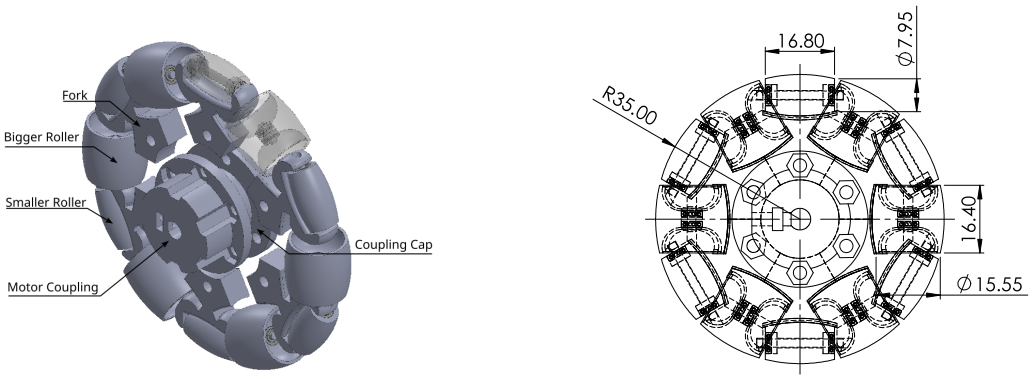
3.2.1.6 Power Supply

The system is powered by a battery pack composed of four 18650 lithium-ion cells, 4.2 V each, connected in series, resulting in a nominal voltage of 16.8 V. Each cell has a capacity of 2200 mAh, and the system draws approximately 3 A during operation, leading to an estimated continuous operating time on the order of 40 minutes under nominal conditions. This configuration provides sufficient capacity while maintaining a compact and lightweight design. The battery supplies power to both the motors and the control electronics through regulated outputs.

3.2.1.7 Omniwheels

To enable omnidirectional motion, each Ballbot wheel consists of twelve unactuated rollers mounted around its circumference, with their rotation axes oriented perpendicular to the wheel's main axis. This configuration allows the wheel to generate traction in one direction while minimizing resistance in the orthogonal direction. Each roller is equipped with bearings to reduce friction and ensure smoother contact with the ball surface. The entire wheel assembly is 3D printed in Polylactic Acid (PLA) filament, except for the bearings.

An exploded view illustrating the main components of the wheel is presented in Figure 3.13a, while the key geometric dimensions are shown in Figure 3.13b.



(a) Exploded view showing the main components of the Ballbot omniwheel assembly.

(b) Main dimensions of the Ballbot omniwheel (all dimensions in mm).

FIGURE 3.13 – Design details of the omniwheel implemented in the Ballbot prototype.

3.2.1.8 Body Structure

The Ballbot's body is based on a modular design that facilitates rapid modifications and the integration of additional sensors or hardware. The structure comprises three vertically stacked levels. The lower level houses the stepper motor and wheel assemblies,

while the battery pack is mounted immediately above it, concentrating the system's mass near the base. The second level houses the IMU, microcontroller, and motor drivers, with a fan positioned between the second and third levels to provide effective heat dissipation for the motor drivers, allowing for higher torque output. A 3D representation illustrating the relative arrangement of the main hardware components is presented in Figure 3.14.

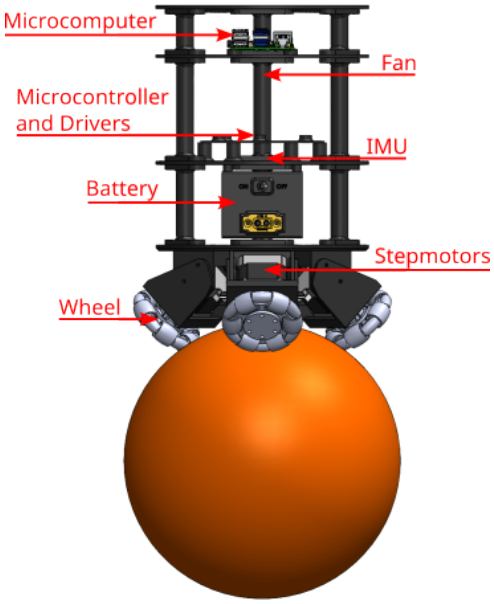


FIGURE 3.14 – 3D representation of the Ballbot prototype highlighting the relative arrangement of its main components.

The upper section of the structure serves as a mounting platform for either the RPLIDAR sensor or an ArUco marker, depending on the experimental configuration, as shown in Figure 3.15. The RPLIDAR is rigidly attached to the top platform using four M3 bolts, while the ArUco marker can be quickly installed in the same position through a magnetic interface after removing the lidar. All structural levels are designed as honeycomb-patterned disks to minimize weight and improve airflow, while the entire body is 3D-printed using PLA material. The overall geometry and primary dimensions of the Ballbot are illustrated in Figure 3.16.

3.2.1.9 Ball

An off-the-shelf basketball serves as the rolling element for the Ballbot prototype, providing a readily available and lightweight platform for omnidirectional motion. The ball has a diameter of 240 mm and a mass of 580 g. To reduce measurement noise in the IMU caused by vibrations resulting from the contact between the wheel and the ball's textured surface, the basketball was carefully sanded to smooth out its surface irregularities. This modification minimizes the effects of surface undulations, providing a more consistent rolling interface and improving the quality of the onboard sensor readings.

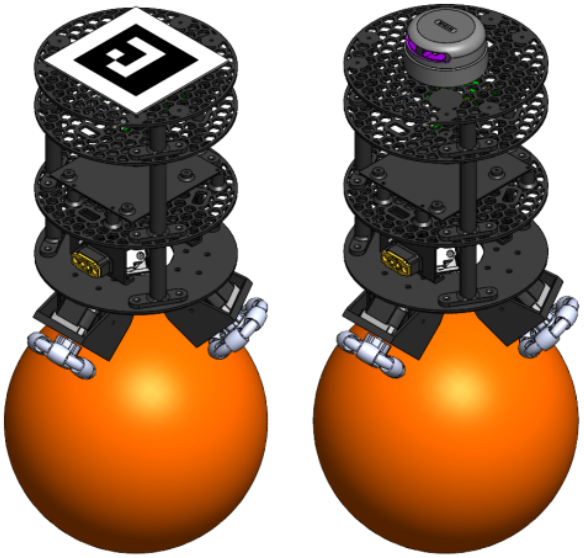


FIGURE 3.15 – 3D renderings of the Ballbot showing configurations with either the RPLIDAR sensor (left) or the ArUco marker (right) mounted on the top section.

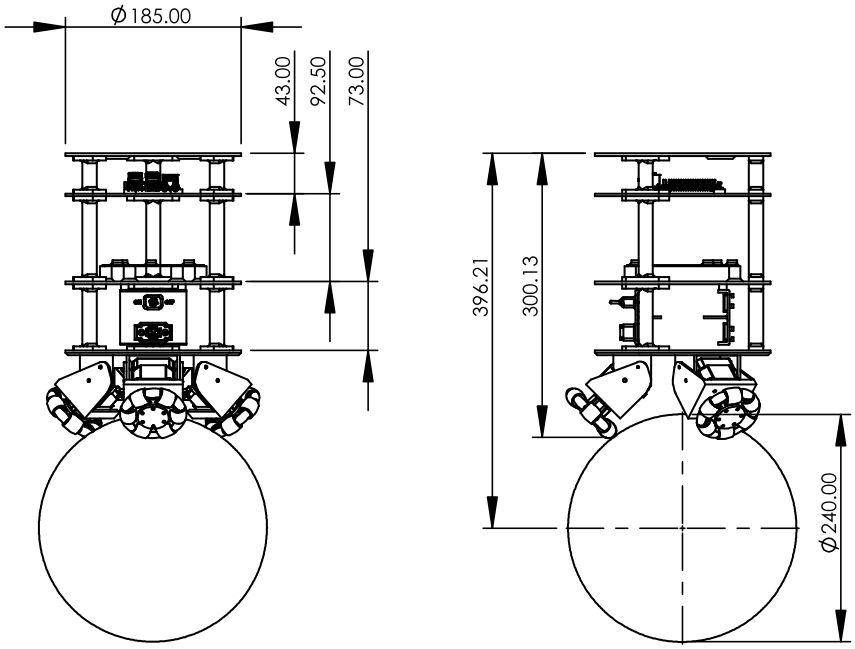


FIGURE 3.16 – Front and left views of the Ballbot prototype showing its main geometric dimensions (all dimensions in mm).

3.2.1.10 Summary

The proposed Ballbot prototype integrates all the hardware components described above into a compact layered design. Table 3.8 presents an overview of the main electronic and mechanical components used in the system. The main geometric and physical properties of the prototype are summarized in Table 3.9. The geometric information, including dimensions and moments of inertia, was extracted from the Computer-Aided Design (CAD) software *SolidWorks*, whereas the physical properties, such as mass, were measured directly on the assembled prototype.

TABLE 3.8 – Summary of Ballbot hardware components.

Component	**Type / Model**
Microcontroller	ESP32
Onboard Computer	Raspberry Pi 4
Stepper Motors	NEMA 17, 17HS4401
Motor Drivers	DRV8825
IMU	Witmotion WT901C
LiDAR Sensor	RPLIDAR A2M8
External Camera	Logitech C270
Omniwheels	12 passive rollers, fully 3D-printed in PLA
Rolling Ball	Standard basketball
Body Structure	3D-printed in PLA

TABLE 3.9 – Geometric and physical properties of the Ballbot prototype.

Property	**Value**	**Unit**
Total Robot Mass	2.3	kg
Robot Diameter	185	mm
Robot Height (off the ball)	396	mm
Robot C.G. (from ball center)	194	mm
Robot Moment of Inertia	$I_{R_{xx}} = 1.93 \times 10^{-2}$ $I_{R_{yy}} = 1.93 \times 10^{-2}$ $I_{R_{zz}} = 1.14 \times 10^{-2}$	kg·m^2
Wheel Mass	3 x 38.7	g
Wheel Moment of Inertia	$I_W = 1.96 \times 10^{-5}$	kg·m^2
Ball Mass	580	g
Ball Diameter	240	mm
Ball Moment of Inertia (Hollow Sphere)	$I_B = 5.57 \times 10^{-3}$	kg·m^2

3.2.2 Software Architecture

The software architecture of the Ballbot prototype follows the same hierarchical principle established in the hardware design, comprising two primary layers: a *low-level control layer* responsible for real-time data acquisition and actuation, and a *high-level layer* dedicated to external communication and LiDAR data processing. This structure ensures modularity, scalability, and reliable real-time operation.

3.2.2.1 Low-Level Control Layer

The control architecture implemented on the ESP32 is developed in C++ and operates at a control frequency of 200 Hz, matching the output rate of the IMU. The dual-core structure of the microcontroller is exploited to separate computation tasks: one core executes the real-time control loop, while the second core handles command transmission to the motor drivers.

3.2.2.1.1 Sensor Calibration and Attitude Estimation

Before control initialization, the IMU's accelerometer and gyroscope undergo an internal firmware-based calibration routine. This procedure requires the robot to be positioned on a leveled surface, as any inclination during calibration can significantly degrade the accuracy of the attitude estimation and, consequently, the overall control performance. The IMU provides attitude angle estimates through a Kalman filter implemented within its firmware, while angular velocity measurements are passed through a low-pass filter to minimize high-frequency noise. Although the IMU provides estimates of all attitude angles, the yaw measurement is disregarded due to challenges in reliably calibrating the magnetometer. Instead, the yaw angle is obtained either through scan matching using the onboard LiDAR or via pose estimation from the external camera. The linear accelerations measured in the body frame are projected onto a body-fixed, ground-aligned reference frame, which preserves the robot's heading while removing roll and pitch, and the translational velocities are subsequently obtained by integrating this leveled acceleration.

3.2.2.1.2 External State Estimation Integration

The x and y position and yaw ψ_r states are derived from external sources depending on the experimental configuration. When the LiDAR is installed, scan matching algorithms provide both position and heading information at 10 Hz, whereas when the ArUco marker is employed, equivalent data are obtained through camera-based pose estimation at 30 Hz. The estimation is processed in the high-level control layer and transmitted to the ESP32

via serial communication. Since these frequencies do not match the control loop rate, delayed measurements are used in the control loop until a new update is obtained from the respective sensor.

3.2.2.1.3 Control Implementation and Actuation

The system employs two complementary control strategies: an LQR controller for balance and translation control in the X–Z and Y–Z planes, and a PD controller for yaw regulation in the X–Y plane. The LQR controller, originally designed for the continuous-time system, is directly implemented in the discrete control loop without discretization. This approach is justified by the sufficiently high control frequency of 200 Hz. Both controllers compute the required torques (T_x, T_y, T_z) to achieve the desired body orientation and position. The resulting torques are subsequently converted into individual wheel torques (T_1, T_2, T_3) through a geometric transformation that accounts for the Ballbot's omni-directional wheel arrangement. Since direct torque control is non-trivial in stepper motors, the torque computed at each control loop is assumed constant over the loop interval and integrated to produce an incremental change in angular velocity, which is accumulated over time to form the angular velocity references (V_1, V_2, V_3) transmitted to the motor drivers via the `AccelStepper` library. To ensure safe operation and prevent motor saturation, the velocity commands are constrained to a maximum of 2 revolutions per second.

3.2.2.1.4 Data Processing and Transmission

During each control iteration, all relevant state variables are transmitted from the microcontroller (ESP32) to the onboard microcomputer (RaspberryPi) via serial communication. The transmitted data packet, delimited by start and end bytes, includes the planar position and velocity components, linear accelerations, angular velocities, attitude angles, wheel angular velocities, and the control loop frequency.

3.2.2.2 High-Level Control Layer

Complementing the low-level control layer, the high-level control layer is executed on the Raspberry Pi microcomputer and is responsible for communication management, data handling, and the integration of external localization sources. This layer is implemented within the ROS framework and organized into modular nodes written in Python, ensuring scalability, maintainability, and extensibility of the system.

3.2.2.2.1 Central Communication Node

The central node of this layer, named `ballbot_node`, runs on the Raspberry Pi and manages the serial communication between the ESP32 microcontroller and the on-board microcomputer. All ROS-related functionality is executed exclusively on the Raspberry Pi, while the ESP32 operates without ROS and is dedicated to low-level sensing and control. The `ballbot_node` unpacks the telemetry data received from the ESP32 and publishes it at a rate of approximately 90 Hz through three dedicated ROS topics:

- `/ballbot_data`, which contains the robot's planar position and velocity, the control loop frequency, and the input velocities of each wheel;
- `/ballbot_imu`, provides all IMU-related measurements; and
- `/ballbot_refs`, includes the reference inputs for the control algorithms.

In addition to data publication, `ballbot_node` subscribes to the topic `/ballbot/odometry/filtered`, which provides the robot's x and y positions and heading information. Depending on the experimental configuration, this topic is published either by the `laser_scan_matcher` node when the LiDAR is employed or by the camera-based localization node when visual pose estimation using an ArUco marker is used. The received position and heading information are transmitted back to the ESP32 via serial communication, thereby supplying the microcontroller with state variables not available from the IMU. This architecture enables the integration of different sensors or higher-level localization algorithms with minimal modification, reinforcing the scalability and modularity of the system.

3.2.2.2.2 Command and Parameter Management

The `ballbot_node` also subscribes to the `/ballbot_cmd` topic, through which the controller parameters and input references can be adjusted and transmitted to the ESP32. Additionally, the `laser_scan_matcher` node operates on the Raspberry Pi, processing LiDAR data in real time to estimate the robot's position and orientation.

3.2.2.2.3 Graphical User Interface

To support real-time monitoring and control, a Graphical User Interface (GUI) was developed to visualize telemetry data, adjust controller parameters, and send reference inputs to the robot, as shown in Figure 3.17. The GUI, also implemented in Python, can be executed on any computer within the same ROS network, enabling flexible operation and supervision of the Ballbot system.

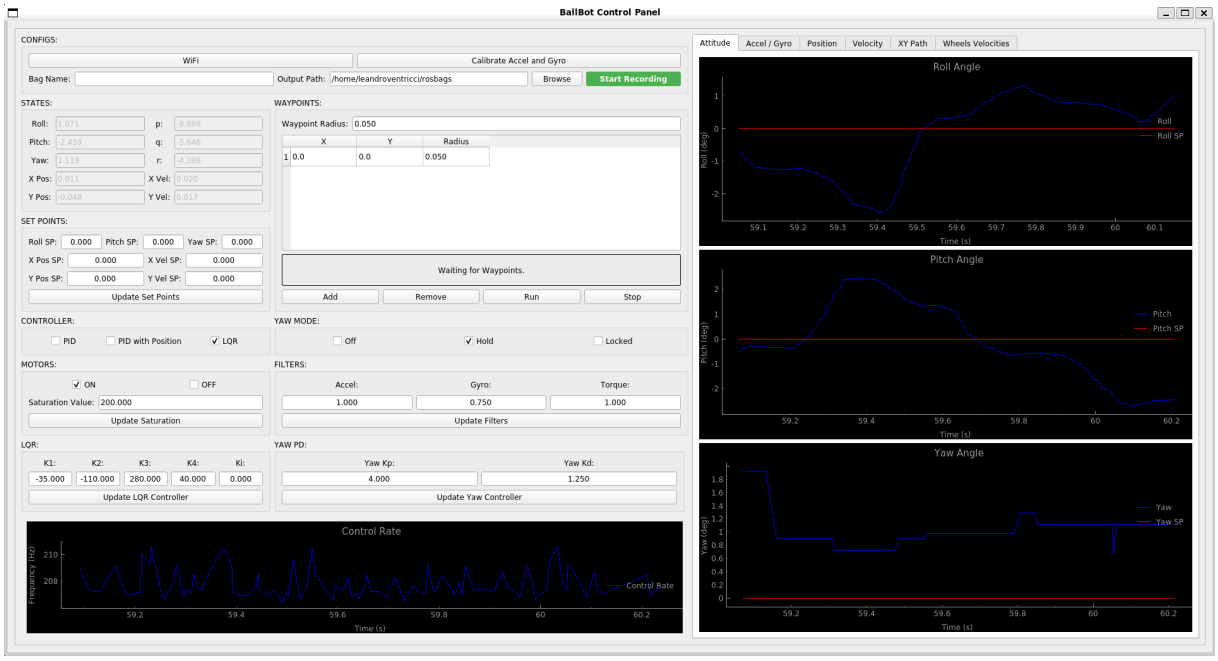


FIGURE 3.17 – Ilustration of the GUI developed for real-time monitoring and control of the Ballbot. The interface allows visualization of telemetry data, parameter tuning, and transmission of reference inputs.

3.2.3 Control Diagram

The overall control architecture of the Ballbot is illustrated in Figure 3.18. The diagram synthesizes the interaction between the embedded control loop and the external estimation modules, highlighting the asynchronous operation at different sampling rates. The control loop executes the LQR and PD algorithms at 200 Hz, receiving position and yaw reference inputs and generating wheel velocity commands. Feedback signals are obtained from the onboard IMU at the same rate, providing angular measurements $(\phi_R, \theta_R, \dot{\phi}_R, \dot{\theta}_R)$ and linear accelerations, which are integrated and transformed into velocities in the leveled reference frame. The external layer operates at lower frequencies, providing global position and yaw angle estimates (x_B, y_B, ψ_R) through either laser-based scan matching at 10 Hz or vision-based ArUco marker detection at 30 Hz. This modular architecture enables further development of the Ballbot platform, including integration of additional sensors or advanced localization algorithms, without requiring major modifications to the underlying control structure.

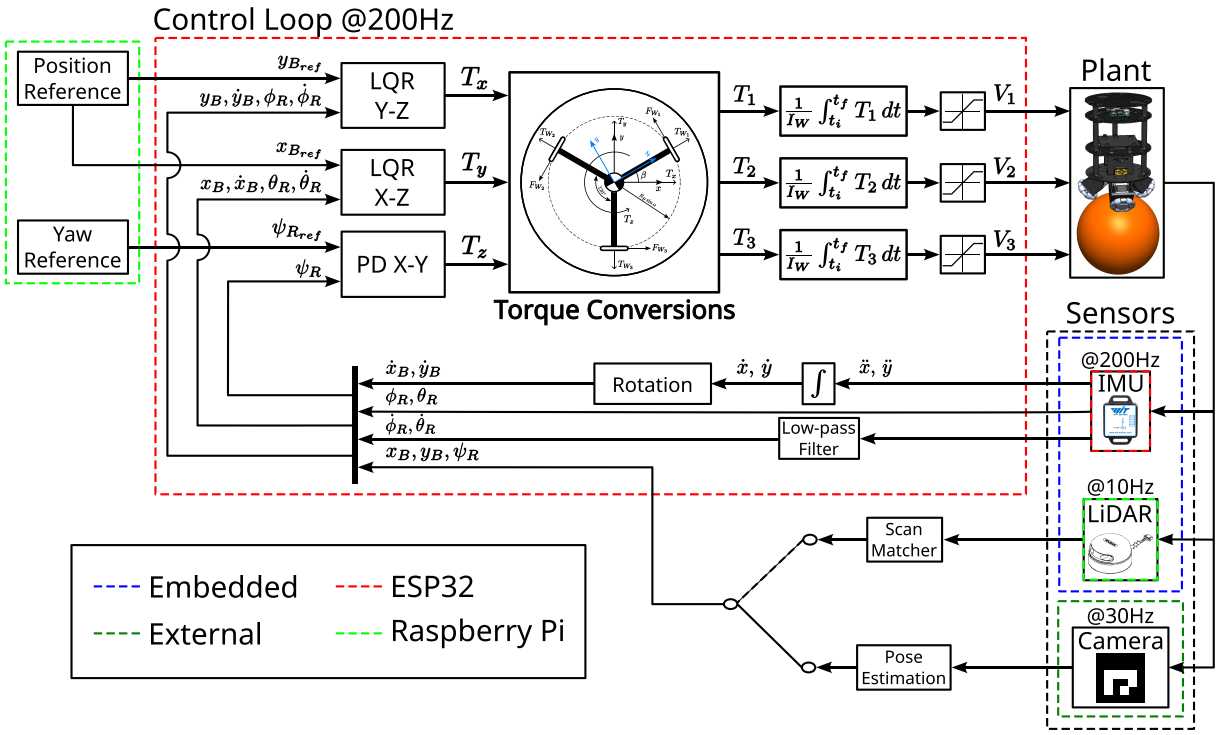


FIGURE 3.18 – Overall control diagram of the Ballbot system.

4 Experiments and Discussion

To validate the proposed control approach, a 3D simulation is first performed using the built-in Ballbot model available in the *CoppeliaSim* simulation sofware. The simulation is used to verify the correctness of the transformations applied to extend the 2D model to the full 3D system and to evaluate the effectiveness of the independent X–Z and Y–Z LQR controllers in regulating the complete 3D dynamics. Following simulation validation, the full system architecture is implemented on the physical prototype. Two distinct experimental configurations are evaluated to assess the proposed architecture. In the first configuration, an external camera estimates the robot's pose using an ArUco marker mounted on top of the Ballbot. In the second configuration, the marker is replaced by a LiDAR sensor, and the robot's planar position (x_B, y_B) and yaw angle (ψ_R) are obtained by processing the LiDAR data through the laser scan matcher ROS package.

4.1 3D Simulation Results

The simulation environment is configured to accurately represent the physical system, with the robot and ball mass properties set to match their real-world values. The controller gains employed in the 3D simulation are identical to those obtained from the 2D modeling and used in the corresponding 2D simulations, ensuring consistency in performance evaluation. It is worth noting that the yaw PD controller was not tested due to difficulties in the simulation setup, which led to results that were not compatible with with the expected real-world behavior. The overall simulation setup and corresponding physical property configurations are illustrated in Figures 4.1 and 4.2. The simulation files for the 3D model are available at the project's GitHub repository: `https://github.com/Intelligent-Machines-Lab/Ballbot_LMI_V2/tree/main/Simulations/3D`.

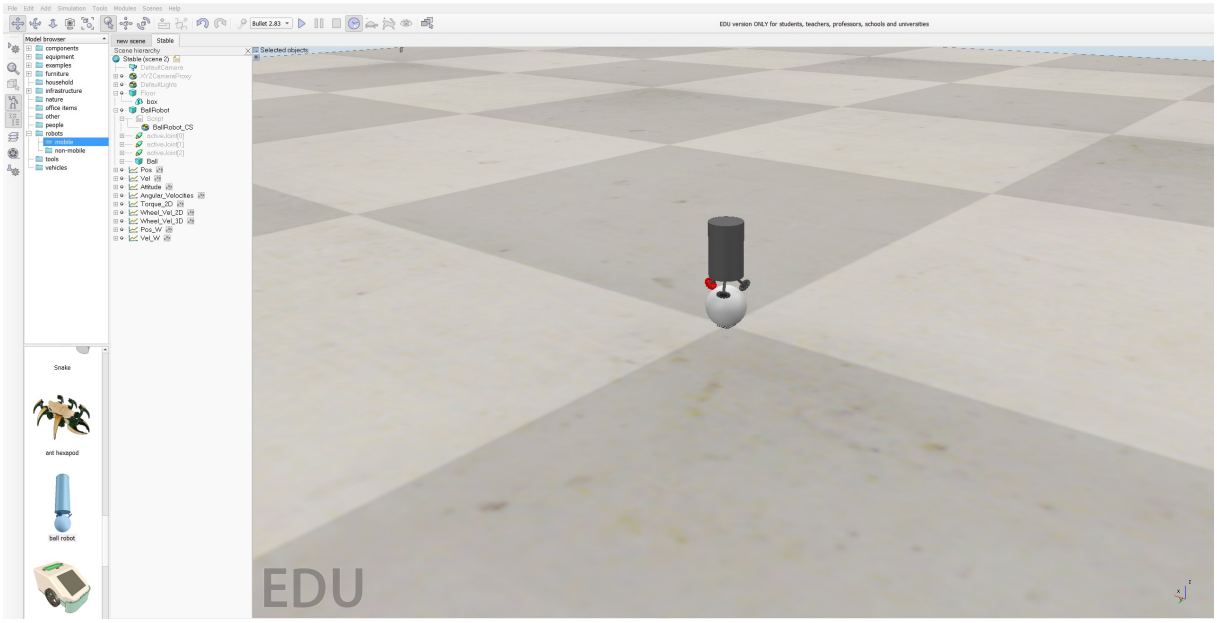


FIGURE 4.1 – CoppeliaSim simulation environment.

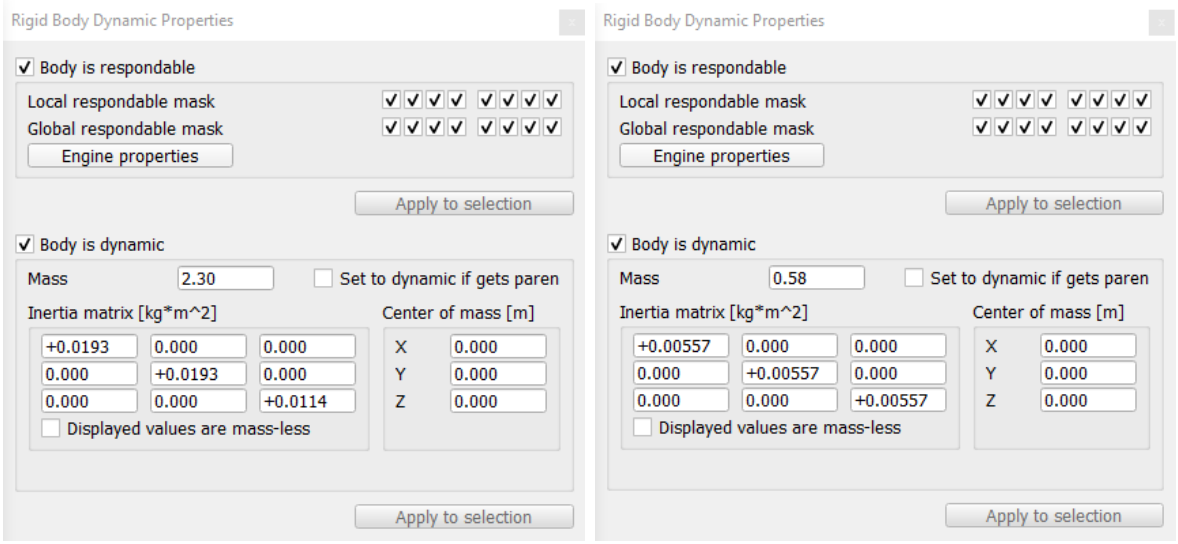


FIGURE 4.2 – Configuration of the robot and ball mass properties in the simulation environment.

4.1.1 Initial Pitch Disturbance

To evaluate the controller's capability to reject attitude disturbances, the Ballbot is initialized with a nonzero pitch angle while maintaining zero position error. This scenario tests the stabilization performance of the LQR controller when the system starts away from its unstable equilibrium point.

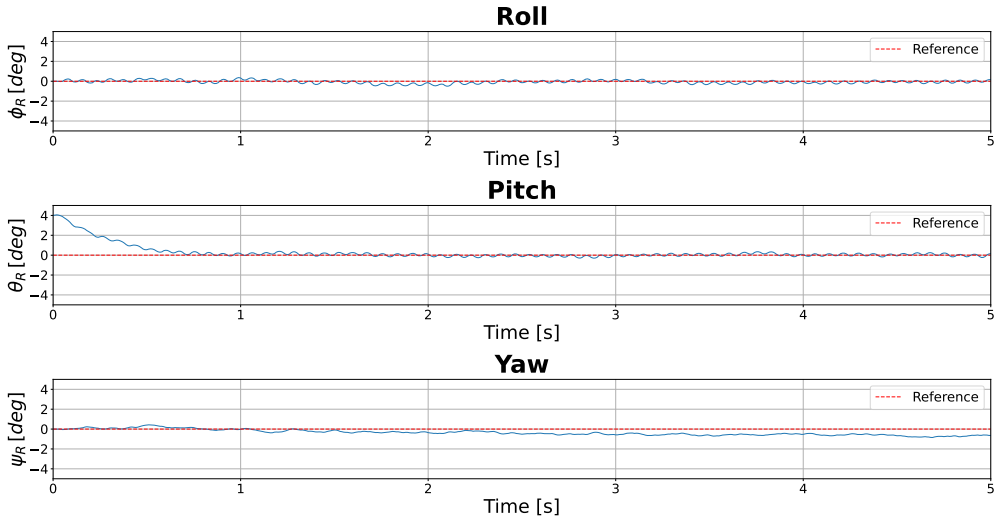


FIGURE 4.3 – Attitude angles for the initial attitude disturbance rejection test.

Figure 4.3 demonstrates that the controller rapidly stabilizes the Ballbot, driving the pitch angle to near-zero values in under 1 second. As shown in Figures 4.4 and 4.5, the attitude stabilization results in minimal position drift, with deviations under 5 cm in both x and y directions, and a position steady-state error remaining below 3 cm.

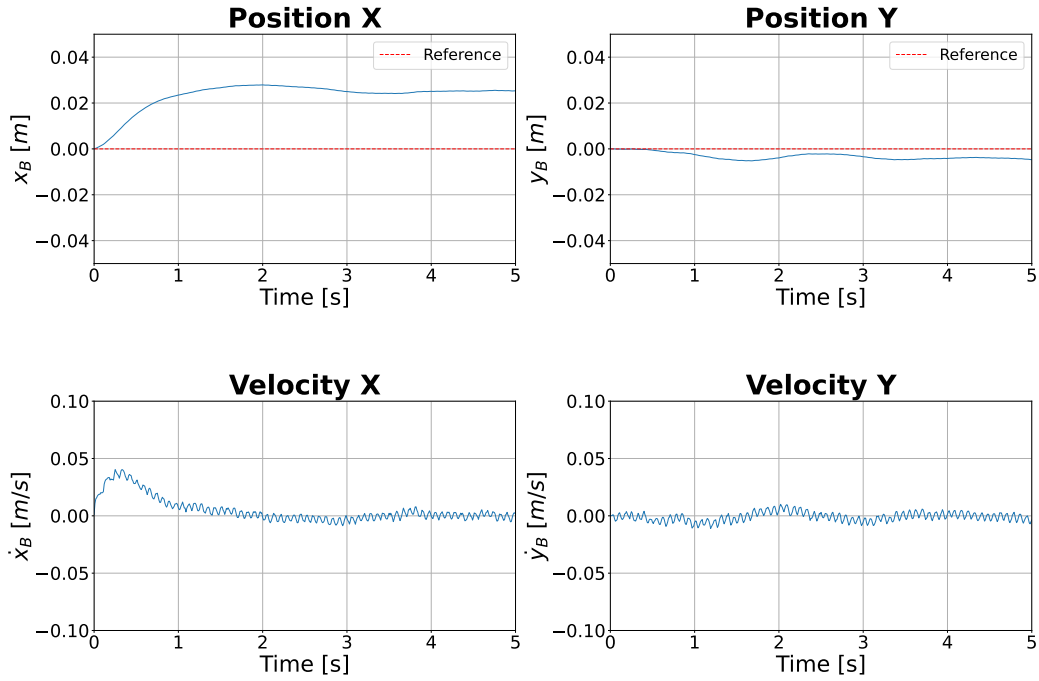


FIGURE 4.4 – Position and velocity output for the initial attitude disturbance rejection test.

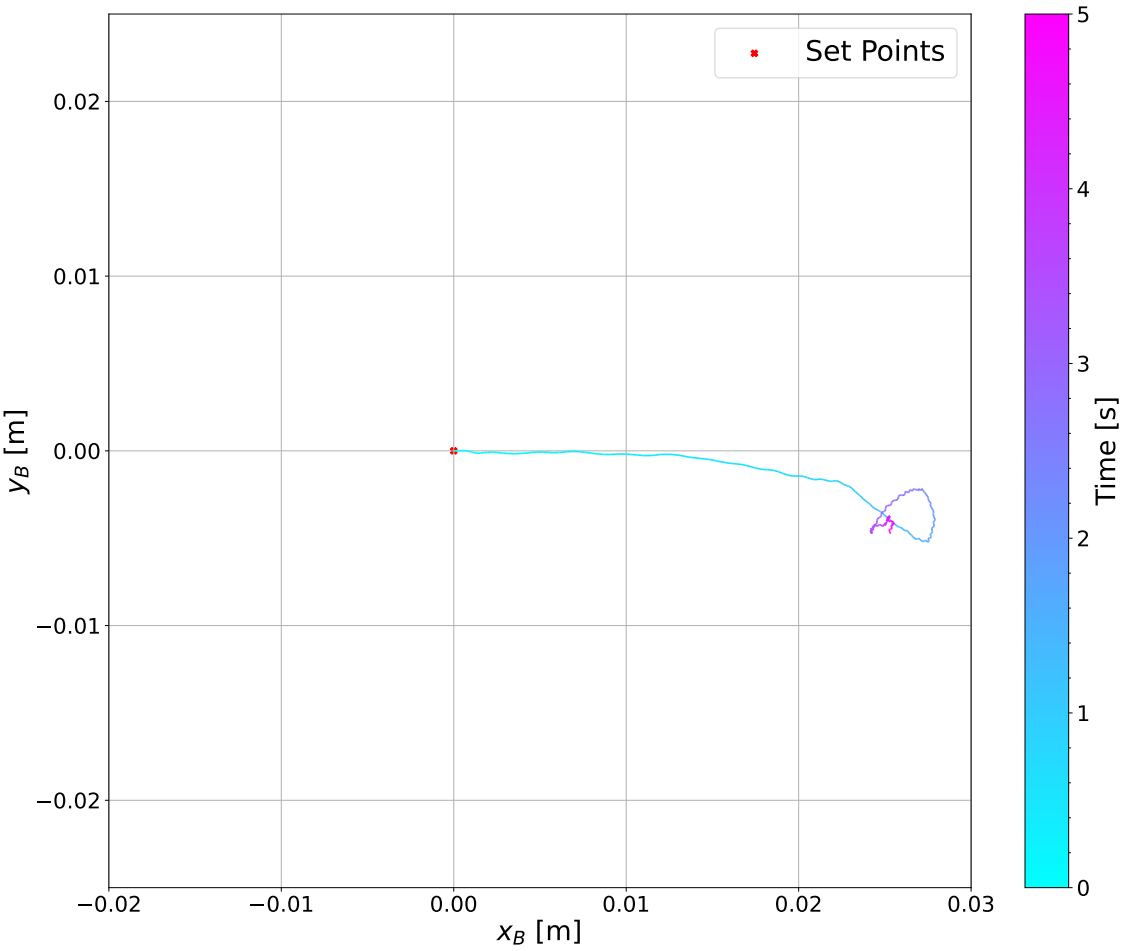


FIGURE 4.5 – X–Y trajectory for the initial attitude disturbance rejection test.

4.1.2 Initial Position Disturbance

This test evaluates the controller's ability to drive the robot from an initial position offset back to the origin while maintaining balance. The Ballbot is initialized with zero attitude angles but with nonzero position error in the x direction.

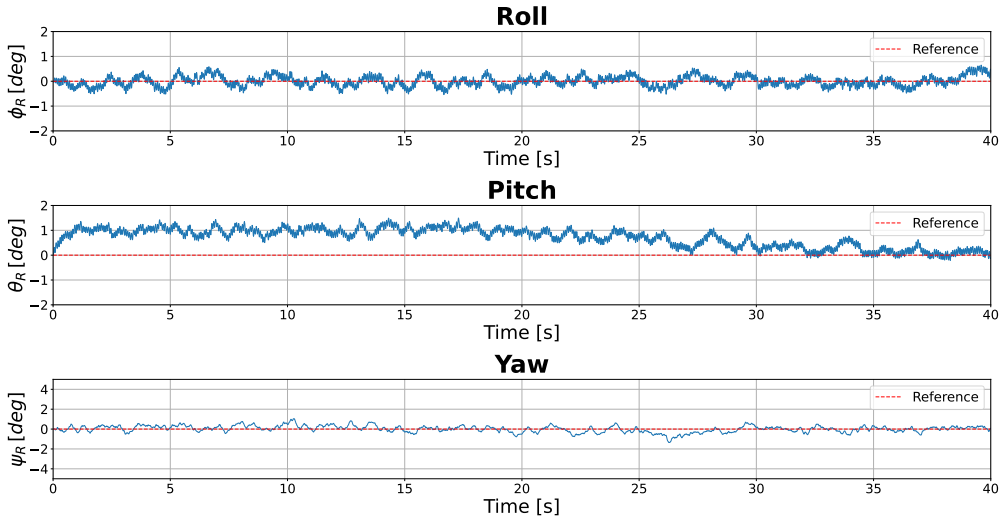


FIGURE 4.6 – Attitude angles for the initial position disturbance rejection test.

Figure 4.6 shows that the attitude angles remain tightly regulated within $\pm 2°$ during the position correction maneuver. As demonstrated in Figures 4.7 and 4.8, the controller successfully regulates the robot's position back to the origin, achieving convergence with residual errors below 5 cm.

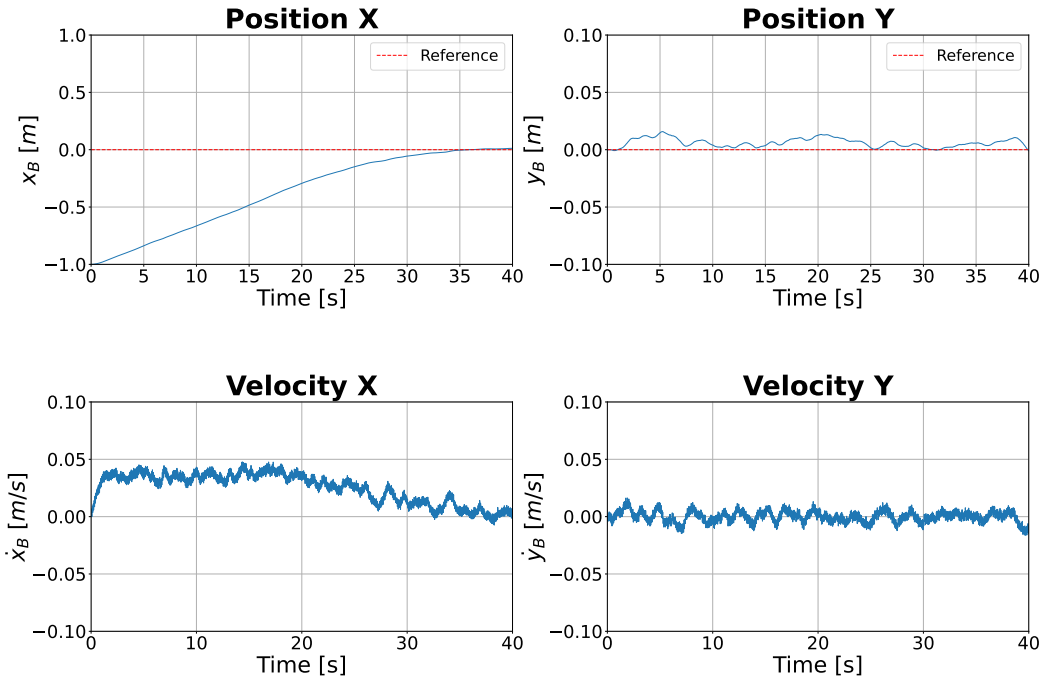


FIGURE 4.7 – Position and velocity output for the initial position disturbance rejection test.

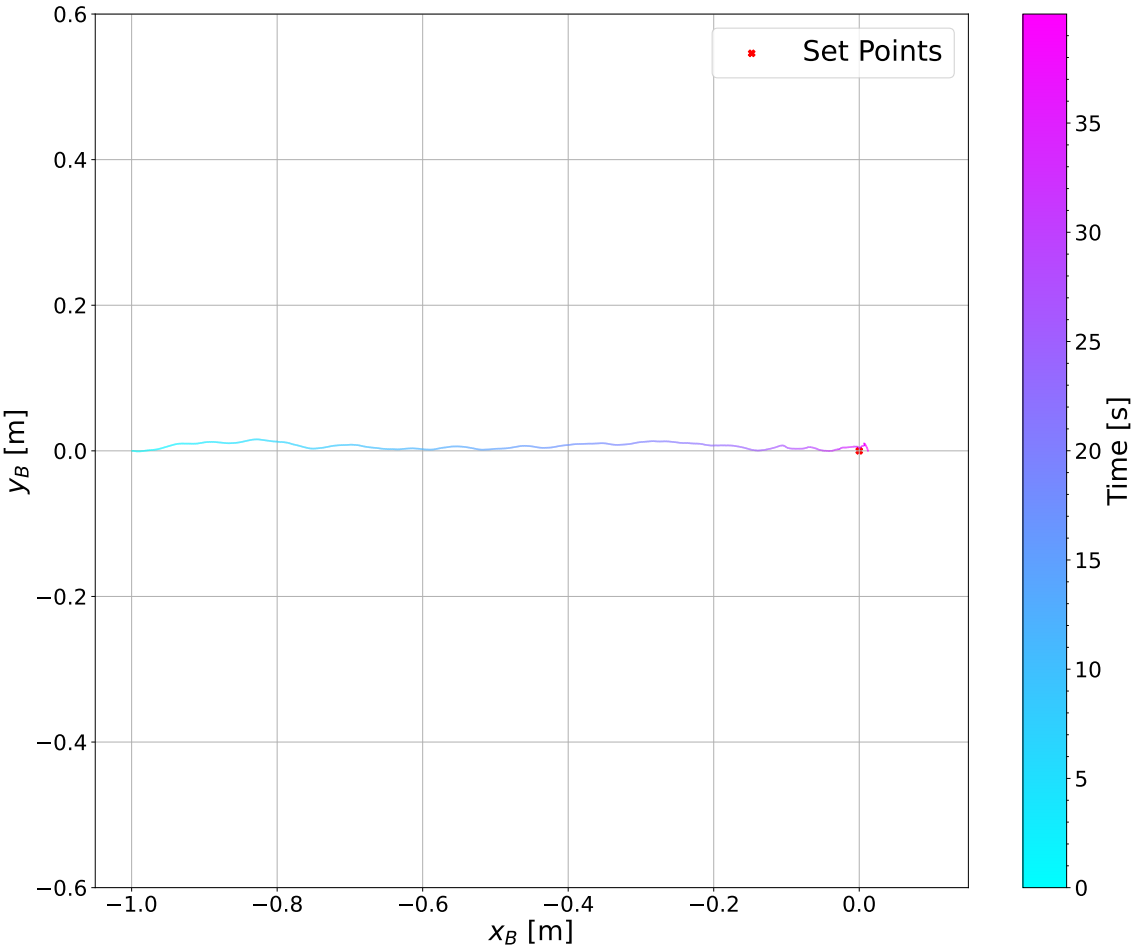


FIGURE 4.8 – X–Y trajectory for the initial position disturbance rejection test.

4.1.3 Combined Initial Position and Attitude Disturbance

To evaluate the controller's robustness under combined disturbances, the Ballbot is initialized with nonzero attitude angles and position offsets. This test case presents a more demanding scenario, simultaneously exciting the system dynamics along both the x and y axes.

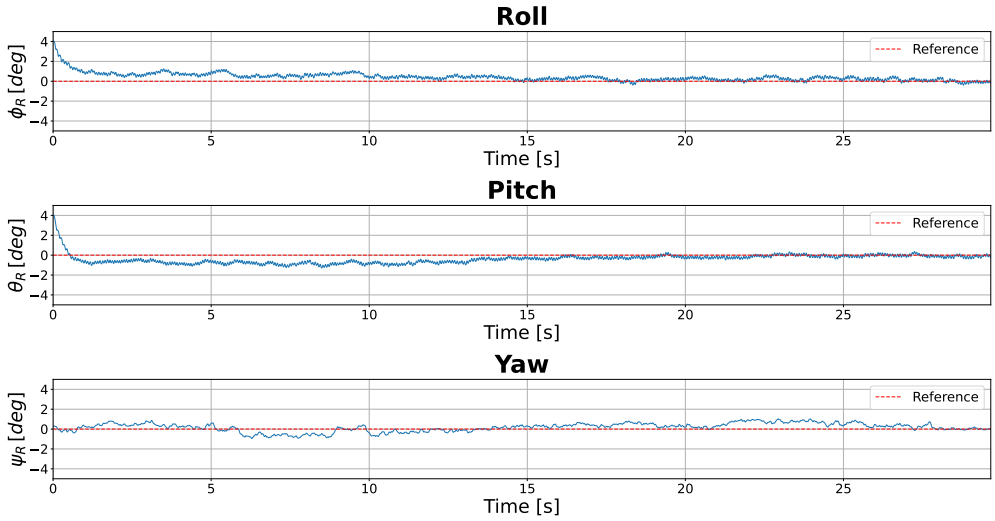


FIGURE 4.9 – Attitude angles for the combined initial position and attitude disturbance rejection test.

The results presented in Figures 4.9, 4.10, and 4.11 demonstrate that the controller successfully stabilizes the system despite the combined disturbances. Both attitude and position converge smoothly to their reference values, with attitude angles maintained within $\pm 2°$ and final position errors below 5 cm, confirming the effectiveness of the control strategy.

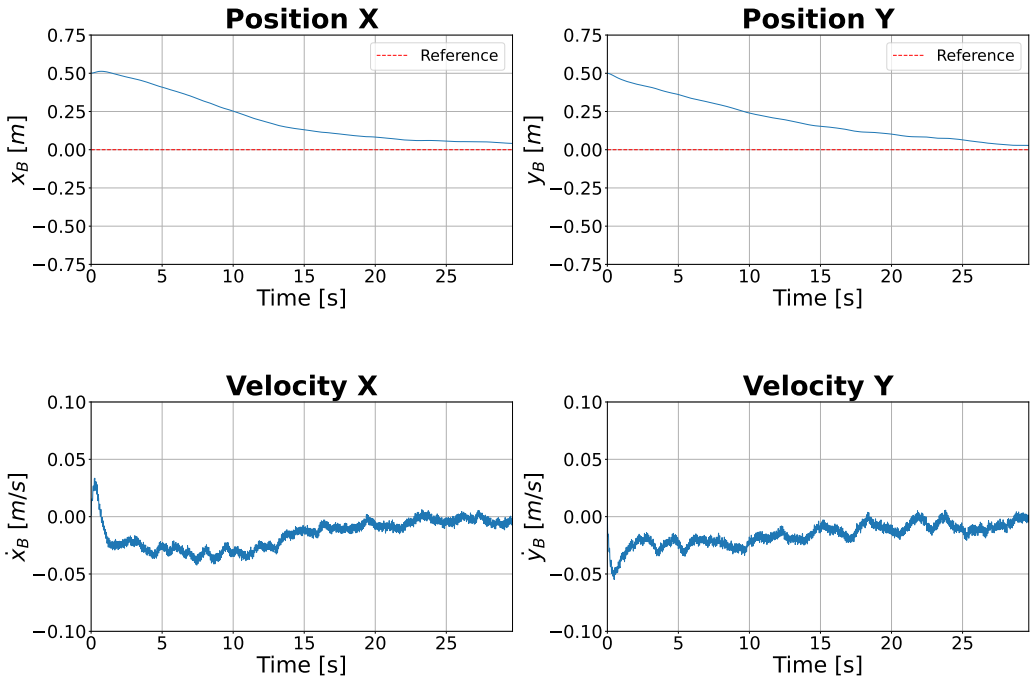


FIGURE 4.10 – Position and velocity output for the combined initial position and attitude disturbance rejection test.

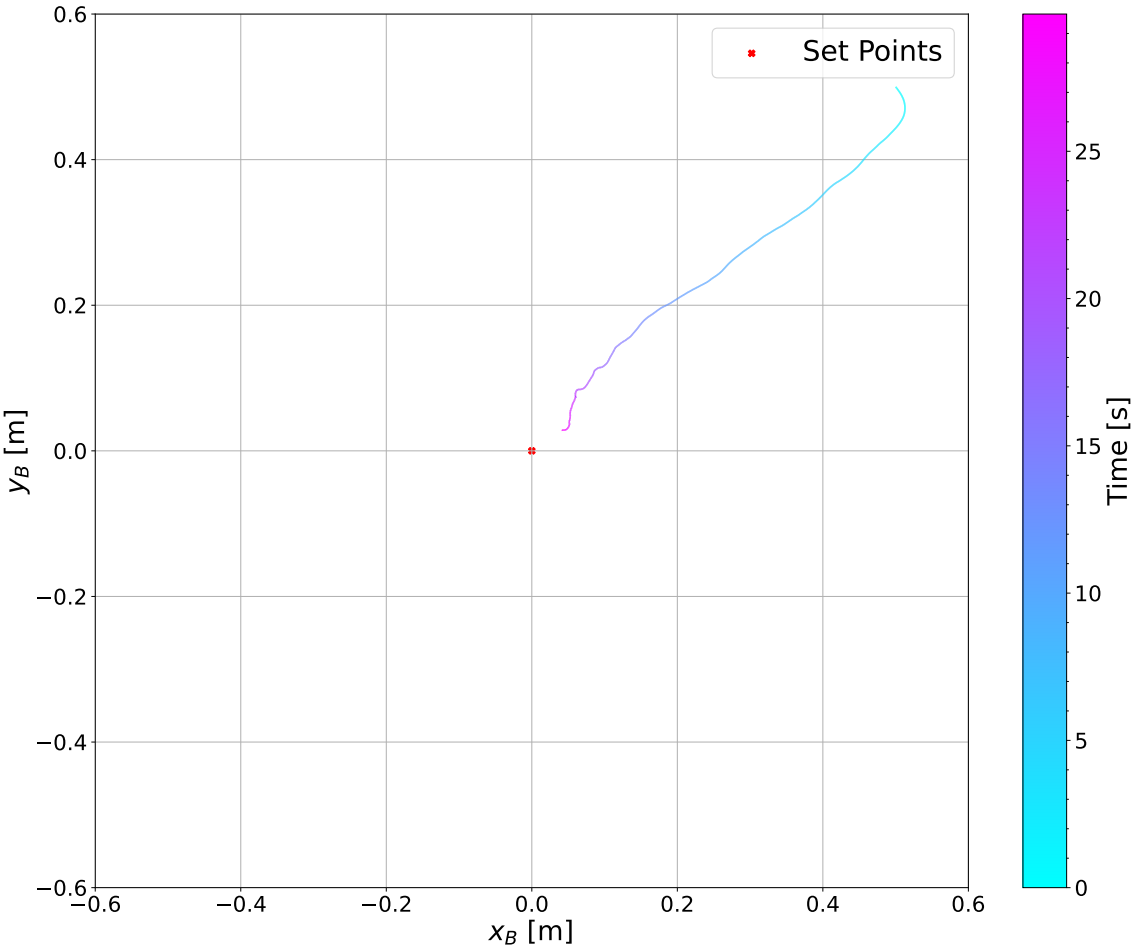


FIGURE 4.11 – X–Y trajectory for the combined initial position and attitude disturbance rejection test.

4.1.4 Path-Following Along the X Direction

To validate the translational dynamics in simulation, a sequence of position references along the x-axis is commanded to the robot. Each subsequent reference is issued once the current target is reached within a tolerance radius of 5 cm.

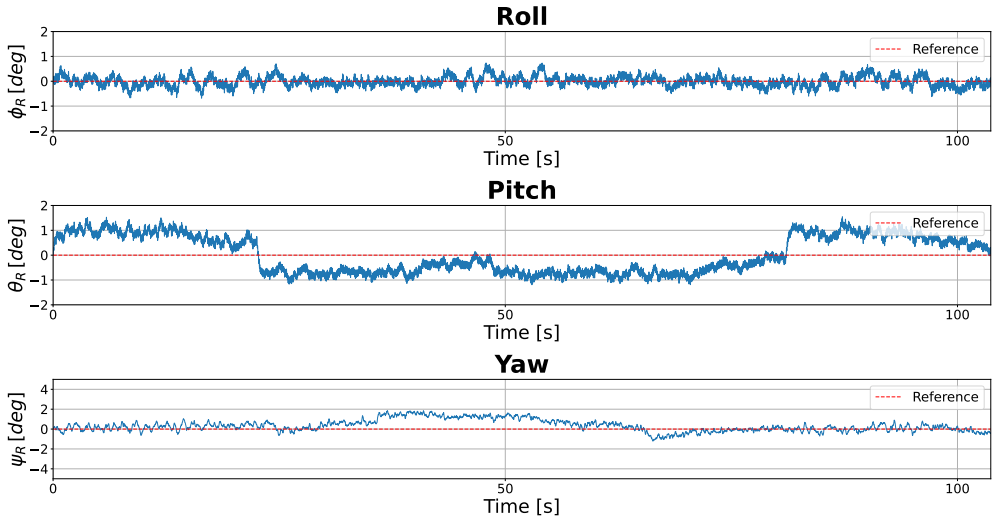


FIGURE 4.12 – Attitude angles for the simulated path-following experiment along the x direction.

Figure 4.12 demonstrates excellent attitude regulation throughout the trajectory, with roll and pitch angles remaining within ±2°. As shown in Figures 4.13 and 4.14, the robot accurately tracks the commanded sequence along the x-axis while maintaining the y-position error below 5 cm.

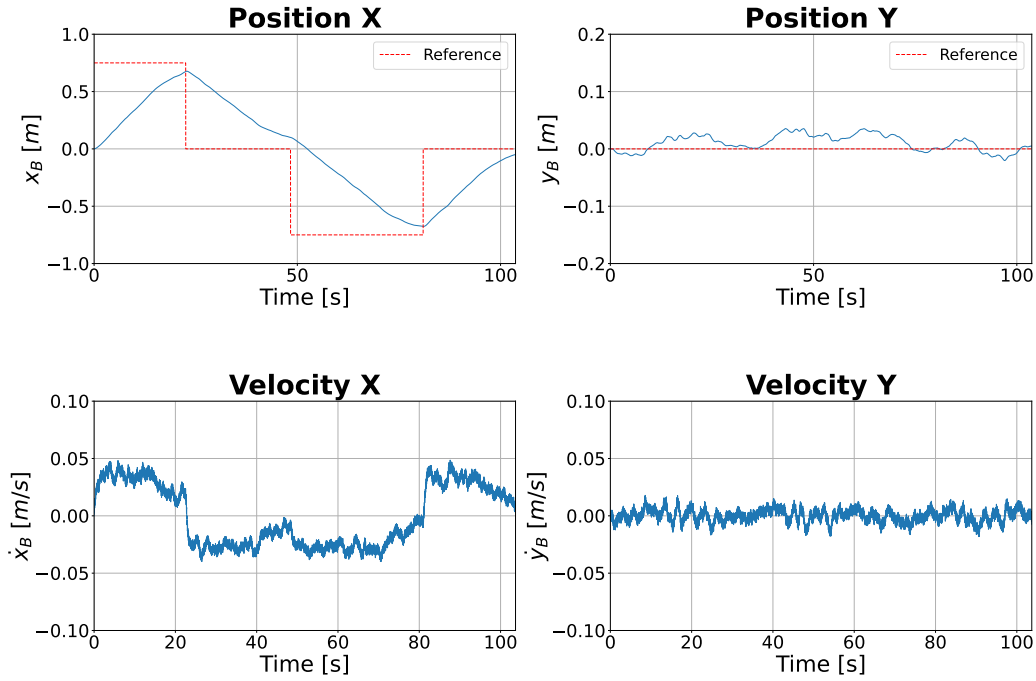


FIGURE 4.13 – Position and velocity output for the simulated path-following experiment along the x direction.

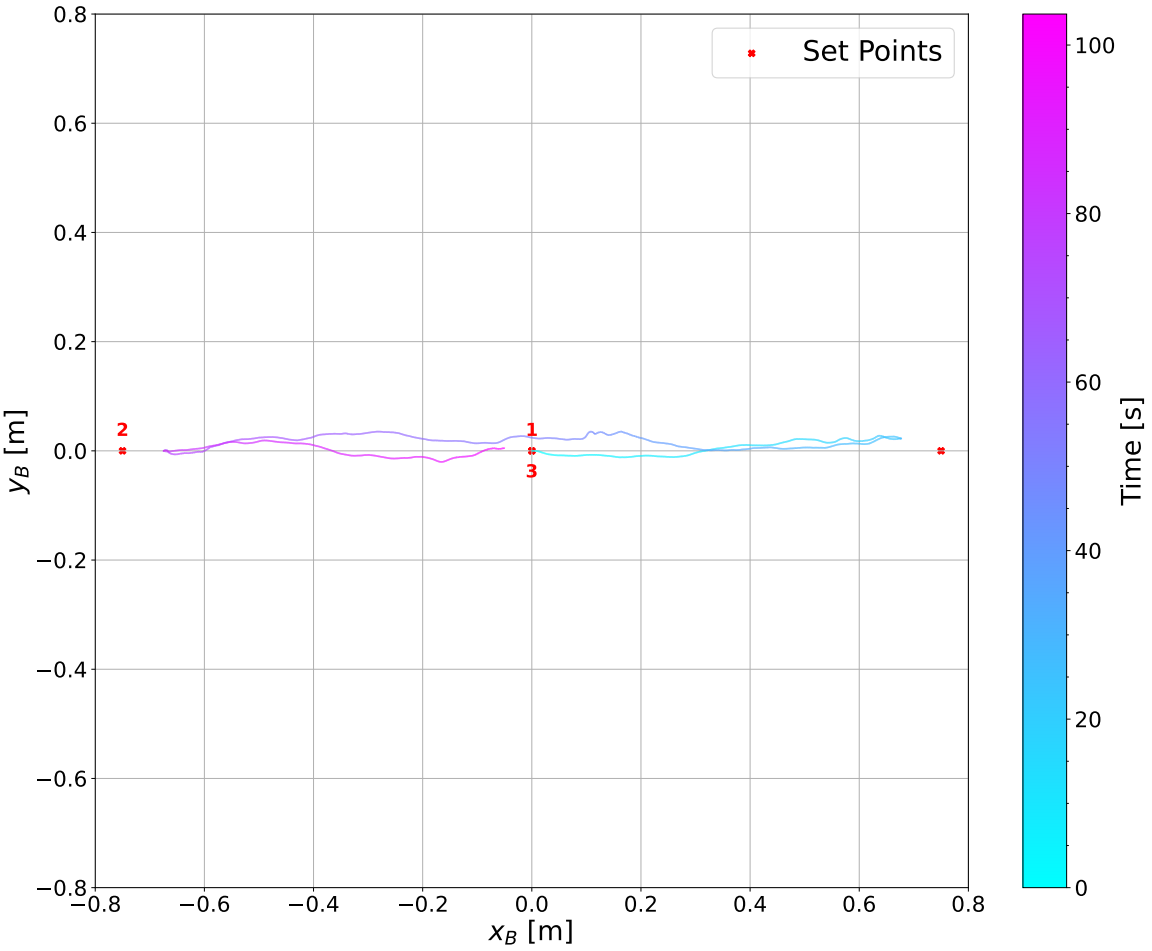


FIGURE 4.14 – X–Y trajectory for the simulated path-following experiment along the x direction.

4.1.5 Path-Following Along the Y Direction

To verify the controller's performance along both x and y axes, a similar experiment is performed along the y-axis. The results for a sequence of four target positions are presented in Figures 4.15, 4.16, and 4.17.

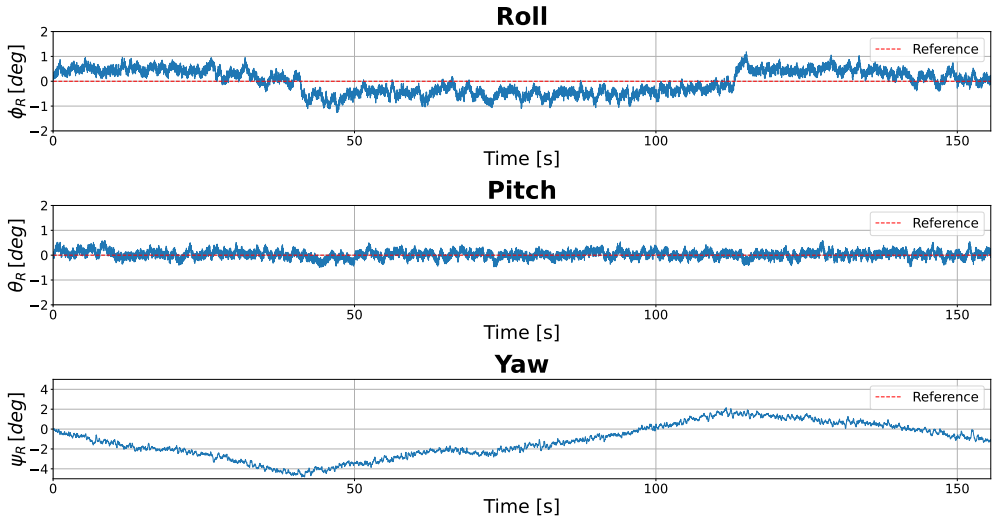


FIGURE 4.15 – Attitude angles for the simulated path-following experiment along the y direction.

The simulation results shows similar performance along both x and y axes, with attitude angles consistently maintained within $\pm 2°$ and position errors below 5 cm.

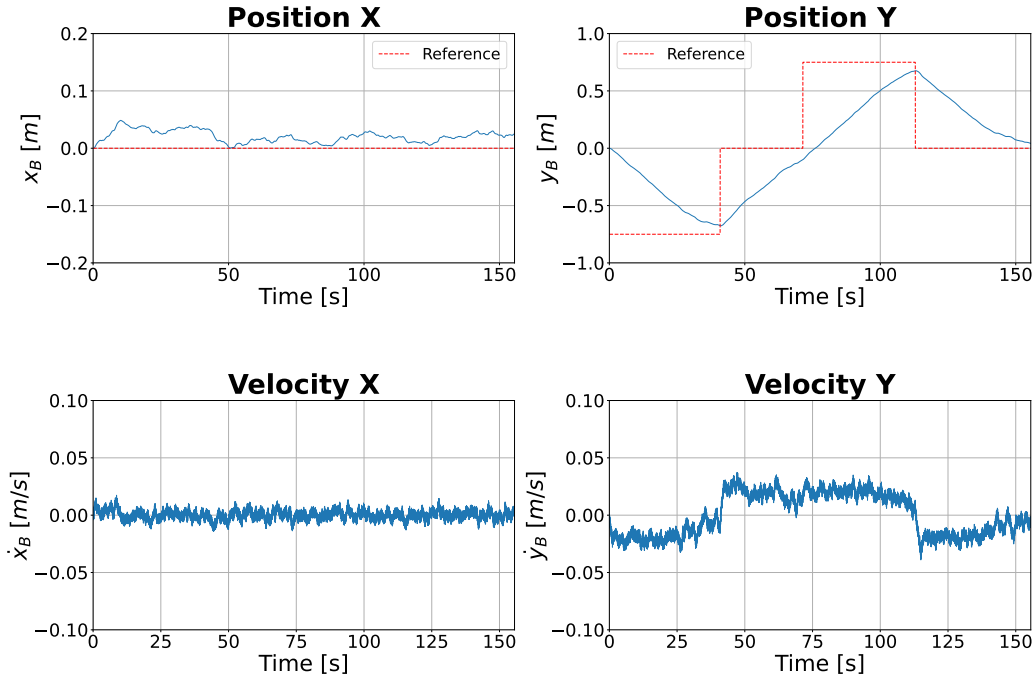


FIGURE 4.16 – Position and velocity output for the simulated path-following experiment along the y direction.

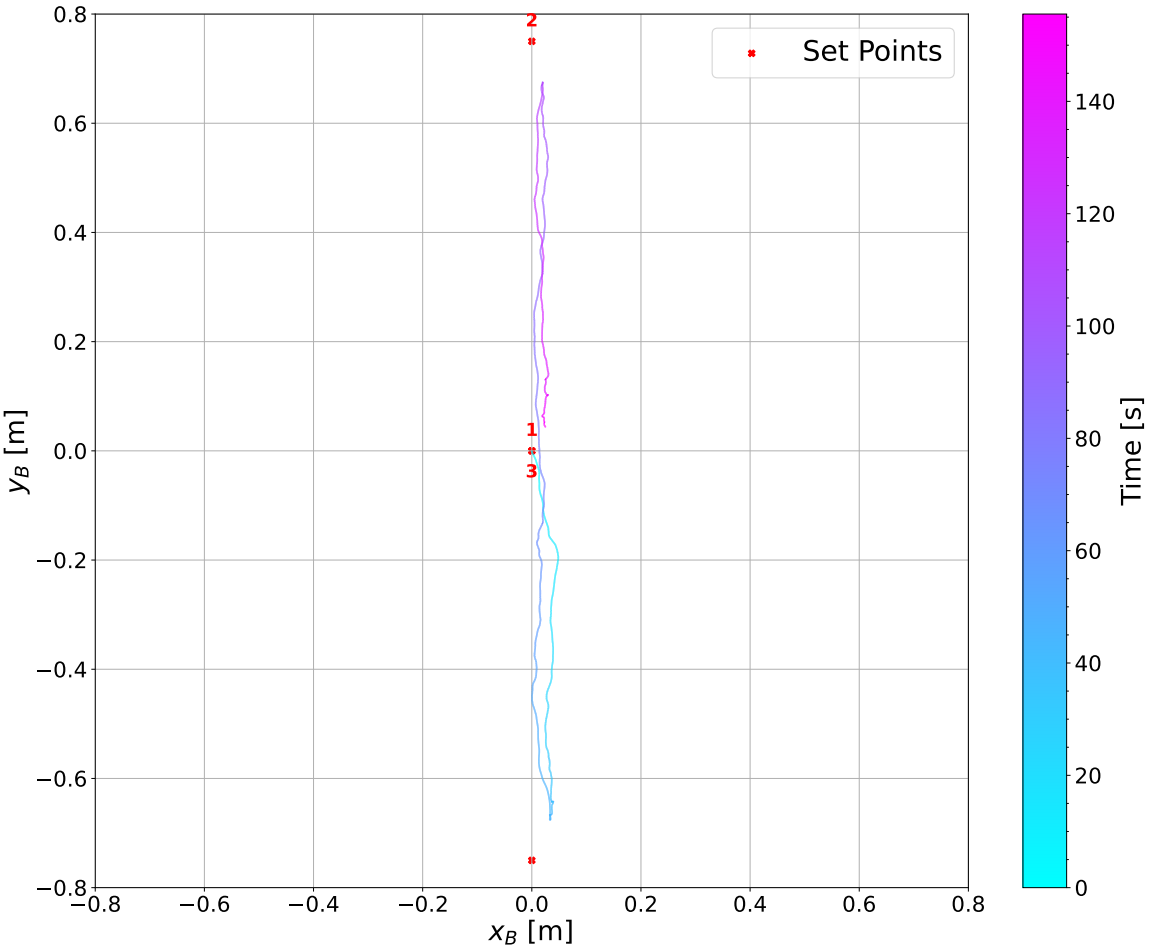


FIGURE 4.17 – X–Y trajectory for the simulated path-following experiment along the y direction.

4.1.6 Path-Following Along a Rectangular Trajectory

To evaluate two-dimensional path-following, the robot is commanded to travel through a rectangular path with seven sequential target points, ultimately returning to the initial position.

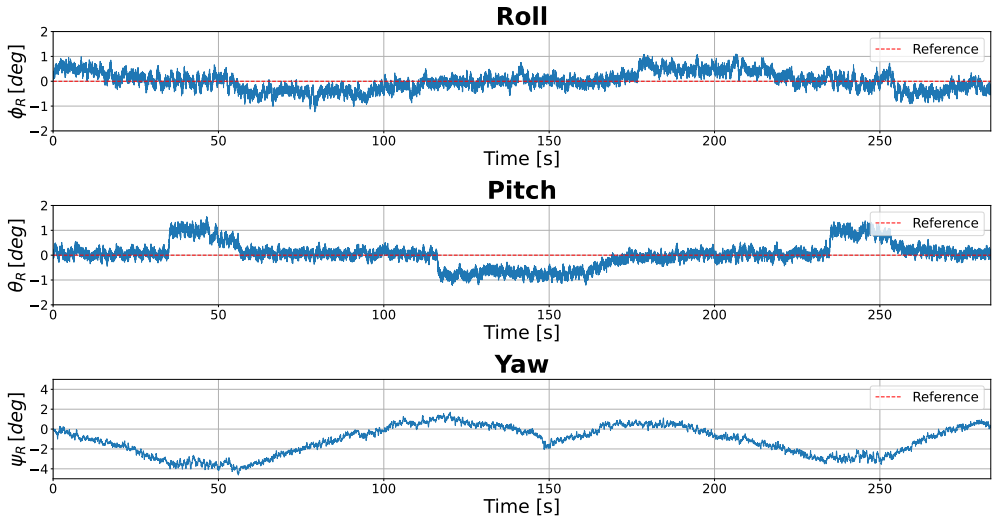


FIGURE 4.18 – Attitude angles for the simulated rectangular trajectory tracking experiment.

Figures 4.18, 4.19, and 4.20 demonstrate that the controller accurately follows the sequence of points defining the rectangular trajectory. The attitude angles remain within $\pm 2°$ throughout the entire path, while position errors stay below 5 cm.

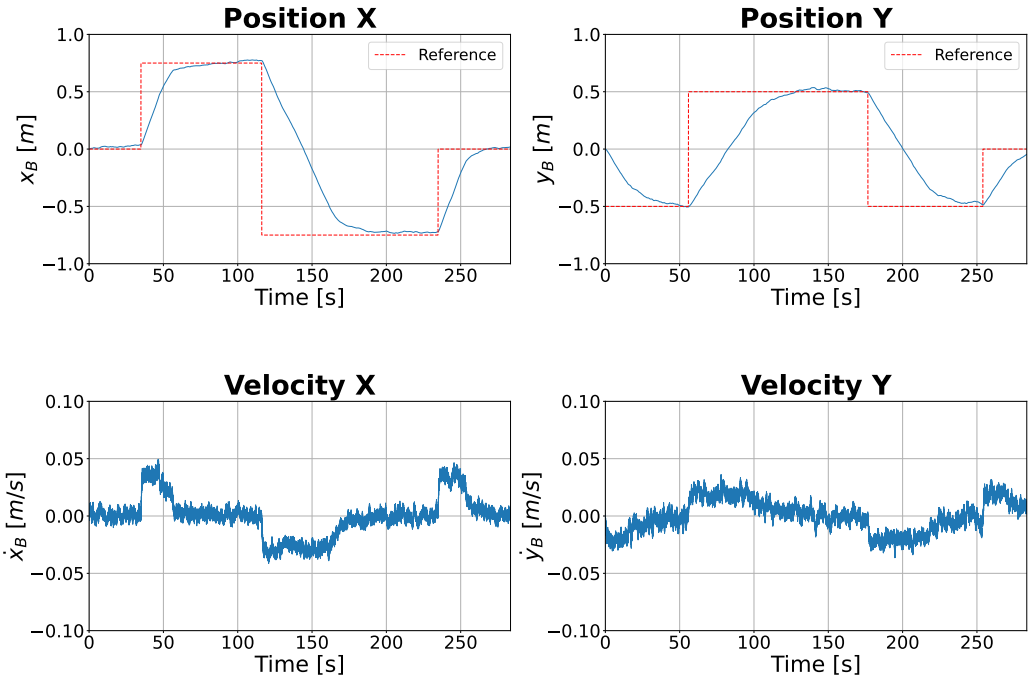


FIGURE 4.19 – Position and velocity output for the simulated rectangular trajectory tracking experiment.

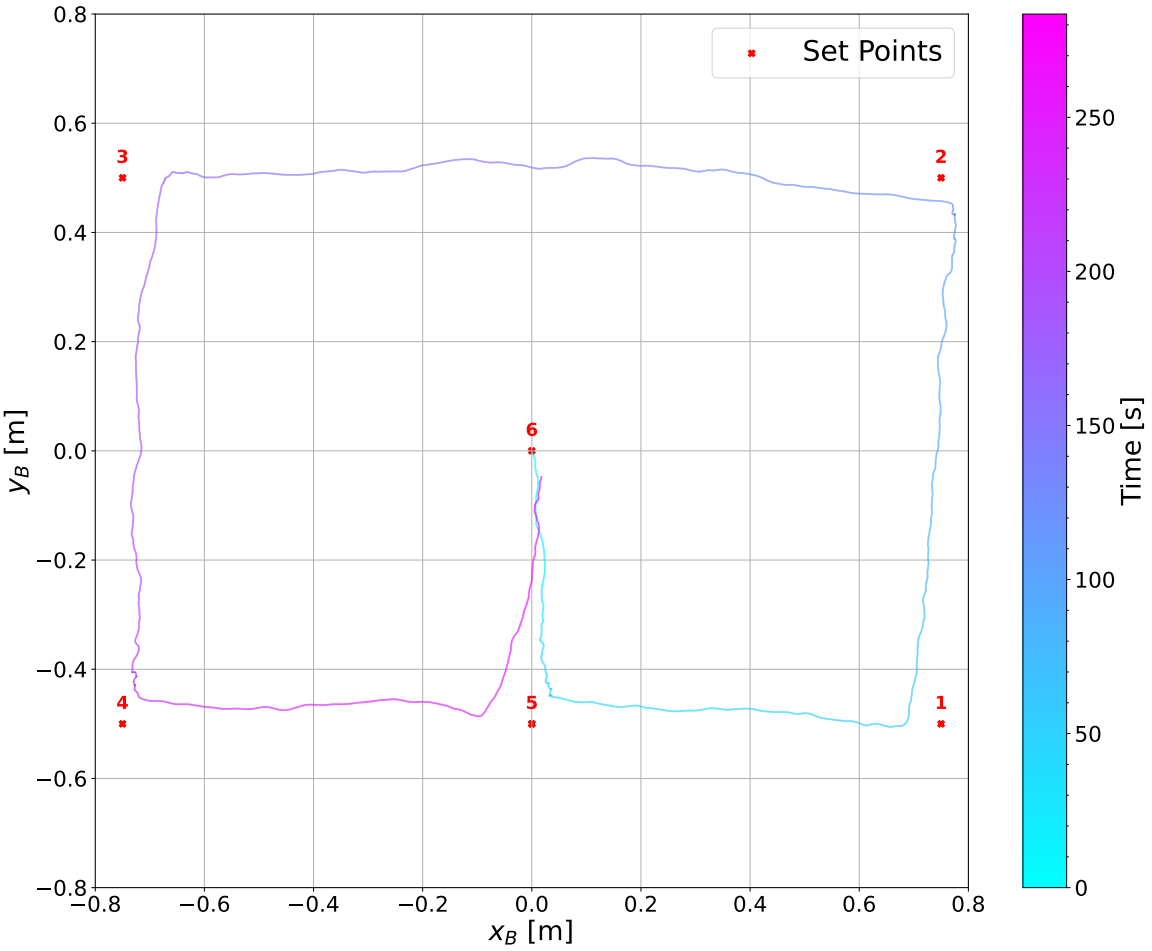


FIGURE 4.20 – X–Y trajectory for the simulated rectangular trajectory tracking experiment.

4.2 Experimental Results

The experiments conducted on the physical prototype are designed to validate the control framework and evaluate the Ballbot system's real-world performance. The control gains initially derived from the LQR design are further fine-tuned to improve response characteristics and compensate for unmodeled dynamics. The resulting gain matrix $\boldsymbol{K}$, used for both the Y–Z and X–Z plane controllers, is given in Equation (4.1). The Ballbot prototype used in the experiments is shown in Figure 4.21. Videos of the experimental results are available at the project's GitHub repository: `https://github.com/Intelligent-Machines-Lab/Ballbot_LMI_V2/tree/main/Experimental%20Results/Videos`.

$$\boldsymbol{K} = \begin{bmatrix} -0.35 & -2.8 & -1.1 & -0.4 \end{bmatrix} \tag{4.1}$$

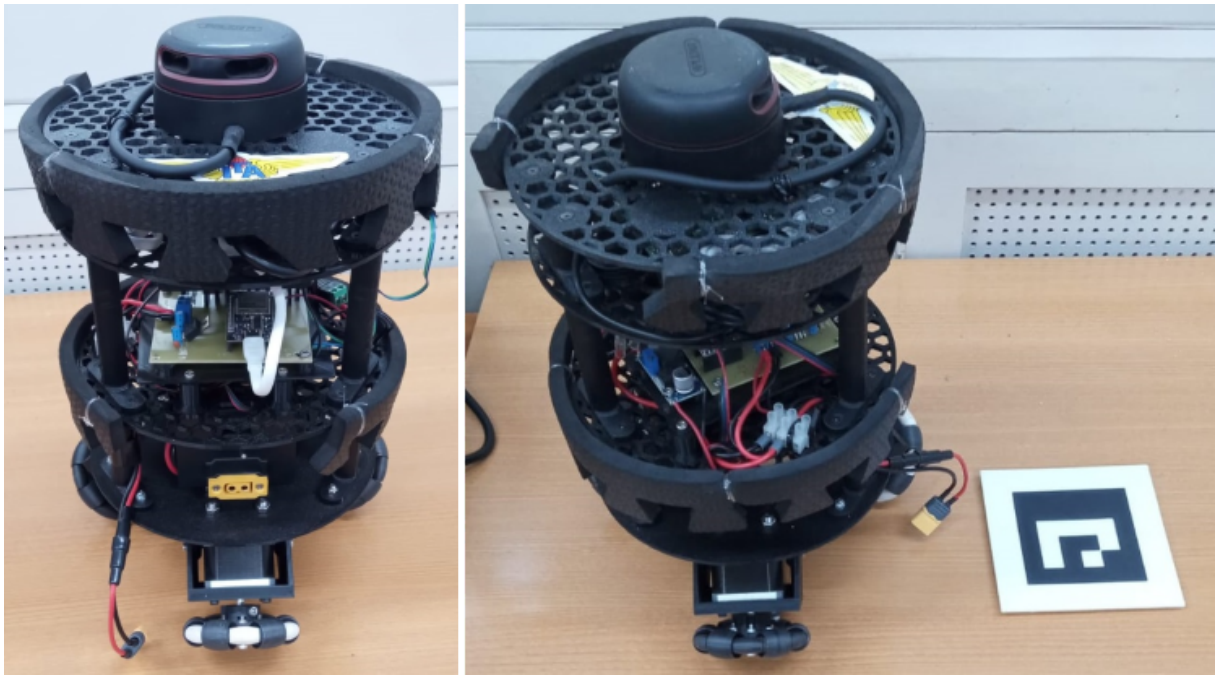


FIGURE 4.21 – Front and side view of the physical Ballbot prototype used for experimental validation.

4.2.1 Camera-Based Experiments

In the camera-based experimental setup, an ArUco marker is mounted on the top section of the Ballbot, while an external camera is positioned above the test area, parallel to the ground plane. The test area measures 2 m in length and 1.5 m in width. Figure 4.22 presents the camera view of the experimental setup, showing the test area and the Ballbot. Several experimental scenarios are evaluated in order of increasing complexity and are presented in the following sections.

FIGURE 4.22 – Camera-based experimental setup showing the overhead camera view of the test area and the Ballbot with the mounted ArUco marker.

4.2.1.1 Balance

To evaluate the controller's capability to stabilize the inherently unstable Ballbot, the LQR gains are computed without considering the position states. This is achieved by assigning near-zero weights to the position-related elements in the weighting matrix $\boldsymbol{Q}$, which results in the corresponding element of the feedback gain matrix $\boldsymbol{K}$ being effectively null. This configuration isolates the balance control behavior, allowing a focused assessment of the attitude stabilization performance. The PD controller is used to controll the yaw attitude, ensuring that the yaw angle remains close to 0°. The results showing the attitude angles, the position, velocity and wheel angular velocities, and the robot's trajectory in the X–Y plane are shown in Figures 4.23, 4.24 and 4.25, respectively.

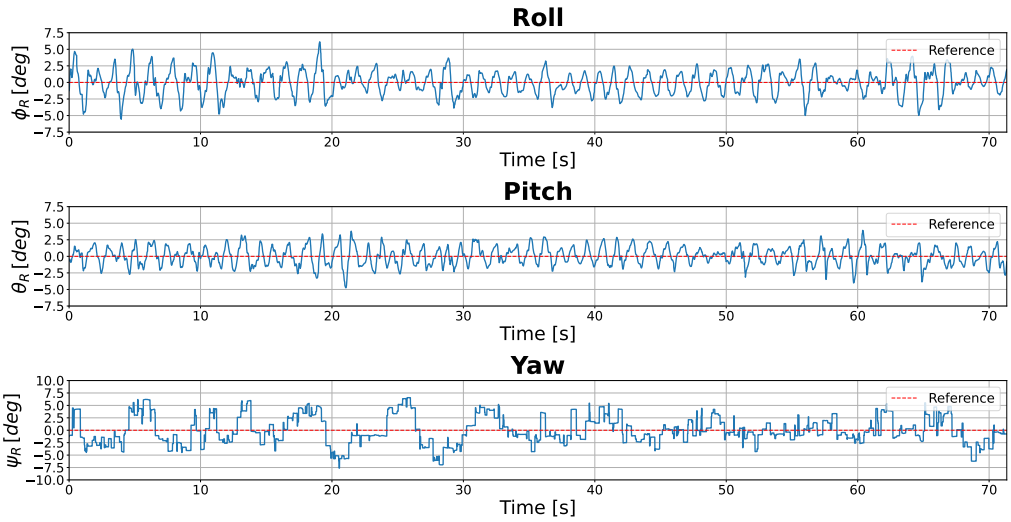


FIGURE 4.23 – Attitude angles for the balance experiment.

Figure 4.23 demonstrates that the controller effectively stabilizes the Ballbot, keeping roll and pitch angles within ±5° and maintaining the yaw angle within a ±10° range. Although the controller does not actively regulate the position, Figures 4.24 and 4.25 show that the robot remains near its initial location, with deviations of approximately 20 cm along both x and y axis.

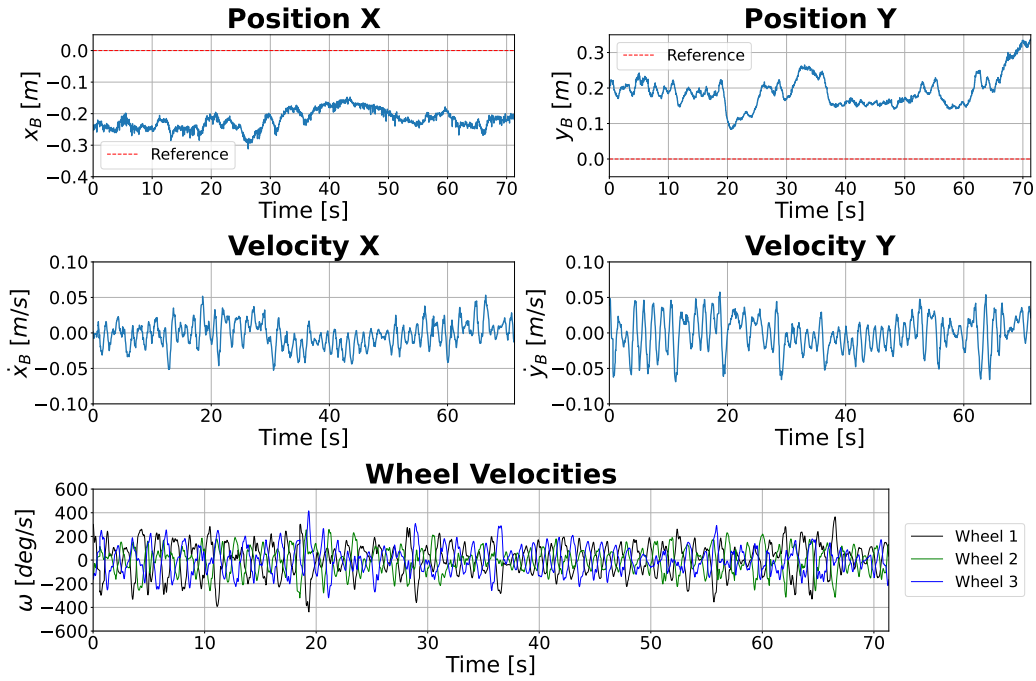


FIGURE 4.24 – Position, velocities and actuators outputs for the balance experiment.

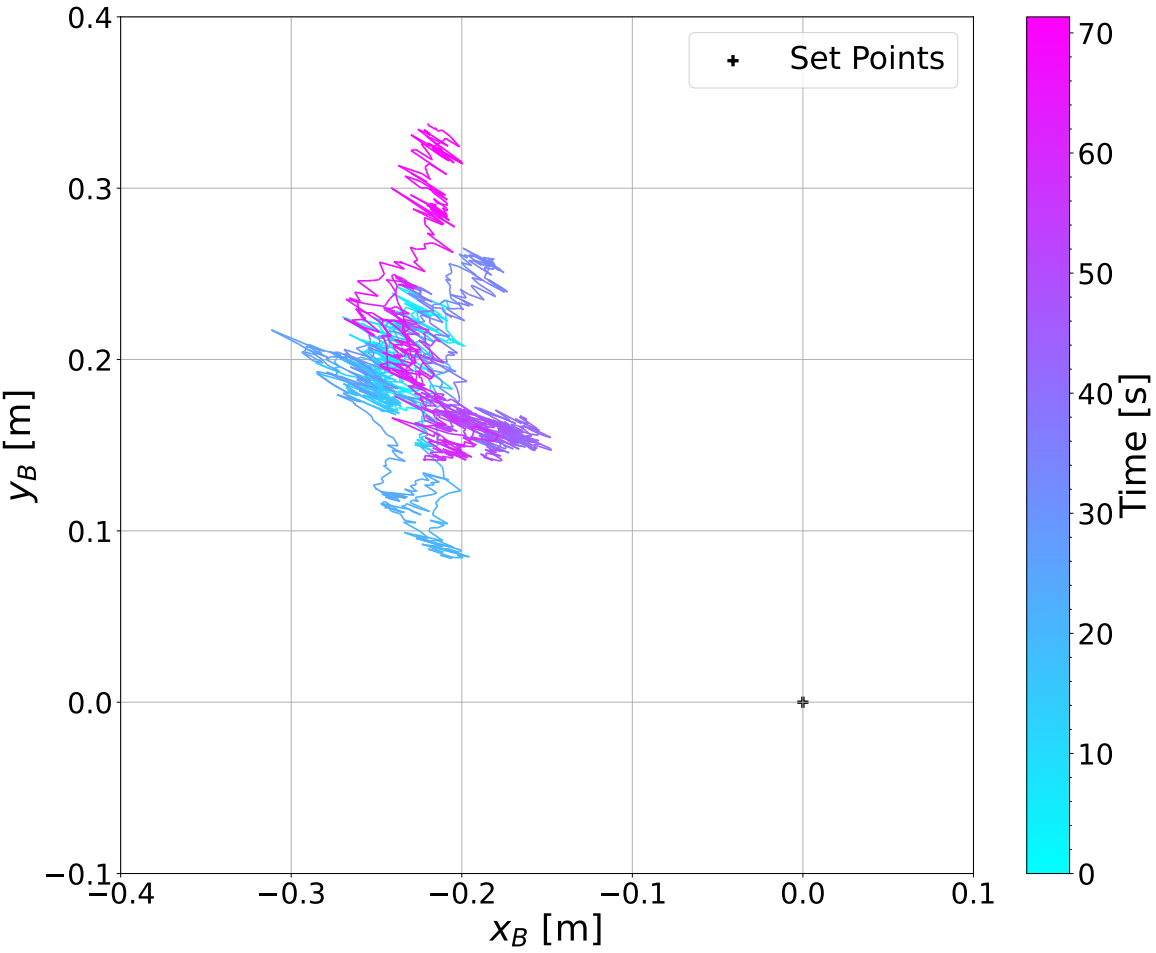


FIGURE 4.25 – X–Y trajectory for the balance experiment.

4.2.1.2 Station-Keeping

For the station-keeping and following experiments, the full $\boldsymbol{K}$ matrix is employed. Figures 4.26, 4.27, and 4.28 show results similar to those obtained in the balance experiment, with roll and pitch angles remaining within $\pm 5°$ and yaw maintained within $\pm 10°$. Although the LQR controller now actively regulates the robot's position, it cannot fully eliminate the initial position error, resulting in a steady-state position offset, with the robot oscilating around the $x = -0.25$ and $y = -0.1$ positions.

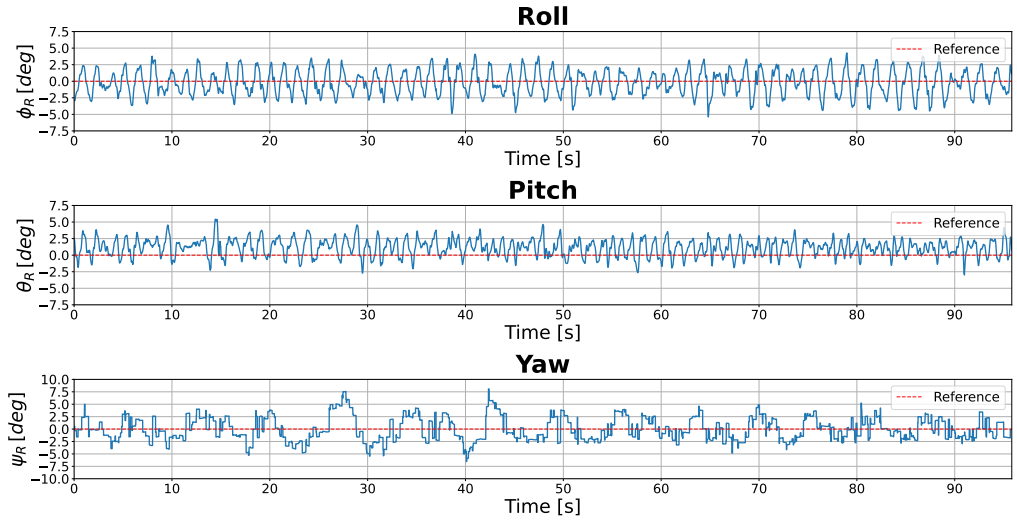


FIGURE 4.26 – Attitude angles for the station-keeping experiment

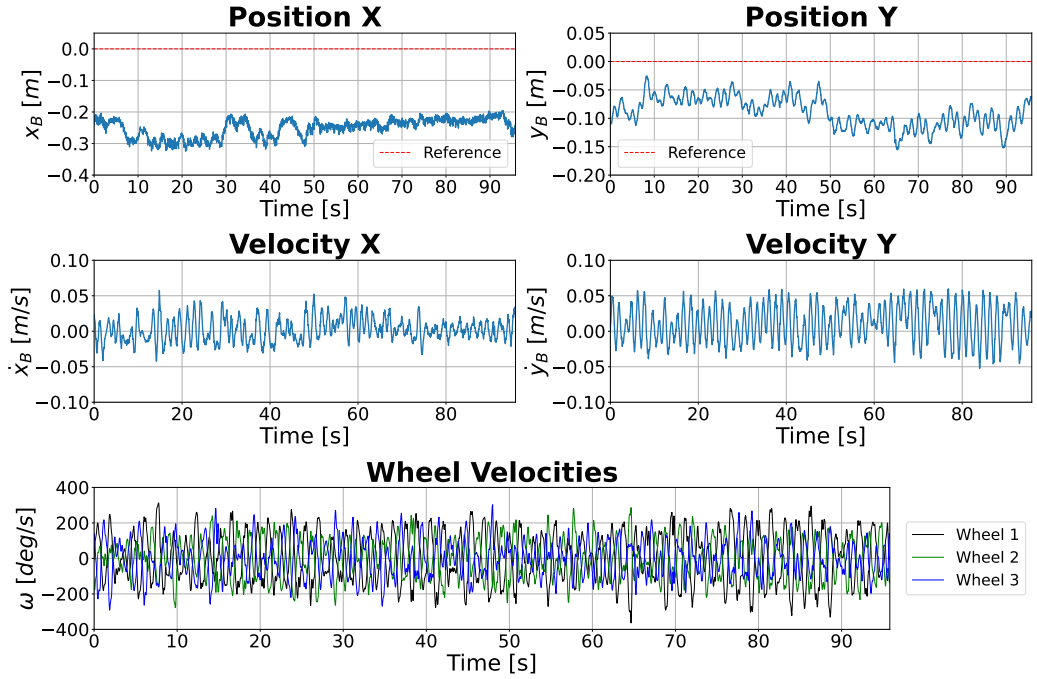


FIGURE 4.27 – Position, velocities and actuators outputs for the station-keeping experiment.

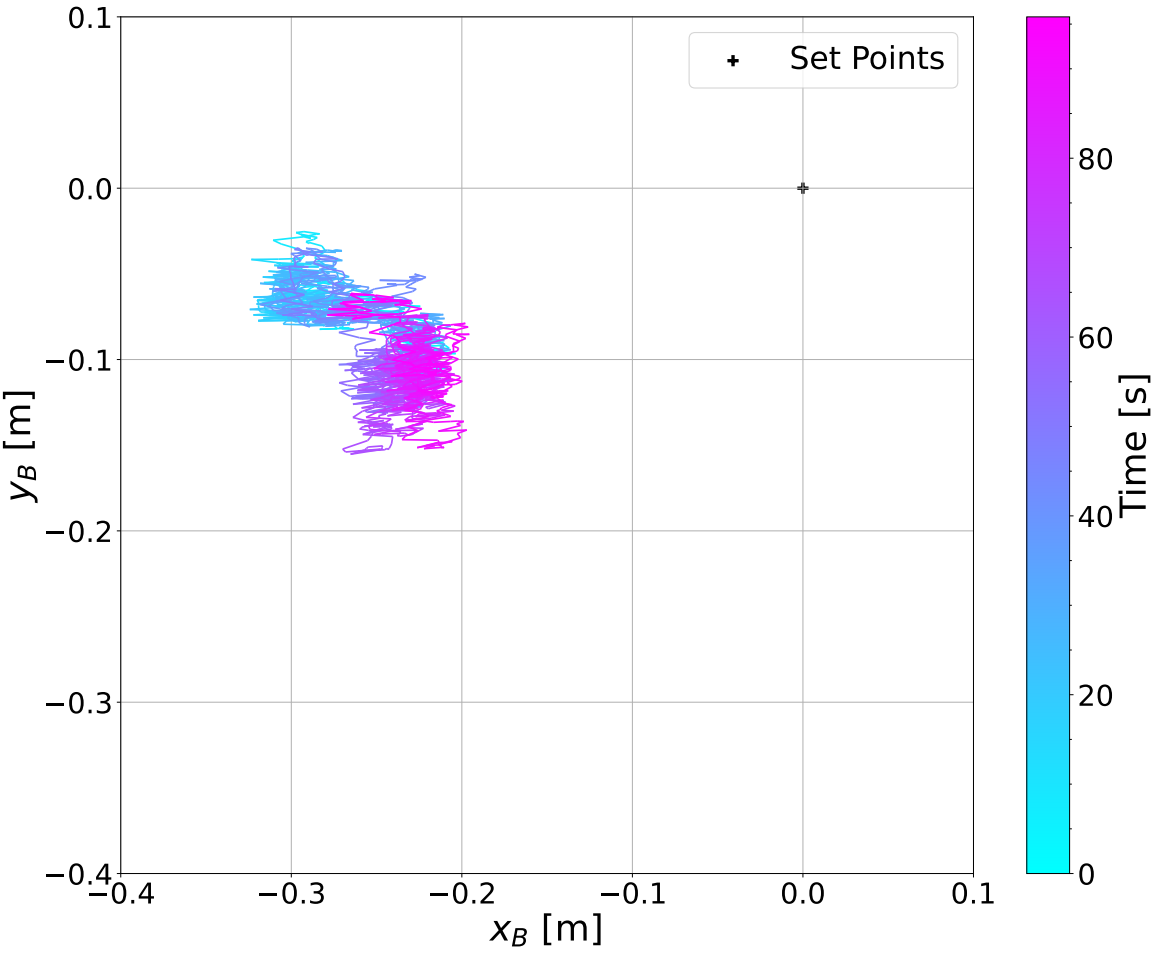


FIGURE 4.28 – X–Y trajectory for the station-keeping experiment.

To compensate for the steady-state position error, an integral action on the position errors is added to the LQR controller, with the integral gain $K_i = 0.15$. Figures 4.29, 4.30, and 4.31 show that the integrator successfully reduces the position error while maintaining the robot's roll, pitch, and yaw within the ranges achieved by the original controller.

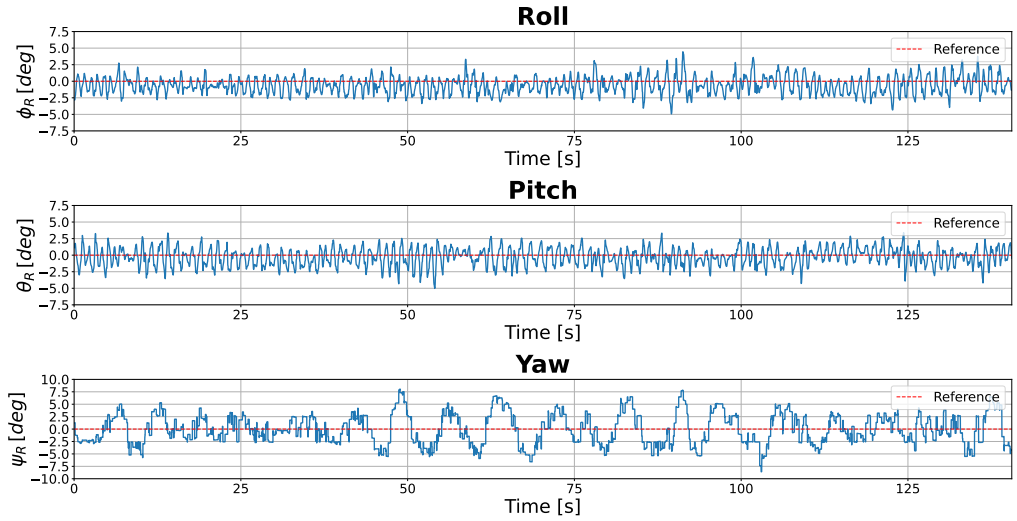


FIGURE 4.29 – Attitude angles for the station-keeping experiment employing the LQR controller augmented with integral action.

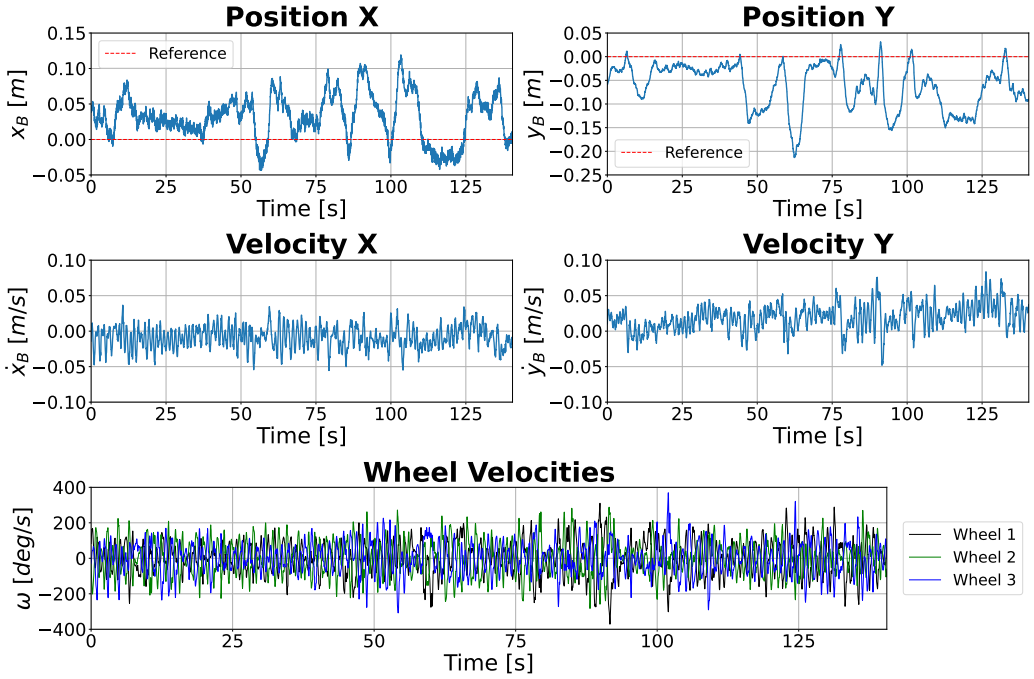


FIGURE 4.30 – Position, velocities and actuators outputs for the station-keeping experiment employing the LQR controller augmented with integral action.

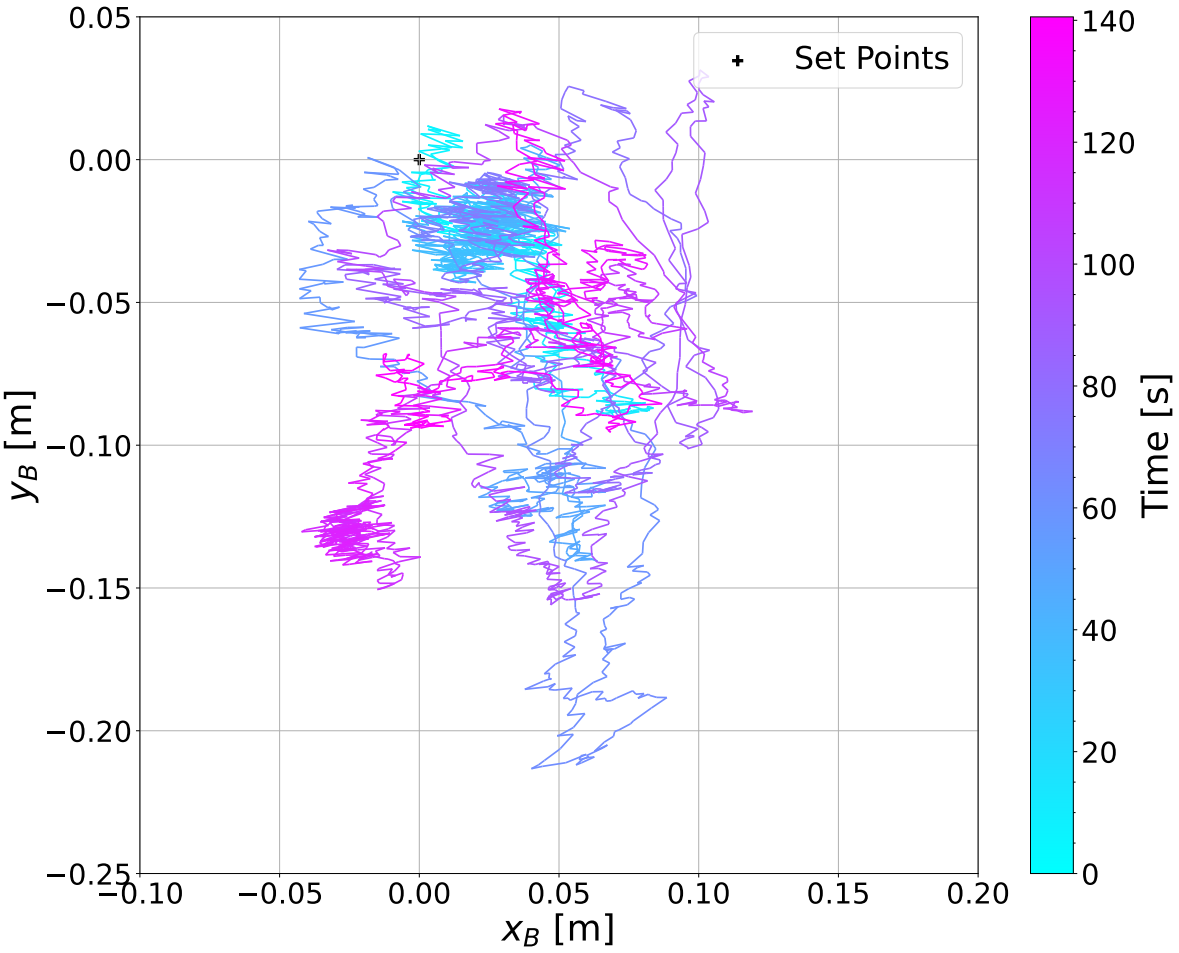


FIGURE 4.31 – X–Y trajectory for the station-keeping experiment employing the LQR controller augmented with integral action.

4.2.1.3 Path-Following Along The X Direction

To evaluate the translational dynamics, a sequence of position references is sent to the robot. Each subsequent reference is issued once the current target is reached within a tolerance radius of 5 cm. The results for a sequence of four points on the x axis are shown in Figures 4.32, 4.33 and 4.34.

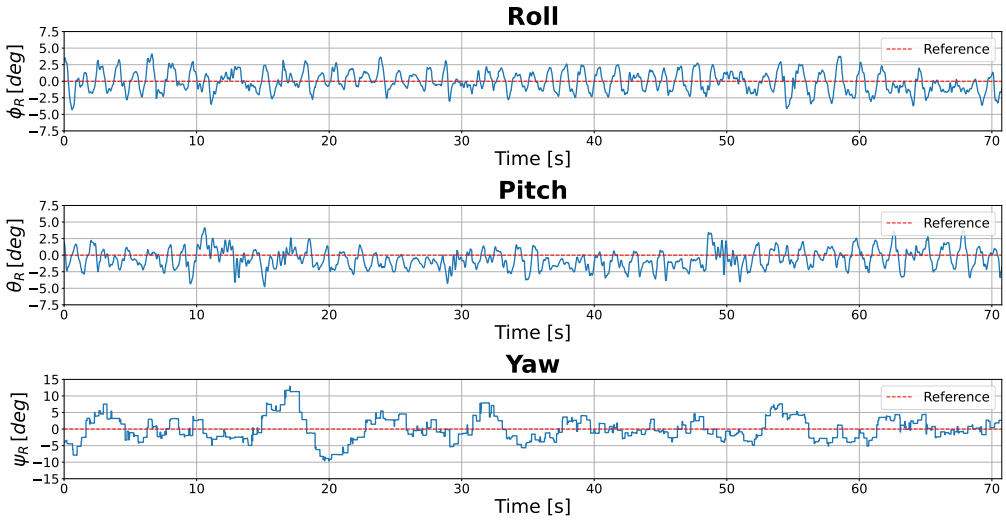


FIGURE 4.32 – Attitude angles for the path-following experiment for a sequence of points along the x direction.

Figures 4.33 and 4.34 show that the sequence of input references along the x-axis is completed in under 40 seconds, while the y-position deviation remains within a 20 cm region around the center position.

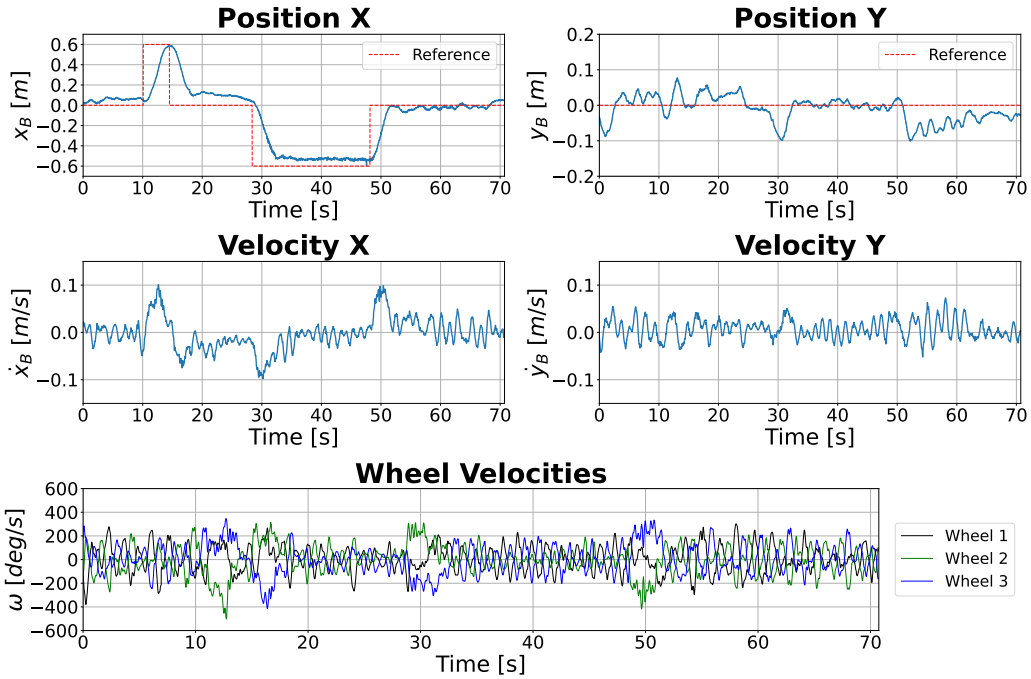


FIGURE 4.33 – Position, velocities and actuators outputs for the path-following experiment with a sequence of points along the x direction.

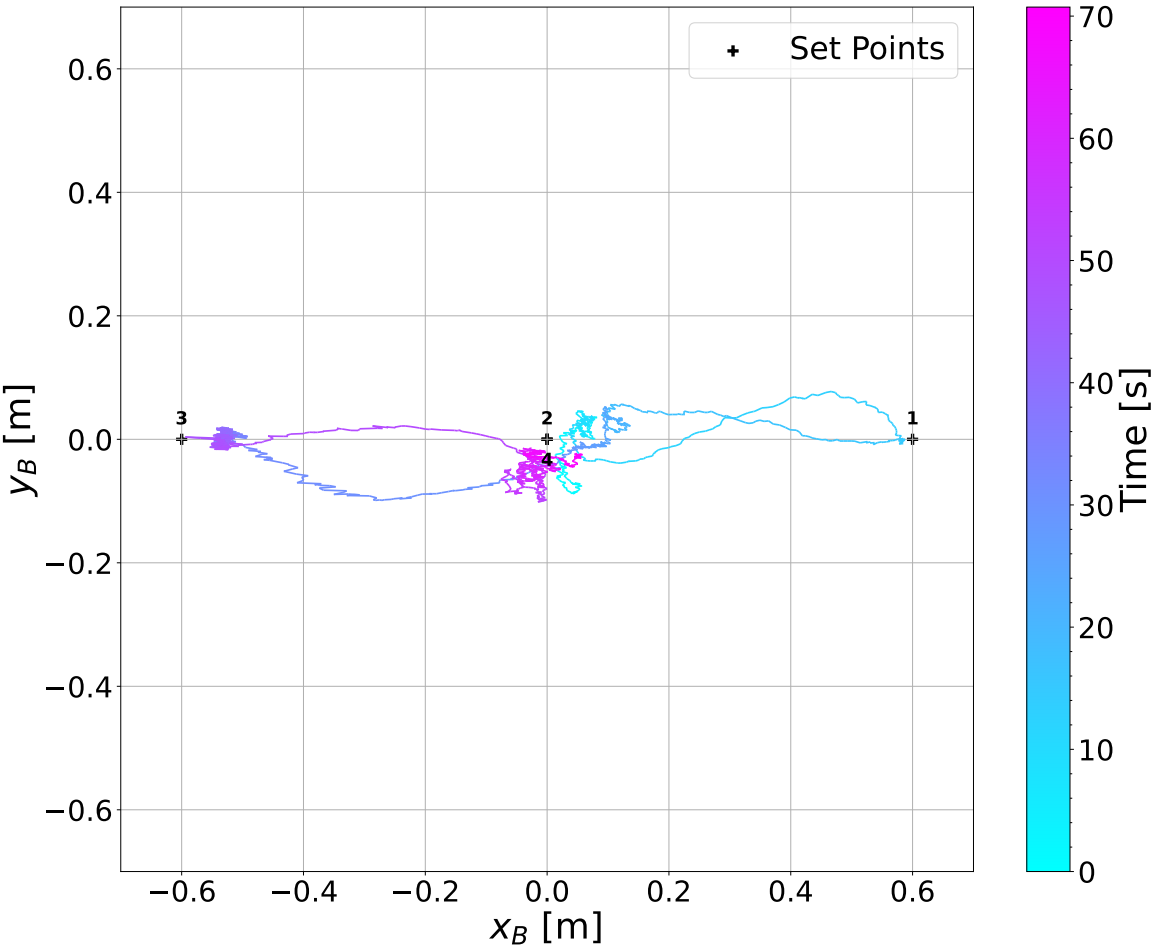


FIGURE 4.34 – X–Y trajectory for the path-following experiment with a sequence of points along the x direction.

4.2.1.4 Path-Following Along The Y Direction

To further assess the controller's capability to track position references, a similar experiment is performed along the y-axis. The results for a sequence of four target positions along this axis are presented in Figures 4.35, 4.36, and 4.37.

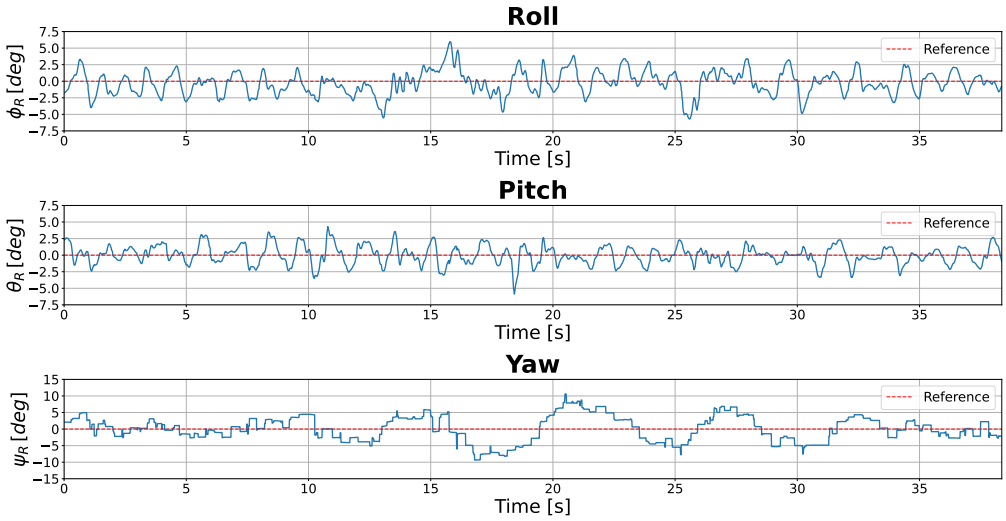


FIGURE 4.35 – Position, velocities and actuators outputs for the path-following experiment with a sequence of points along the y direction.

The sequence of input references is completed in under 20 seconds, as shown in Figures 4.36 and 4.37. Similar to the experiment along the x-axis, the deviation along the perpendicular axis remains within 20 cm.

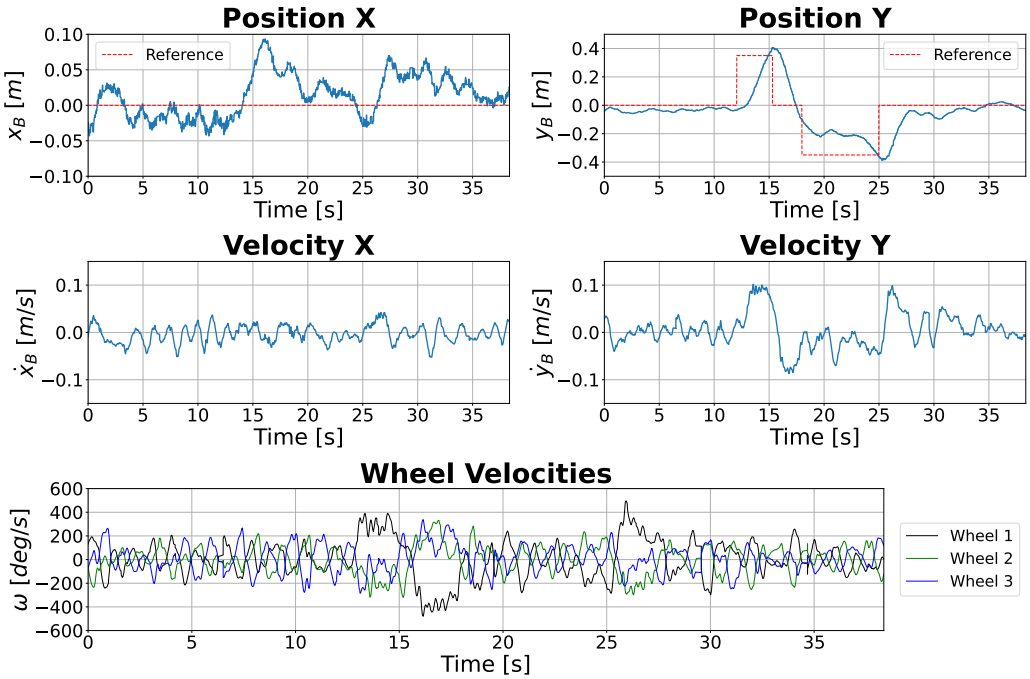


FIGURE 4.36 – Position, velocities and actuators outputs for the path-following experiment with a sequence of points along the y direction.

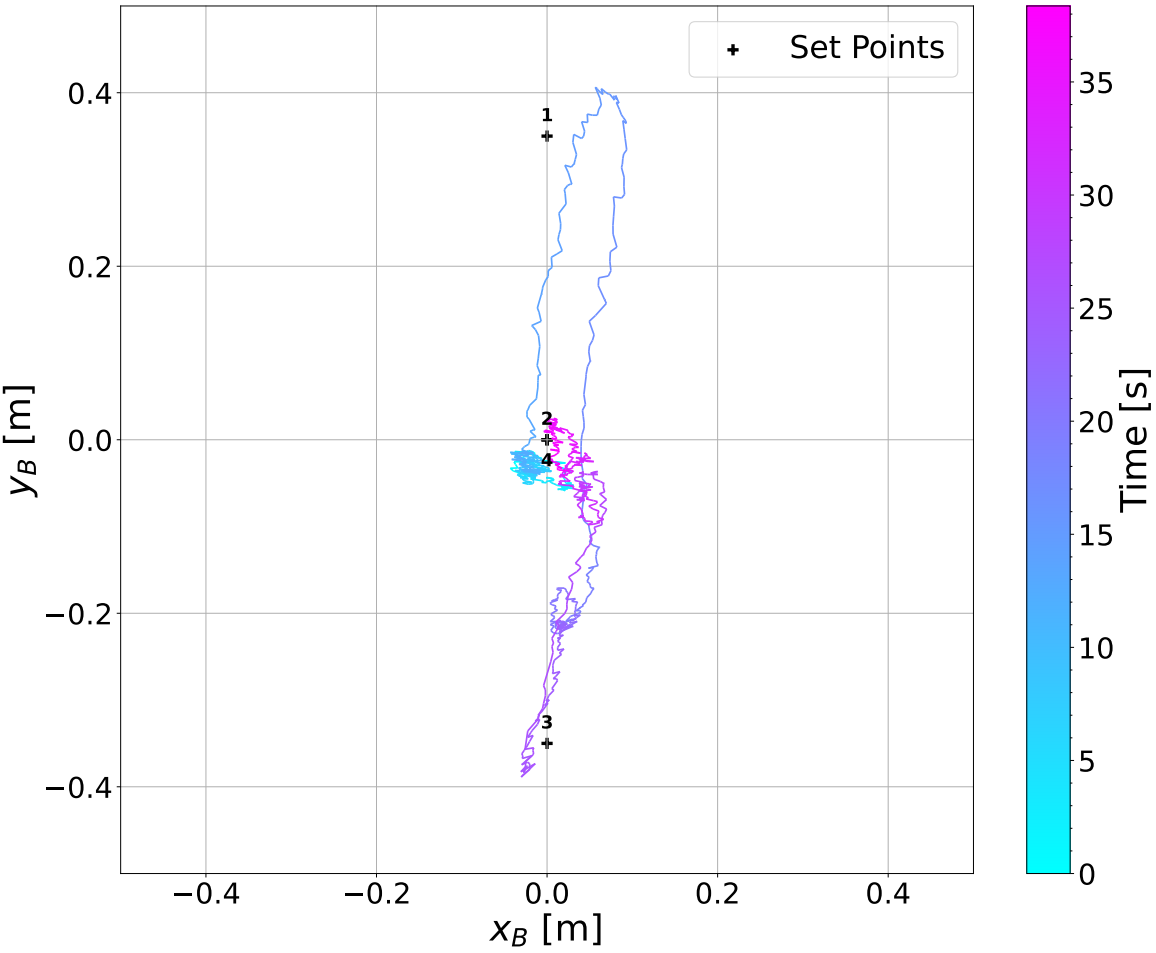


FIGURE 4.37 – X–Y trajectory for the path following experiment with a sequence of points along the y direction.

4.2.1.5 Path-Following Along A Rectangular Trajectory

To evaluate the controller's performance in executing two-dimensional trajectories, a sequence of position references forming a rectangular path is employed. The robot is commanded to move through seven target points and, upon completing the sequence, return to the initial central position.

Figures 4.38, 4.39, and 4.40 show that the sequence of points is completed in under one and a half minutes. Throughout the trajectory, the roll and pitch angles remain within $\pm 6°$, while the yaw angle stays within a $\pm 10°$ range.

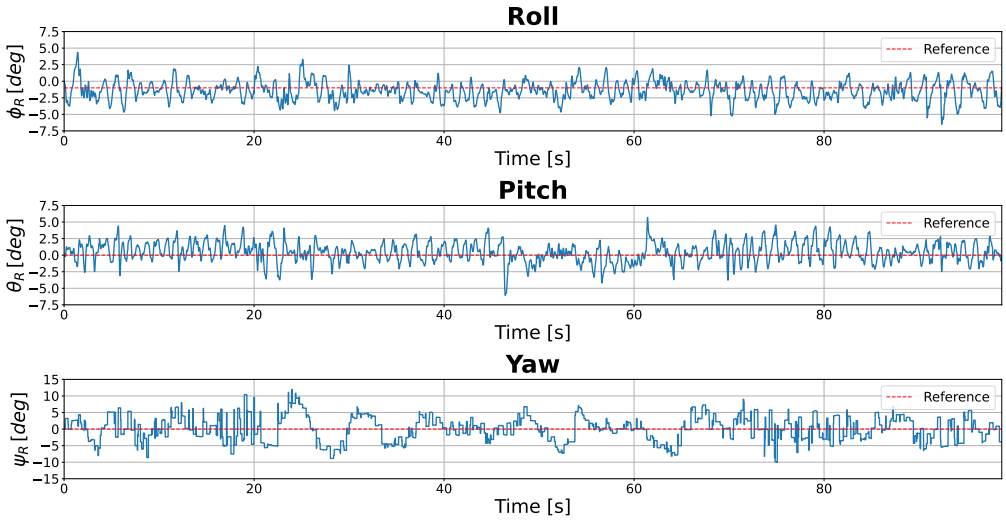


FIGURE 4.38 – Attitude angles for the path-following experiment along the rectangular trajectory.

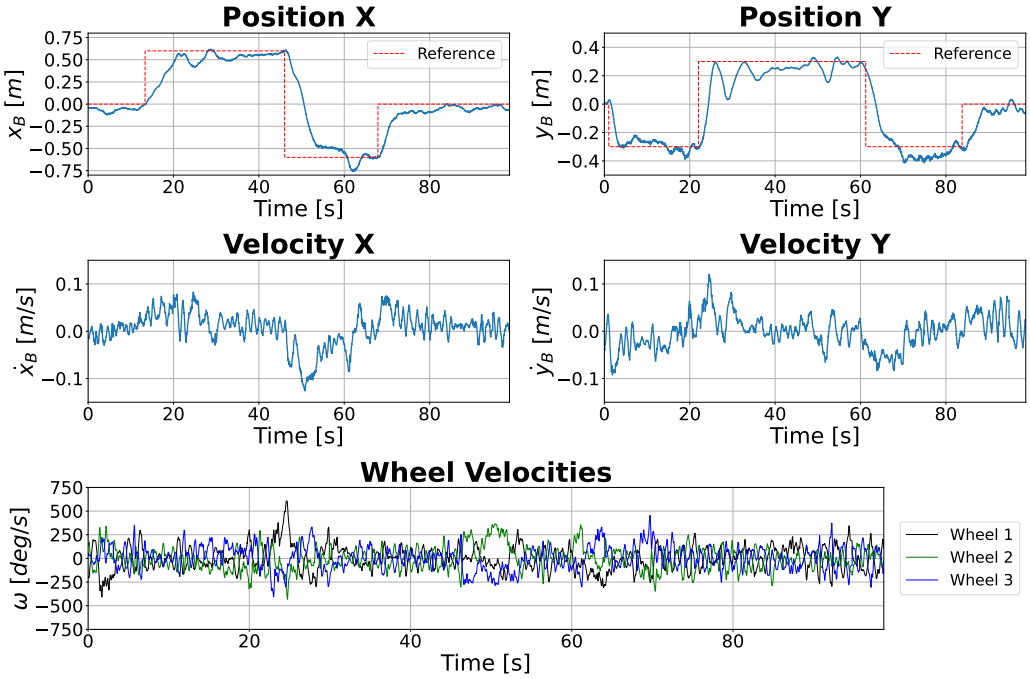


FIGURE 4.39 – Position, velocity, and actuator outputs for the path-following experiment along the rectangular trajectory.

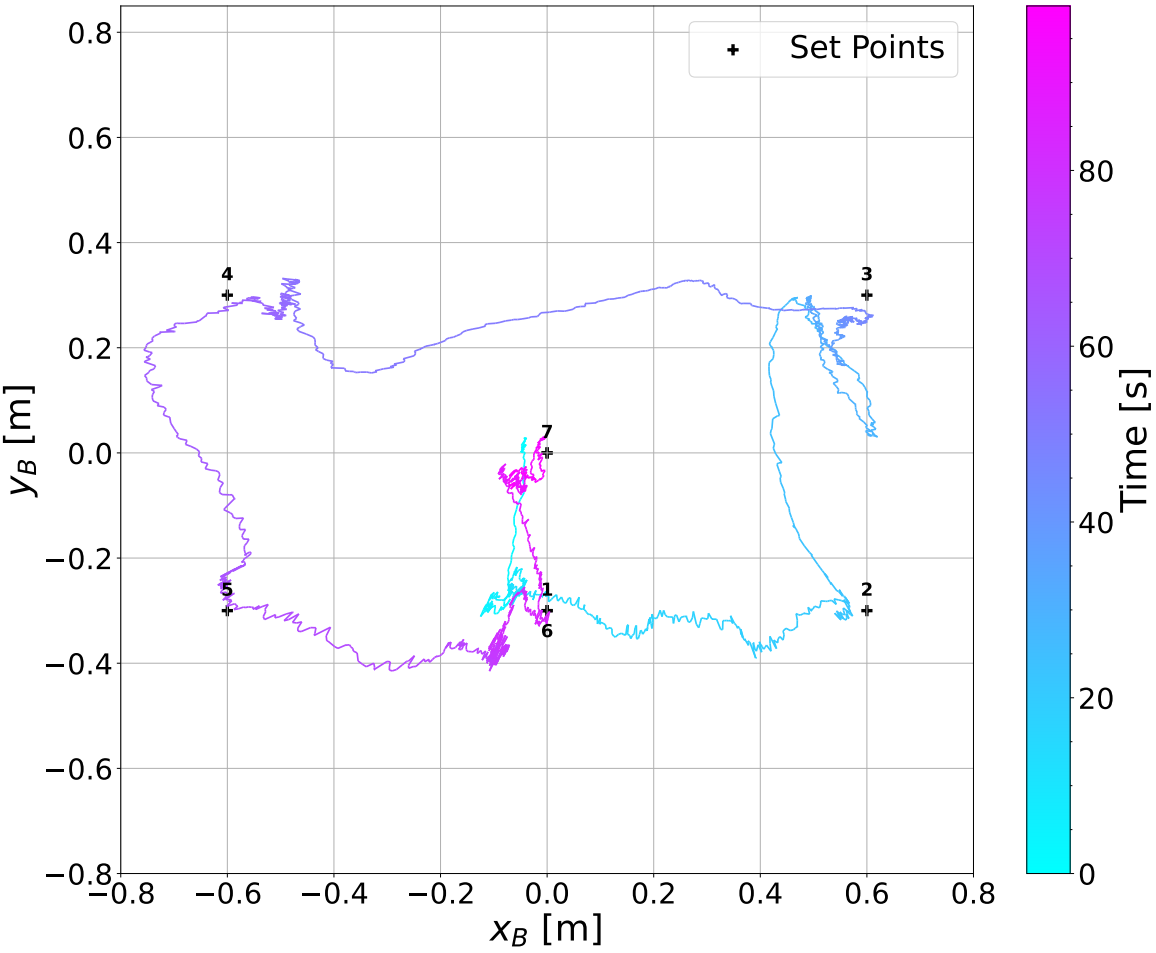


FIGURE 4.40 – X–Y trajectory for the path-following experiment along the rectangular trajectory.

4.2.1.6 Yaw

To evaluate the controller's performance in regulating the yaw attitude, a series of step inputs of ±45°, ±90°, and ±30° are applied to the yaw reference. The experimental results are presented in Figures 4.41, 4.42, and 4.43. The controller successfully tracks the yaw commands, demonstrating effective attitude regulation across the tested range. However, the control action induces disturbances in the robot's position, resulting in deviations of approximately 50 cm along the y direction and 30 cm along the x direction.

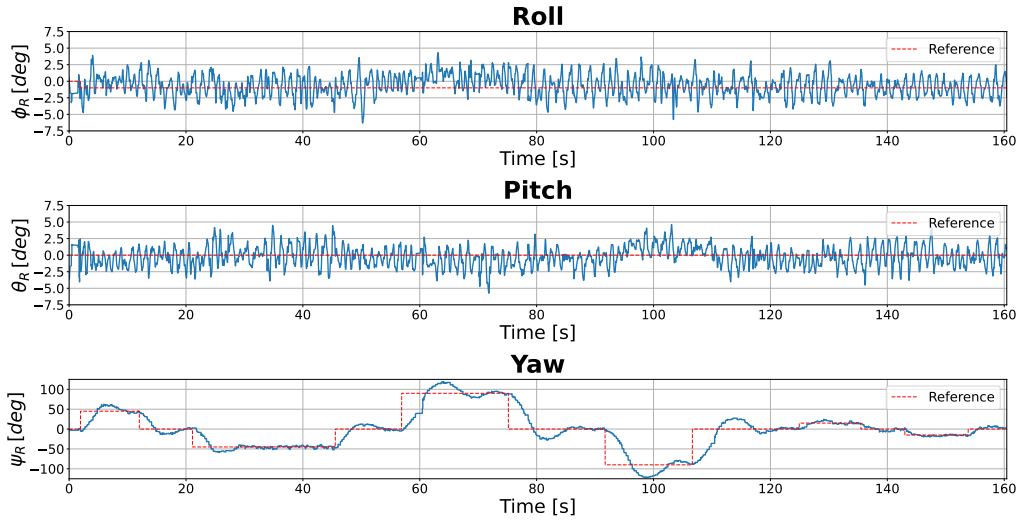


FIGURE 4.41 – Attitude angles for the yaw control experiment.

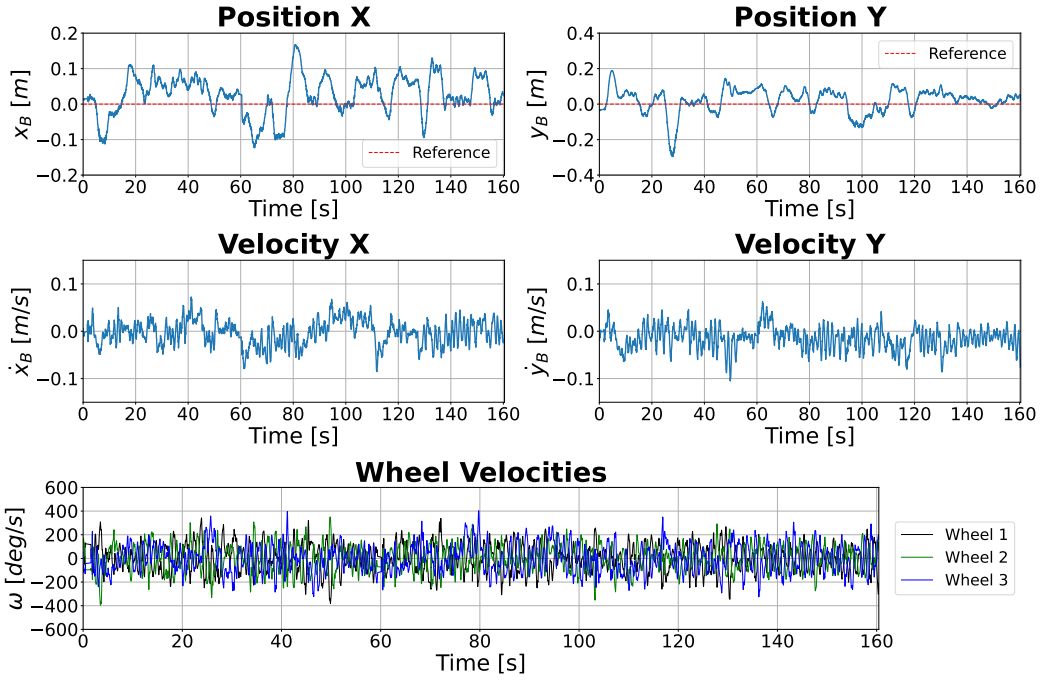


FIGURE 4.42 – Position, velocity, and actuator outputs for the yaw control experiment.

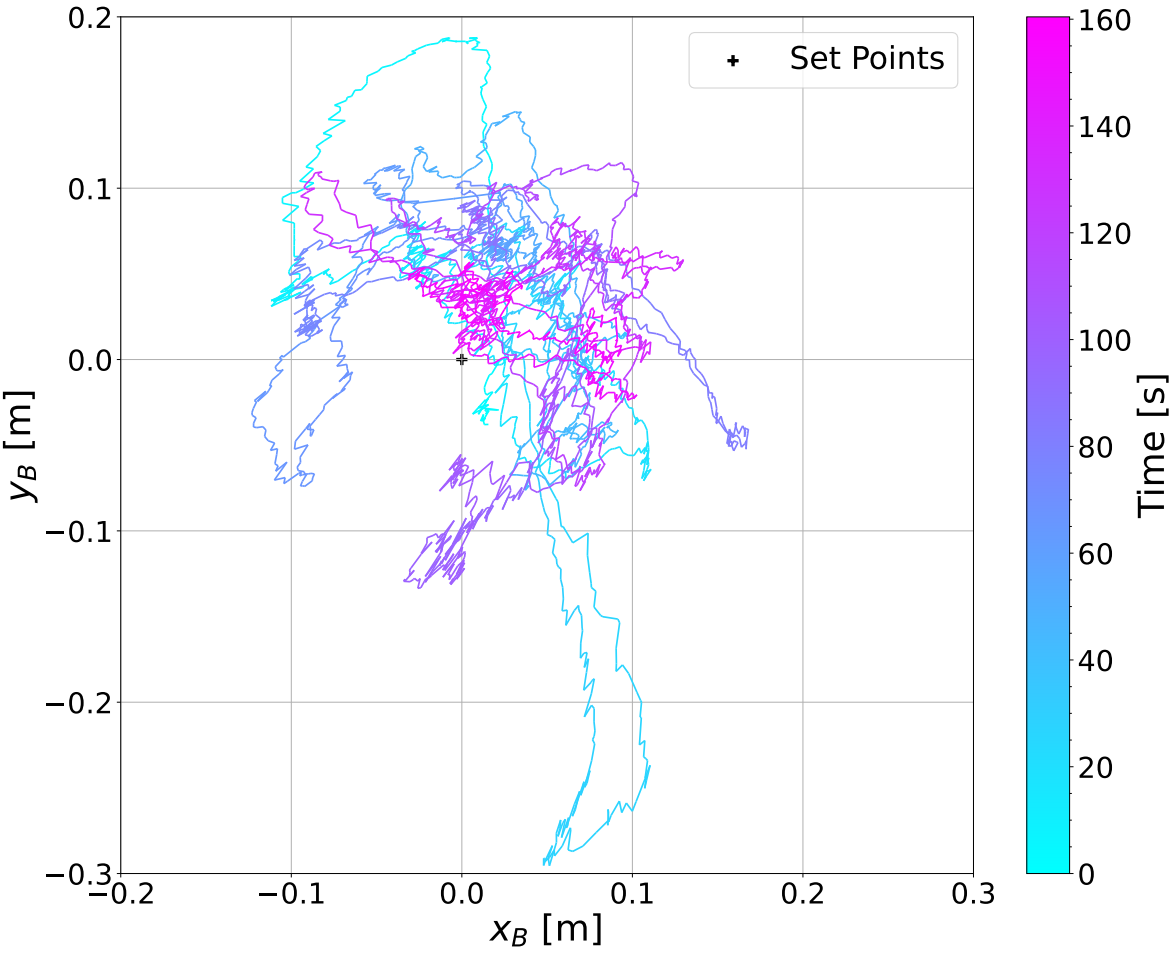


FIGURE 4.43 – X–Y trajectory for the yaw control experiment.

4.2.1.7 Path-Following Along A Rectangular Trajectory With Yaw Aligning

In this experiment, the yaw reference is adjusted to align the robot's heading with the direction of the target point during motion. This strategy aims to evaluate the controller's capability to coordinate translational and rotational movements while following a two-dimensional trajectory. The same sequence of position references forming a rectangular path, previously used in the fixed-yaw experiment, is employed here.

The robot moves sequentially toward each target point, adjusts its heading until it aligns with the direction of the next waypoint, waits for approximately one second to stabilize its orientation, and then proceeds to the following target. This process is repeated until all waypoints in the rectangular trajectory are reached, concluding at the initial central position.

The results, shown in Figures 4.44, 4.45, and 4.46, indicate that the robot successfully completes the trajectory while maintaining overall stability. The yaw angle effectively tracks the desired orientation toward each target point, demonstrating proper coordination

between heading and position control. However, the position deviations observed are higher than those obtained in the case where the yaw was kept fixed at 0°, with increased lateral displacements occurring.

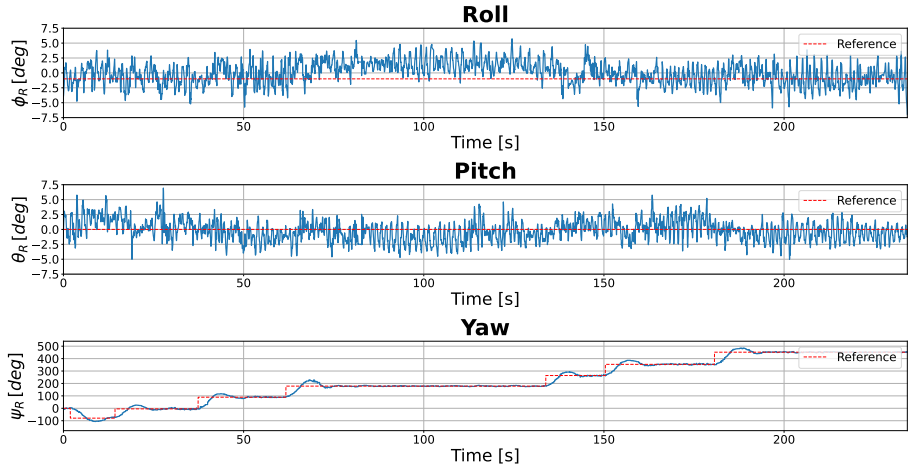


FIGURE 4.44 – Attitude angles for the rectangular path-following experiment with yaw alignment toward the target points.

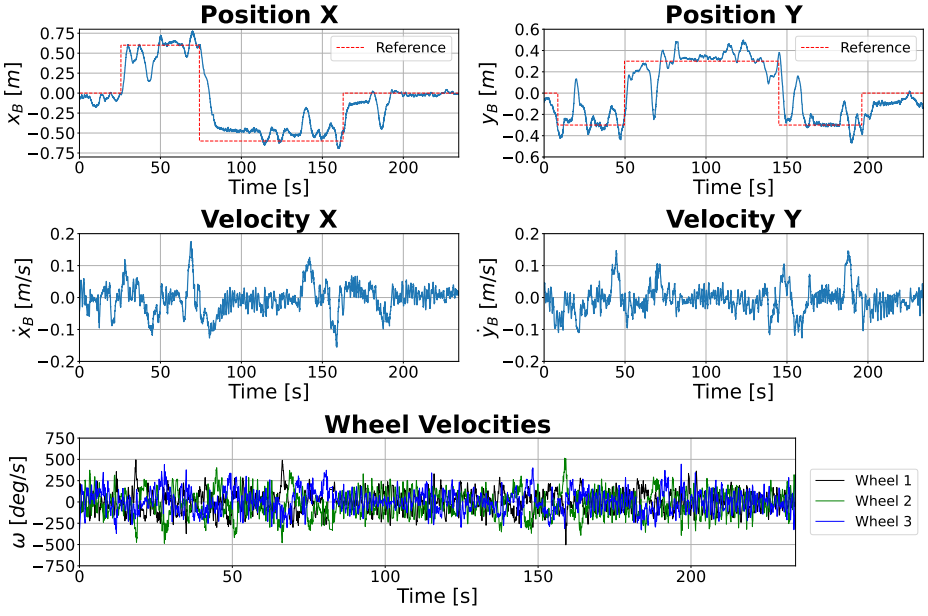


FIGURE 4.45 – Position, velocity, and actuator outputs for the rectangular path-following experiment with yaw alignment toward the target points.

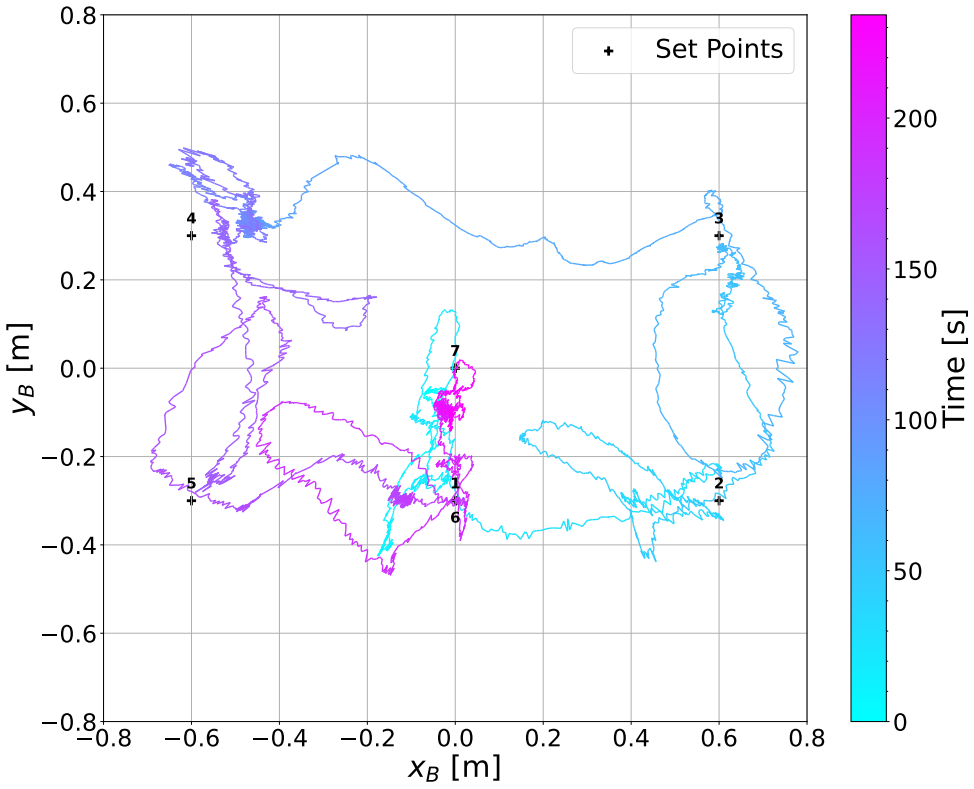


FIGURE 4.46 – X–Y trajectory for the rectangular path-following experiment with yaw alignment toward the target points.

4.2.2 LiDAR-Based Experiments

The following experiments employ the LiDAR as the position feedback sensor for the Ballbot, providing position updates at a frequency of 10 Hz. The experiments conducted replicate those performed with the camera-based system, allowing for a direct comparison between both sensors.

4.2.2.1 Balance

Figures 4.47, 4.48, and 4.49 show the results for the balancing experiment using LiDAR feedback. The controller successfully stabilizes the Ballbot around the upright position, maintaining roll and pitch angles within $\pm 5°$. Despite the lower update frequency of the LiDAR measurements, the robot remains near the initial position with minimal drift, demonstrating the robustness of the LQR controller to delayed position data.

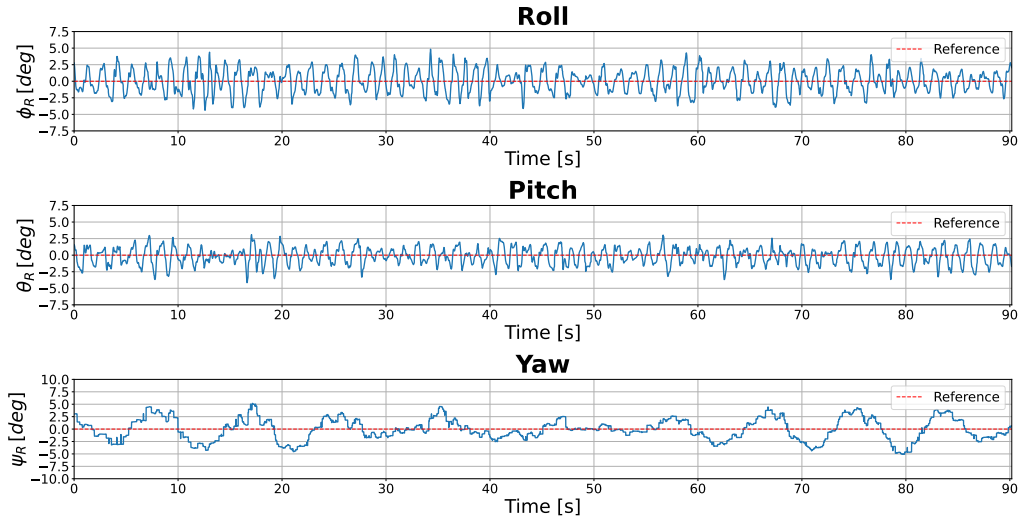


FIGURE 4.47 – Attitude angles for the balance experiment with LiDAR feedback.

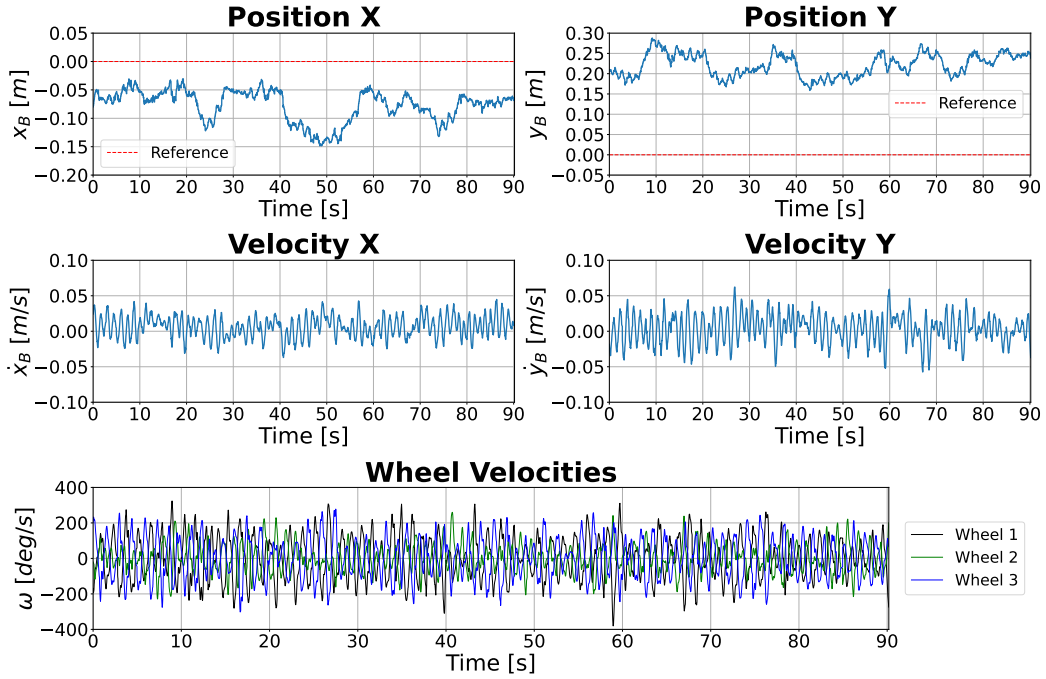


FIGURE 4.48 – Position, velocities, and actuator outputs for the balance experiment with LiDAR feedback.

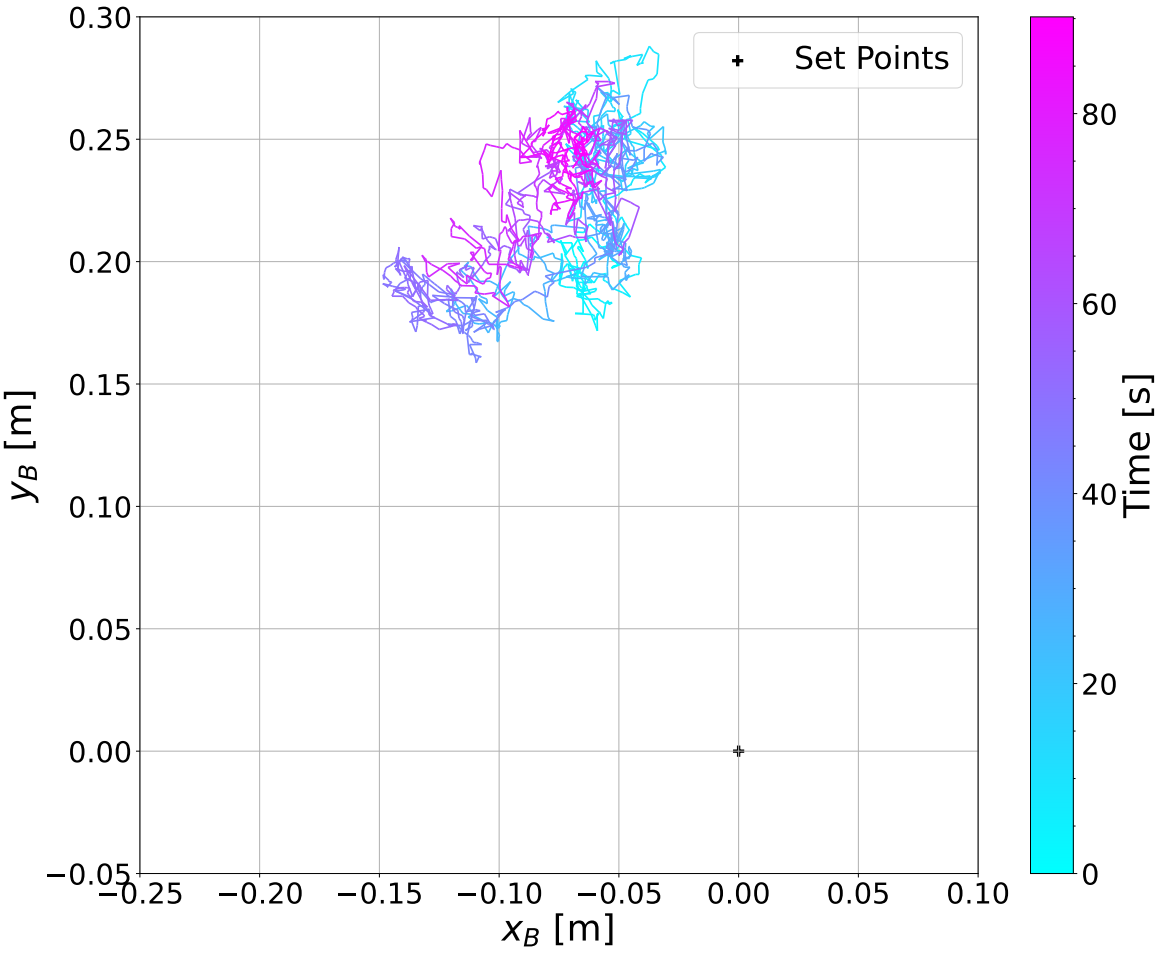


FIGURE 4.49 – X–Y trajectory for the balance experiment with LiDAR feedback.

4.2.2.2 Station-Keeping

In the station-keeping experiment, the controller is evaluated for its ability to maintain the robot's stability and regulate its position around the origin. The results, shown in Figures 4.50, 4.51, and 4.52, indicate that, consistent with the experiments using camera-based position feedback, the Ballbot remains stable, with a small steady-state position error. The figures also indicate that, at the conclusion of the experiment, the Ballbot loses balance and falls from over the ball due to a stall of the stepper motor.

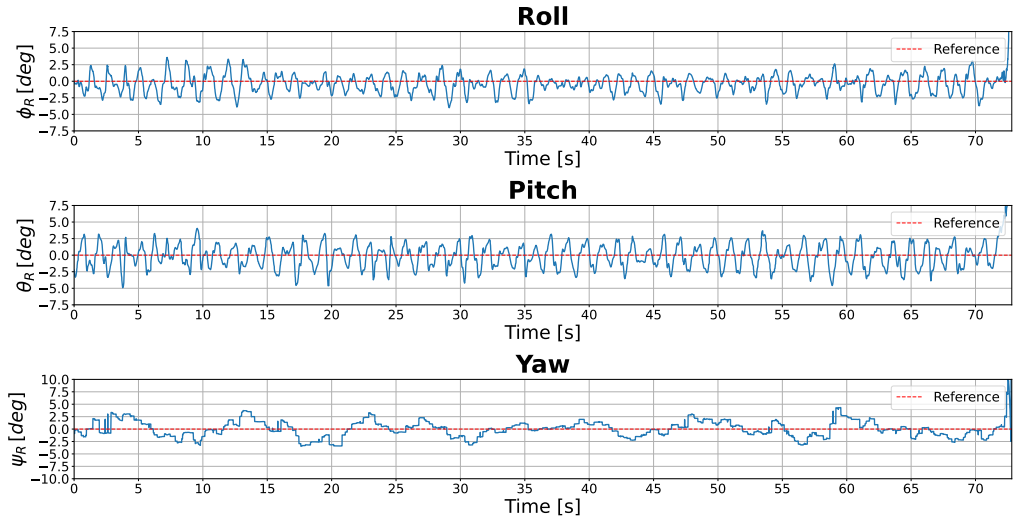


FIGURE 4.50 – Attitude angles for the station-keeping experiment with LiDAR feedback.

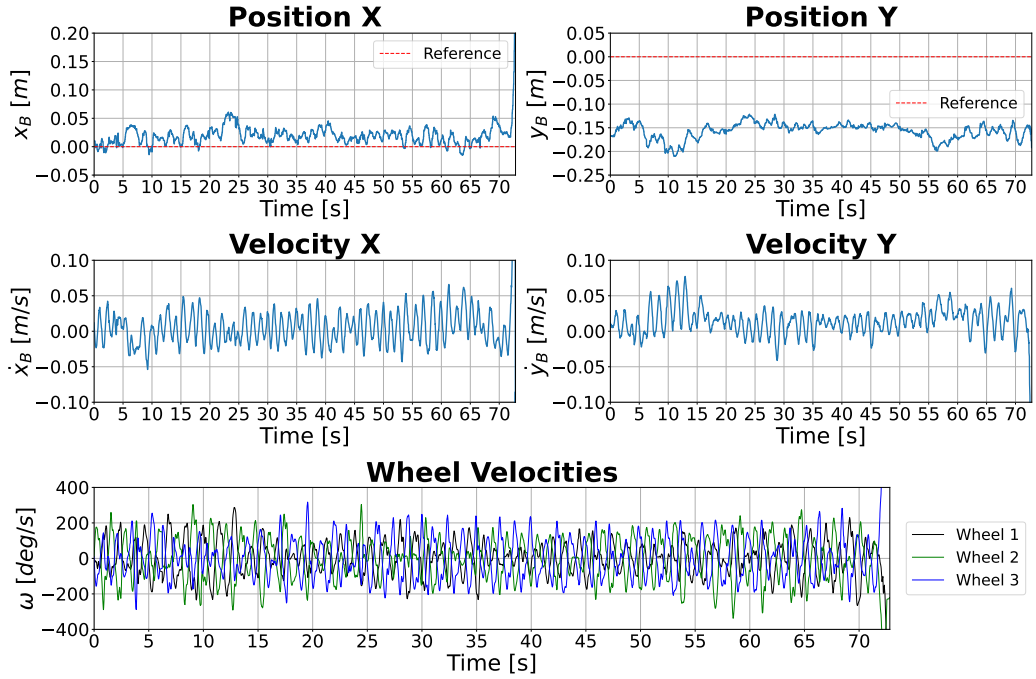


FIGURE 4.51 – Position, velocities, and actuator outputs for the station-keeping experiment with LiDAR feedback.

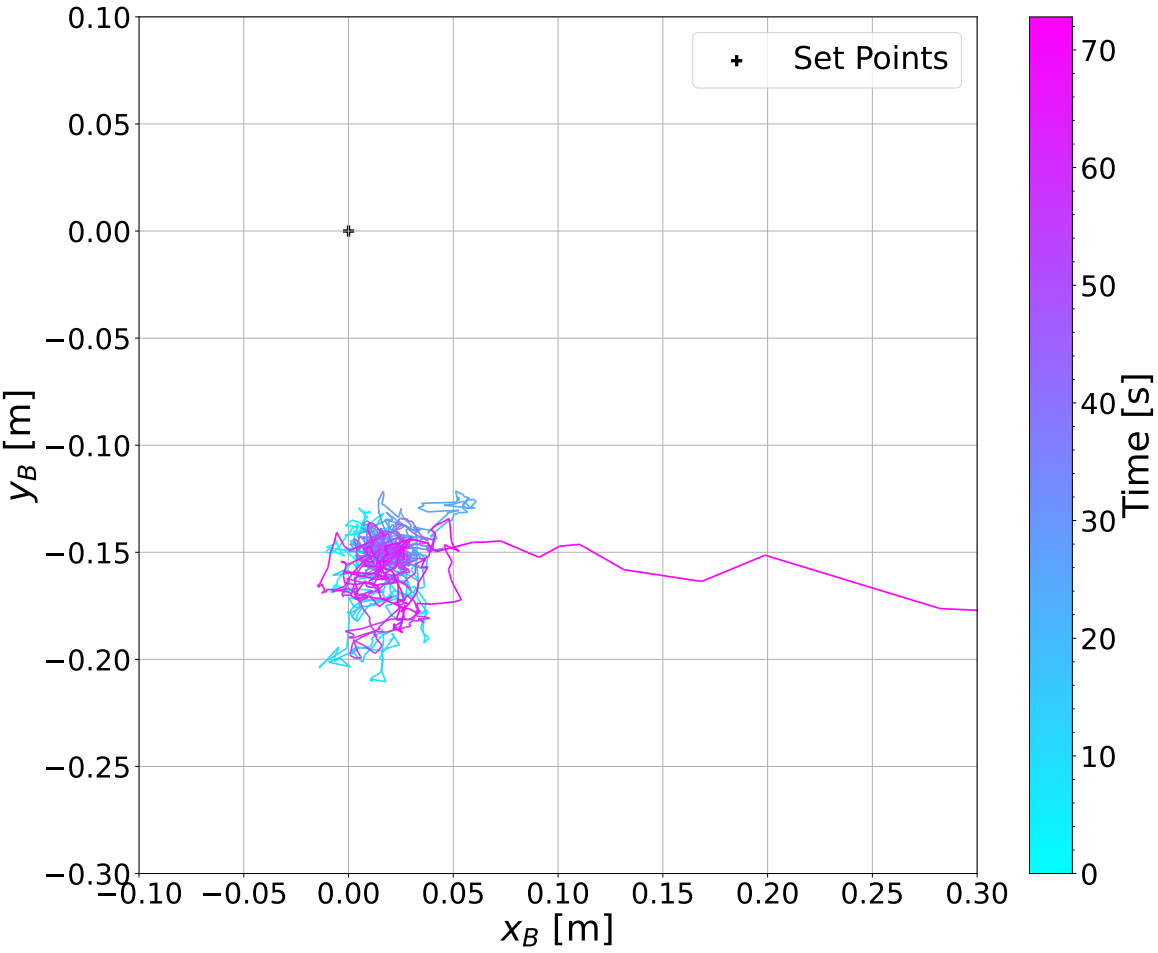


FIGURE 4.52 – X–Y trajectory for the station-keeping experiment with LiDAR feedback.

To reduce the position steady-state error, the LQR controller is modified to include integral action. The results of the station-keeping experiment with the integral-action controller are presented in Figures 4.53, 4.54, and 4.55. These results show that the modified controller effectively reduces the steady-state position error while maintaining the robot's stability.

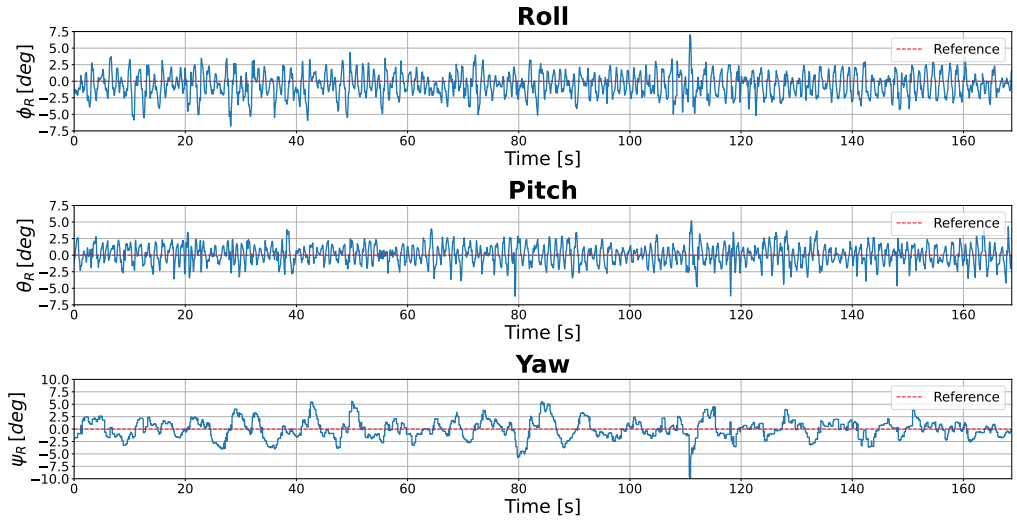


FIGURE 4.53 – Attitude angles for the station-keeping experiment with integral action using LiDAR feedback.

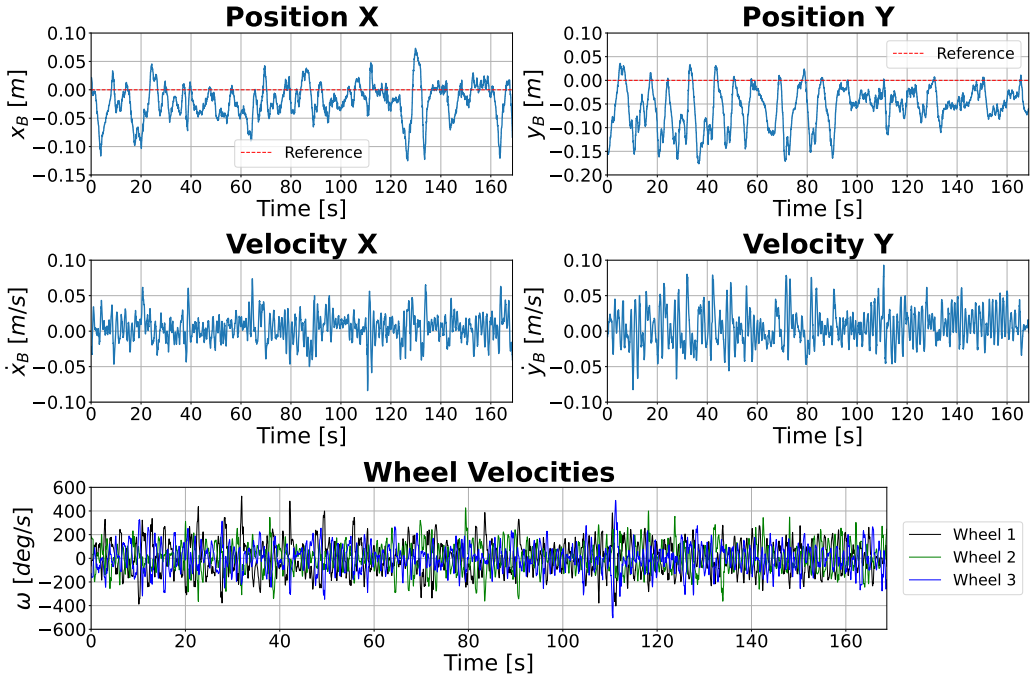


FIGURE 4.54 – Position, velocities, and actuator outputs for the station-keeping experiment with integral action using LiDAR feedback.

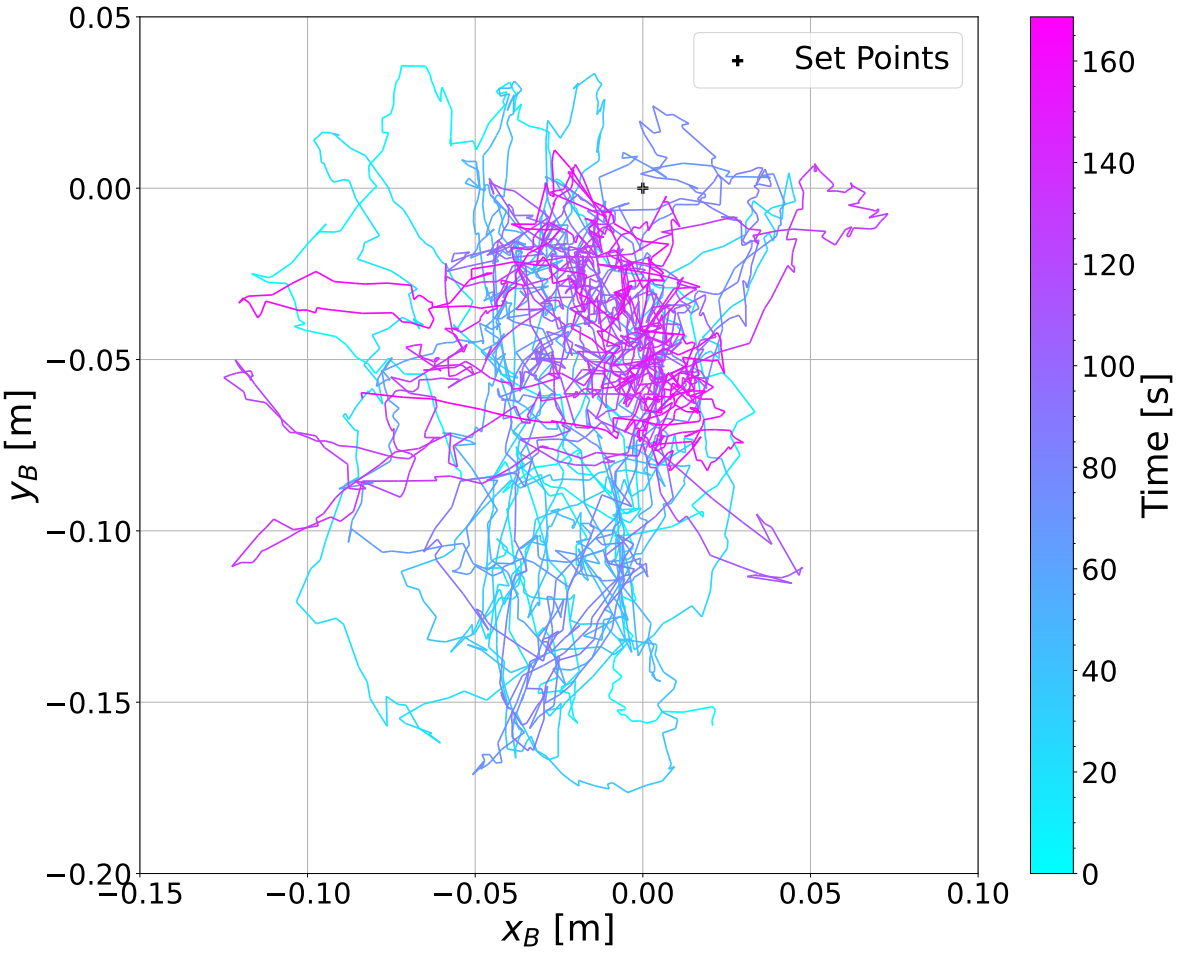


FIGURE 4.55 – X–Y trajectory for the station-keeping experiment with integral action using LiDAR feedback.

4.2.2.3 Path-Following Along The X Direction

The path-following experiment along the x-axis, shown in Figures 4.56, 4.57, and 4.58, demonstrates that the Ballbot successfully tracks the sequence of input references. The trajectory is completed in under 40 seconds, and the lateral deviation along the y-axis remains within approximately 20 cm around the central position, confirming stable performance with the LiDAR-based control loop.

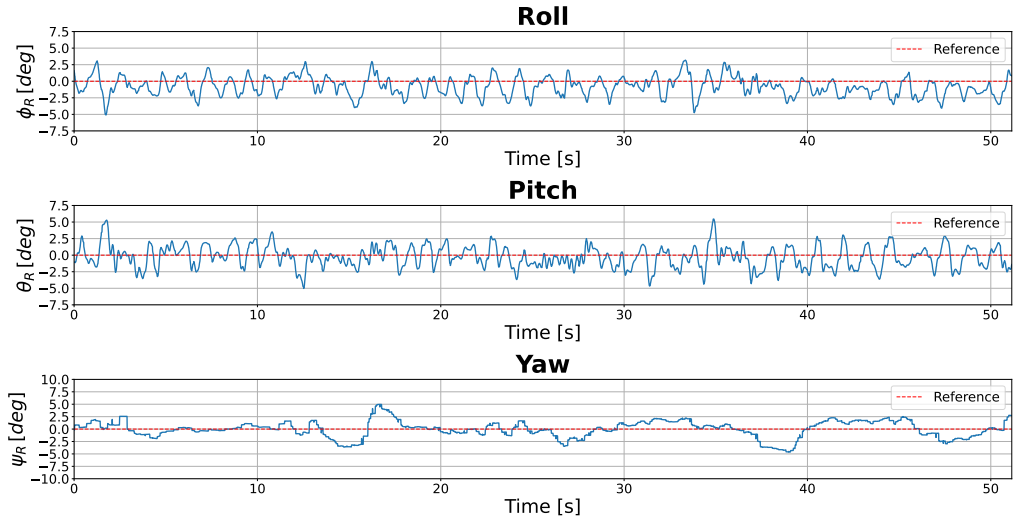


FIGURE 4.56 – Attitude angles for the path-following experiment along the x direction using LiDAR feedback.

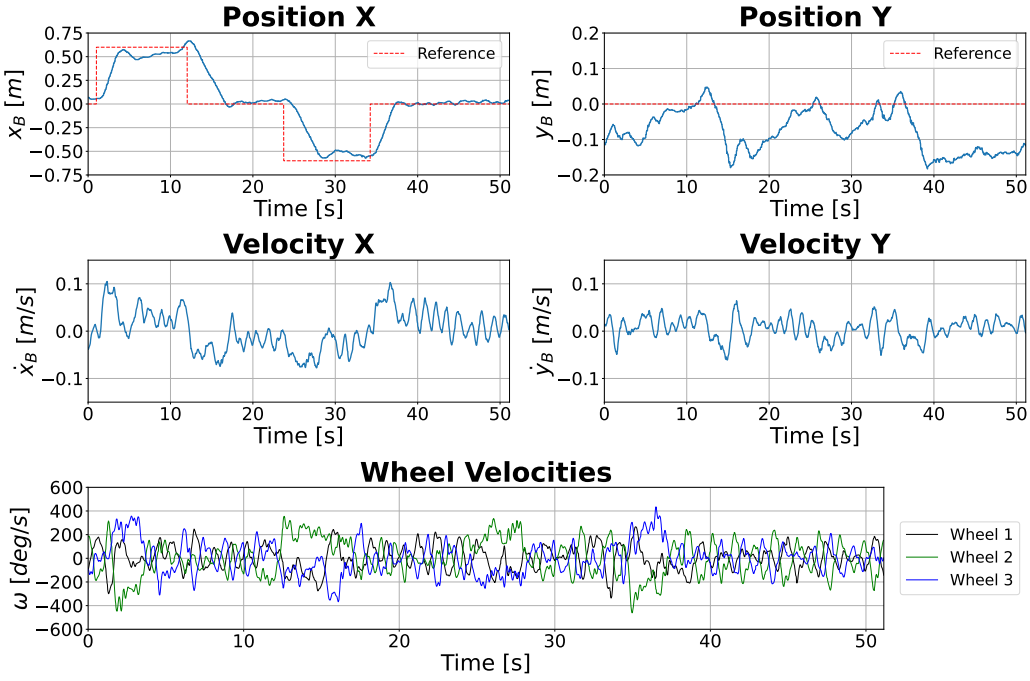


FIGURE 4.57 – Position, velocities, and actuator outputs for the path-following experiment along the x direction using LiDAR feedback.

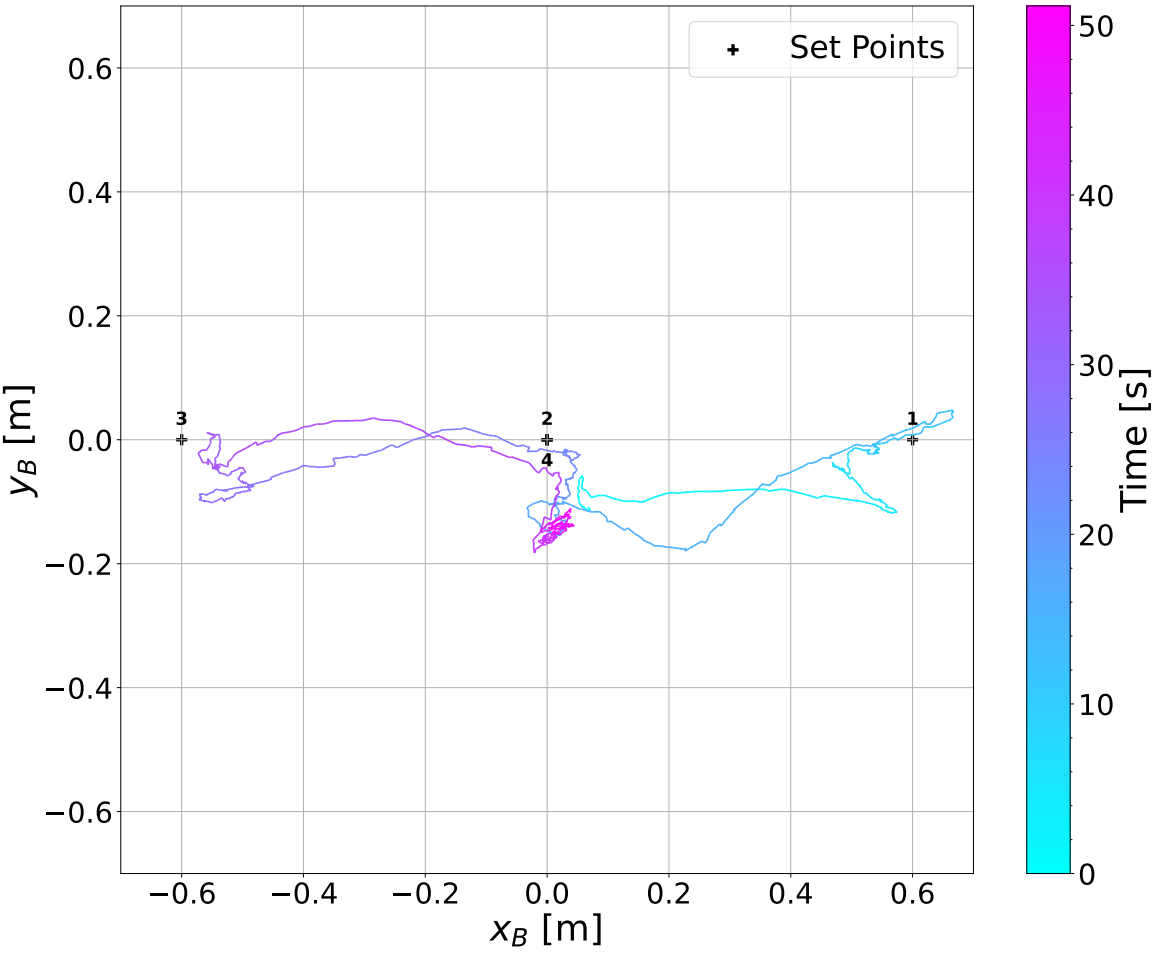


FIGURE 4.58 – X–Y trajectory for the path-following experiment along the x direction using LiDAR feedback.

4.2.2.4 Path-Following Along The Y Direction

A similar experiment is performed along the y-axis, as presented in Figures 4.59, 4.60, and 4.61. The sequence of references is completed in less than 30 seconds, with the deviation along the x-axis maintained below 20 cm. The results confirm that the controller effectively tracks position references along both directions using LiDAR position estimates.

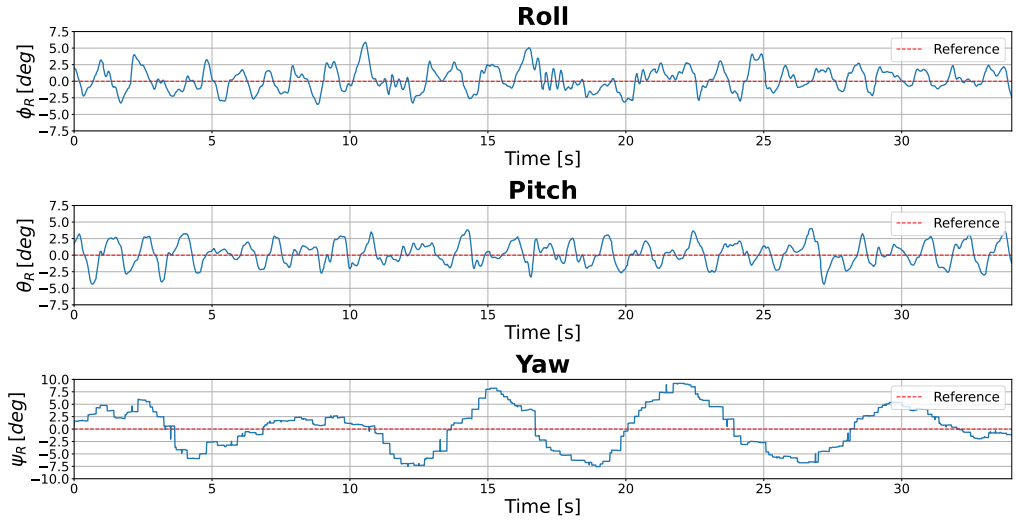


FIGURE 4.59 – Attitude angles for the path-following experiment along the y direction using LiDAR feedback.

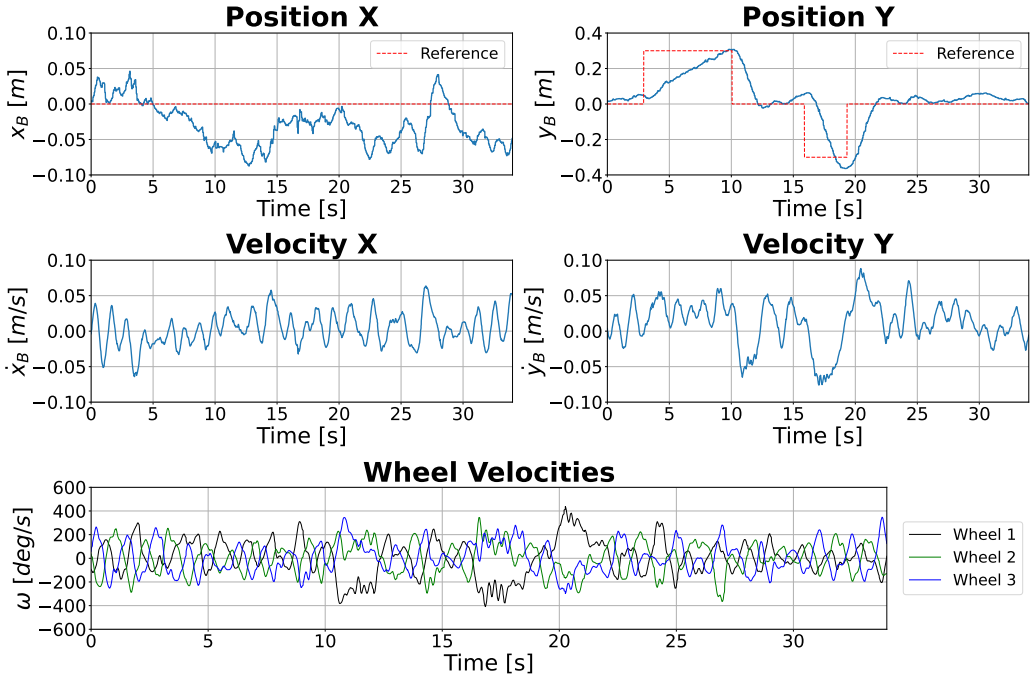


FIGURE 4.60 – Position, velocities, and actuator outputs for the path-following experiment along the y direction using LiDAR feedback.

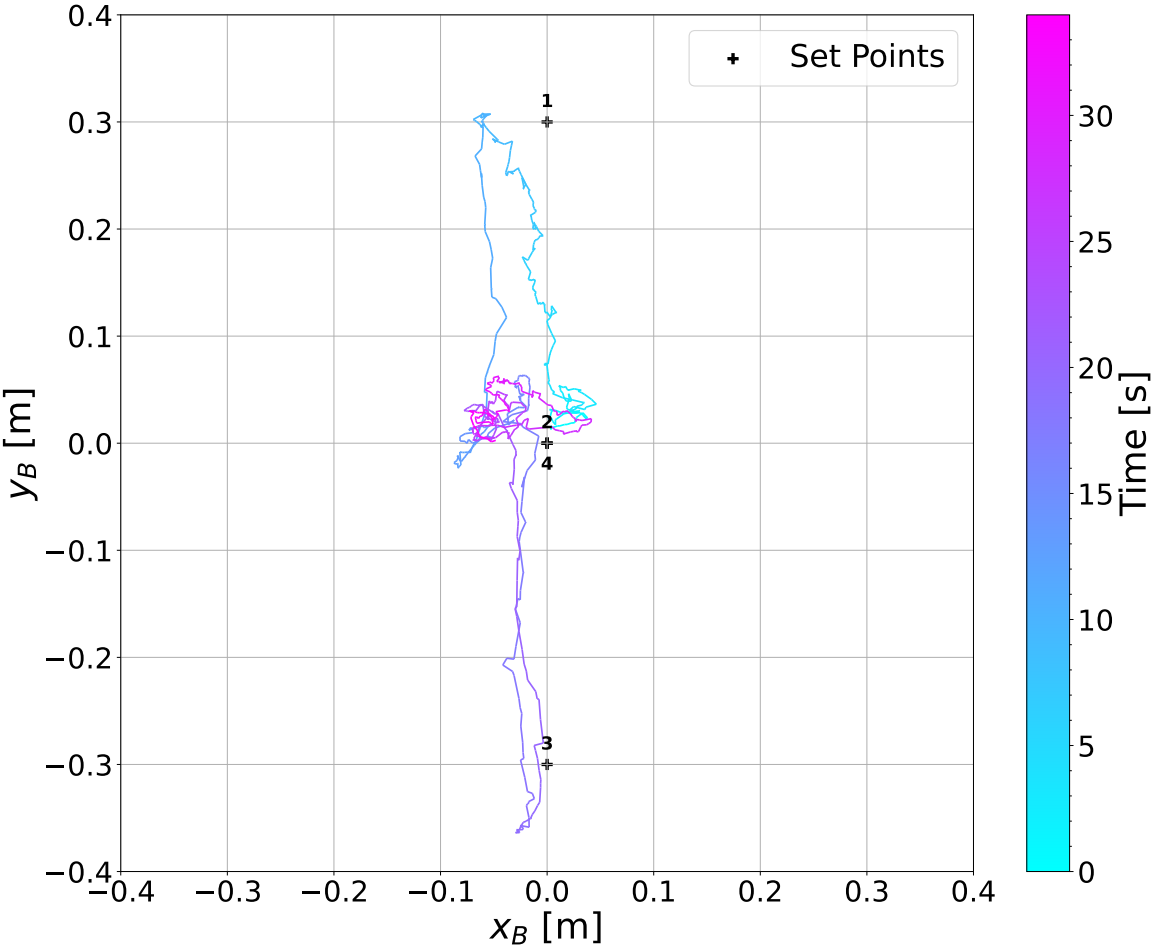


FIGURE 4.61 – X–Y trajectory for the path-following experiment along the y direction using LiDAR feedback.

4.2.2.5 Path-Following Along A Rectangular Trajectory

Finally, Figures 4.62, 4.63, and 4.64 present the results for the two-dimensional trajectory tracking experiment following a rectangular path. The Ballbot moves through seven target points defining the rectangular and returns to the initial position at the end of the sequence. The complete trajectory is executed in approximately 60 seconds, with roll and pitch angles maintained within $\pm 6°$ and yaw within $\pm 10°$. The results demonstrate the controller's ability to stabilize and guide the robot along a multi-directional path even under delayed LiDAR feedback conditions.

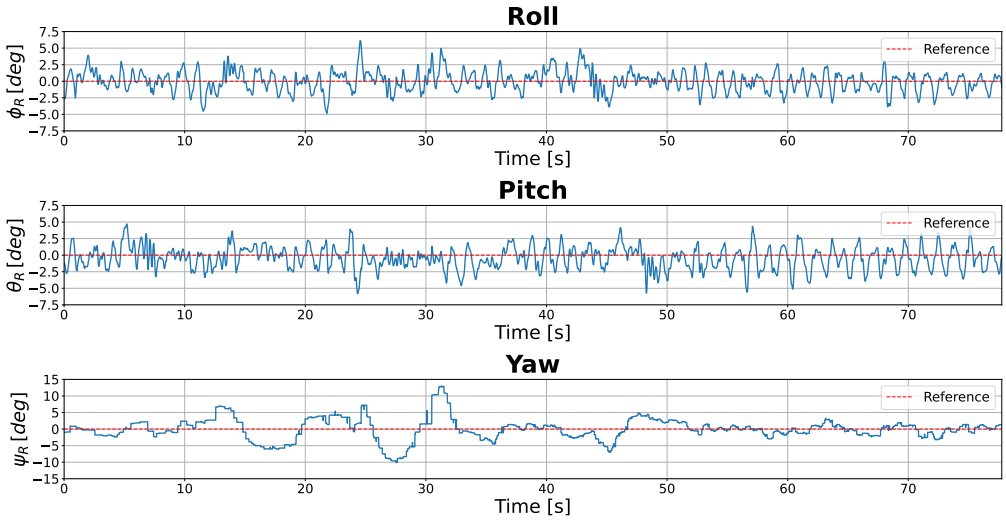


FIGURE 4.62 – Attitude angles for the rectangular path-following experiment using LiDAR feedback.

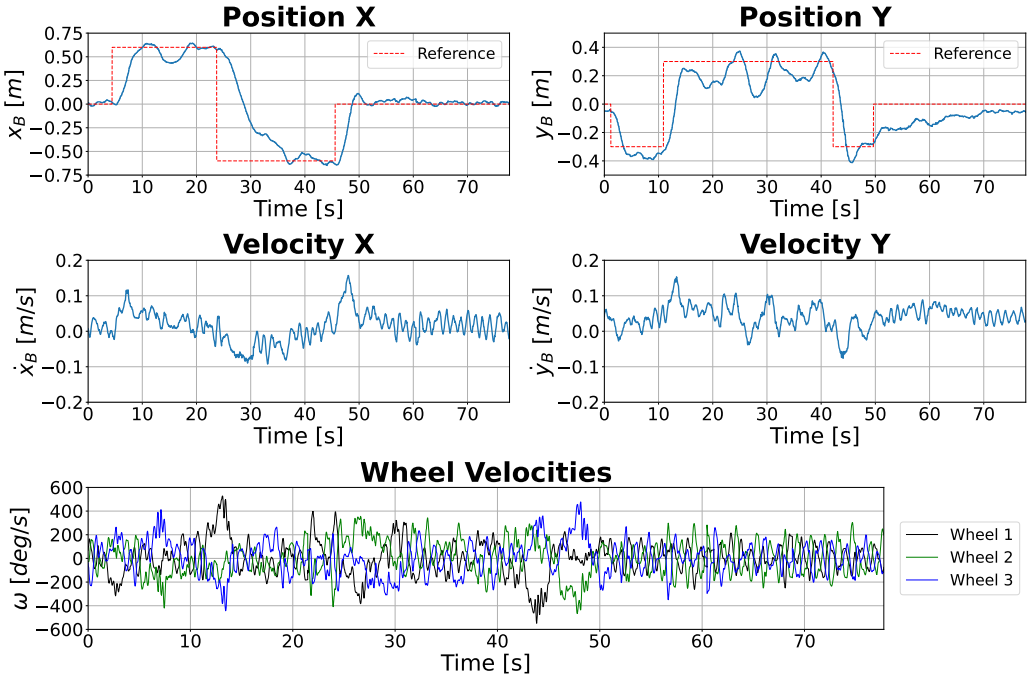


FIGURE 4.63 – Position, velocities, and actuator outputs for the rectangular path-following experiment using LiDAR feedback.

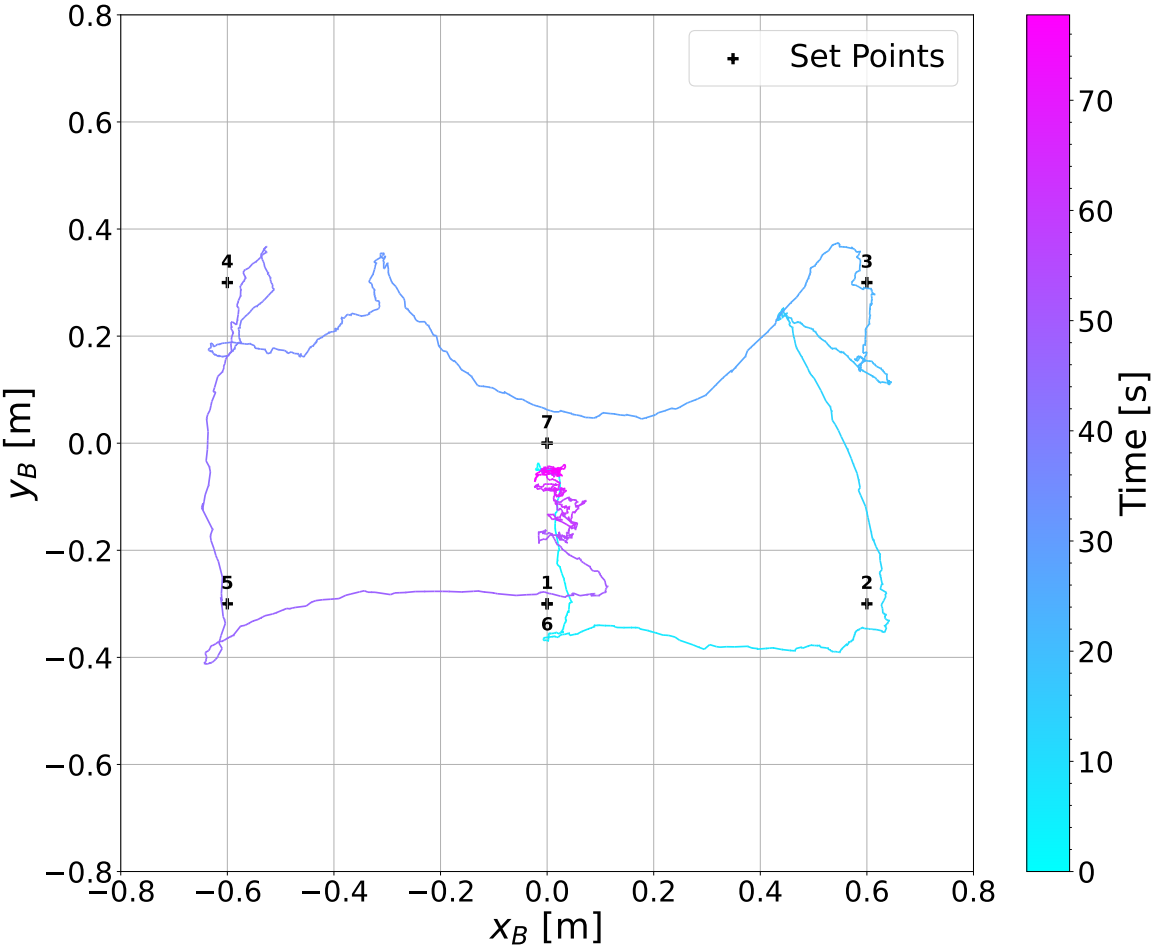


FIGURE 4.64 – X–Y trajectory for the rectangular path-following experiment using LiDAR feedback.

4.3 Results Analysis and Comparison

4.3.1 2D Simulation to 3D Simulation Comparison

Comparing the 2D and 3D simulation results for the case with an initial position offset, it can be observed that in the 2D simulation, the Ballbot returns to the origin in under 10 seconds, whereas in the 3D simulation, approximately 35 seconds are required for the robot to reach the reference position. Examining the attitude angles, the 2D simulation shows a smooth pitch motion of approximately $+2°$ in the x direction before returning to the upright equilibrium point. In contrast, the 3D simulation exhibits high-frequency oscillations around $+1°$. This difference in pitch behavior directly affects the linear velocity: in the 2D case, the Ballbot reaches a maximum velocity of about 0.2 m/s, while in the 3D case, the maximum velocity remains below 0.05 m/s. The observed attitude oscillations in both roll and pitch suggest the presence of coupled dynamics between the two planes, potentially causing one controller to interfere with the other.

Similarly, for the initial pitch disturbance scenario, both simulations demonstrate that the Ballbot recovers its upright equilibrium attitude in under 1 second. In the 3D simulation, however, oscillations are present, unlike the 2D case where the attitude response remains smooth. It is also worth noting that the 3D simulation exhibits a steady-state position error, primarily affecting the x direction, with an offset of approximately 3 cm.

4.3.2 3D Simulation to Experimental Comparison

Similarly to the 3D simulation results, the experimental data also exhibit the same oscillatory behavior in attitude. However, in the experimental case, the oscillation amplitude is larger than that observed in the simulations. This difference can be attributed to measurement noise from the attitude sensors and to nonlinear effects inherent to the physical system, such as dead zones and backlash caused by the clearance in the wheel rollers necessary for assembly.

The steady-state position error observed in the 3D simulations is also present in the physical system, though with a slightly higher magnitude. This discrepancy can be explained by the same nonlinearities, as well as IMU calibration errors arising from the difficulty of ensuring a perfectly leveled surface during calibration. Such errors may cause the robot to maintain an attitude slightly misaligned with its true upright position, leading to a consistent translational drift.

Furthermore, in all path-following experiments, the time required for the robot to complete the trajectory is shorter than in the 3D simulations. This can be explained by the higher maximum pitch and roll angles reached in the experiments, as larger lean angles toward the direction of motion result in greater translational velocities.

4.3.3 Camera to LiDAR Comparison

The results show that the controller's performance is essentially the same for the two sensors providing pose information, despite the camera operating at a higher frequency (30 Hz) compared to the LiDAR (10 Hz) and the LiDAR introducing additional noise to the IMU due to vibrations caused by its rotation.

5 Conclusions

The proposed system architecture for the Ballbot framework successfully achieved the objectives set out for this work. By adopting a layered architecture, the framework maintained a high-frequency control loop, essential for stable balancing, while simultaneously enabling external communication and real-time monitoring of the robot's full state. This modular design enabled flexible hardware integration, allowing sensors to be quickly swapped and tested, facilitating the evaluation of different pose estimation methods, such as the PnP using external cameras, and the Laser Scan Matching using LiDAR. The results indicate that the performance of both the LQR and PD controllers remains consistent across the different sensors, despite differences in update frequency and noise characteristics, with the LiDAR introducing additional vibrations that influence the IMU readings.

While the system demonstrated satisfactory control in both simulations and experiments, several real-world limitations were identified. Experimental data exhibited larger attitude oscillations compared to 3D simulations, likely due to sensor measurement noise and nonlinear effects inherent to the physical system, such as pitch-yaw coupling, dead zones and backlash in the wheel rollers. The steady-state position error was also slightly higher in experiments, resulting from the nonlinearities and IMU calibration errors caused by the difficulty of ensuring a perfectly leveled surface.

Furthermore, the stepper motor employed in the system was verified to be operating at the limit of its output torque. Earlier iterations of the robot, with higher mass, were unable to maintain position control, as the required torque exceeded the motor's capabilities and resulted in stalls, as evidenced in the LiDAR Station-Keeping experiment. This finding highlights the importance of appropriate actuator selection in ensuring stable and reliable performance.

Overall, the proposed architecture provided a scalable and adaptable platform for continued research and education. It enables rapid experimentation with new sensors and control strategies, while offering valuable insights into the practical challenges of implementing balance control on dynamically unstable robots.

5.1 Future Work

Future work should focus on addressing the current system limitations and enhancing the overall performance of the Ballbot platform. The main directions for future development include:

- Replace the current stepper motors with higher-torque alternatives to prevent stalling and increase control authority.
- Redesign the wheels to reduce roller clearance and minimize backlash and dead-zone effects.
- Mechanically isolate the IMU to mitigate vibrations originating from the wheel–ball interaction, motors, and LiDAR rotation.
- Investigate different control approaches, such as gain-scheduling, to improve stability and performance under varying operating conditions.
- Implement a state estimation algorithm for full-system state reconstruction, such as a Kalman filter.
- Extend the control framework to include translational velocity and yaw-rate regulation.
- Implement a full three-dimensional dynamic model and controller to address the coupled dynamics of the system and improve overall performance.

Bibliography

ASTROM, K. J.; MURRAY, R. M. **Feedback Systems: An Introduction for Scientists and Engineers**. Princeton: Princeton University Press, 2008.

BLONK, J. **Modeling and control of a ball-balancing robot**. 139 p. Dissertation (Master's Thesis) — University of Twente, 2014.

CENSI, A. An icp variant using a point-to-line metric. *In*: **2008 IEEE International Conference on Robotics and Automation**. **Proceedings** [...]. Pasadena: IEEE, 2008. p. 19–25. Available at: <https://doi.org/10.1109/ROBOT.2008.4543181>.

FANKHAUSER, P.; GWERDER, C. **Modeling and control of a ballbot**. 88 p. Dissertation (B.S. thesis) — Eidgenössische Technische Hochschule Zürich, 2010.

FARSHIDIAN, F.; NEUNERT, M.; BUCHLI, J. Learning of closed-loop motion control. *In*: **2014 IEEE/RSJ International Conference on Intelligent Robots and Systems**. **Proceedings** [...]. Chicago: IEEE, 2014. p. 1441–1446. Available at: <https://doi.org/10.1109/IROS.2014.6942746>.

FURGALE, P.; SOMMER, H.; MAYE, J.; REHDER, J.; SCHNEIDER, T.; OTH, L. **Kalibr: the Kalibr Visual-Inertial Calibration Toolbox**. 2022. Available at: <https://github.com/ethz-asl/kalibr>. Accessed on September 29, 2025.

JESPERSEN, T. K. **Kugle - Modelling and Control of a Ball-balancing Robot**. 274 p. Dissertation (Master's Thesis) — Aalborg University, 2019.

JESUS, P. H. d. **Projeto, Construção e Testes de Um Robô Móvel Omnidirecional do Tipo Ballbot**. 149 p. Dissertation (Master's Thesis) — Instituto Tecnológico de Aeronáutica, São José dos Campos, Brazil, 2021.

KIRK, D. E. **Optimal Control Theory: An Introduction**. Englewood Cliffs: Prentice-Hall, 1970.

KUMAGAI, M.; OCHIAI, T. Development of a robot balancing on a ball. *In*: **2008 International Conference on Control, Automation and Systems**. **Proceedings** [...]. Seoul: IEEE, 2008. p. 433–438. Available at: <https://doi.org/10.1109/ICCAS.2008.4694680>.

KUMAGAI, M.; OCHIAI, T. Development of a robot balanced on a ball — application of passive motion to transport —. *In*: **2009 IEEE International Conference on Robotics and Automation**. **Proceedings** [...]. Kobe: IEEE, 2009. p. 4106–4111. Available at: <https://doi.org/10.1109/ROBOT.2009.5152324>.

LAUWERS, T.; KANTOR, G.; HOLLIS, R. A dynamically stable single-wheeled mobile robot with inverse mouse-ball drive. *In*: **Proceedings 2006 IEEE International Conference on Robotics and Automation, 2006. ICRA 2006. Proceedings** [...]. Orlando: IEEE, 2006. p. 2884–2889. Available at: <https://doi.org/10.1109/ROBOT.2006.1642139>.

MINNITI, M. V.; FARSHIDIAN, F.; GRANDIA, R.; HUTTER, M. Whole-body MPC for a dynamically stable mobile manipulator. **IEEE Robotics and Automation Letters**, v. 4, n. 4, p. 3687–3694, 2019.

MINNITI, M. V.; GRANDIA, R.; FÄH, K.; FARSHIDIAN, F.; HUTTER, M. Model predictive robot-environment interaction control for mobile manipulation tasks. *In*: **2021 IEEE International Conference on Robotics and Automation (ICRA)**. **Proceedings** [...]. Xi'an: IEEE, 2021. p. 1651–1657. Available at: <https://doi.org/10.1109/ICRA48506.2021.9562066>.

NAGARAJAN, U.; KANTOR, G.; HOLLIS, R. The ballbot: An omnidirectional balancing mobile robot. **The International Journal of Robotics Research**, v. 33, n. 6, p. 917–930, 2014.

NAGARAJAN, U.; KIM, B.; HOLLIS, R. Planning in high-dimensional shape space for a single-wheeled balancing mobile robot with arms. *In*: **2012 IEEE International Conference on Robotics and Automation**. **Proceedings** [...]. Saint Paul: IEEE, 2012. p. 130–135. Available at: <https://doi.org/10.1109/ICRA.2012.6225065>.

OPENCV. **Detection of ArUco Markers**. 2024. Available at: <https://docs.opencv.org/4.x/d5/dae/tutorial_aruco_detection.html>. Accessed on September 29, 2025.

OTH, L.; FURGALE, P.; KNEIP, L.; SIEGWART, R. Rolling shutter camera calibration. *In*: **2013 IEEE Conference on Computer Vision and Pattern Recognition**. **Proceedings** [...]. Portland: IEEE, 2013. p. 1360–1367. Available at: <https://doi.org/10.1109/CVPR.2013.179>.

ROS WIKI. **laser_scan_matcher**. 2024. Available at: <https://wiki.ros.org/laser_scan_matcher>. Accessed on September 29, 2025.

SHOMIN, M.; FORLIZZI, J.; HOLLIS, R. Sit-to-stand assistance with a balancing mobile robot. *In*: **2015 IEEE International Conference on Robotics and Automation (ICRA)**. **Proceedings** [...]. Seattle: IEEE, 2015. p. 3795–3800. Available at: <https://doi.org/10.1109/ICRA.2015.7139727>.

SHUT, R.; HOLLIS, R. Development of a humanoid dual arm system for a single spherical wheeled balancing mobile robot. *In*: **2019 IEEE-RAS 19th International Conference on Humanoid Robots (Humanoids)**. **Proceedings** [...]. Toronto: IEEE, 2019. p. 499–504. Available at: <https://doi.org/10.1109/Humanoids43949.2019.9034999>.

SONNLEITNER, F.; SHU, R.; HOLLIS, R. L. The mechanics and control of leaning to lift heavy objects with a dynamically stable mobile robot. *In*: **2019 International Conference on Robotics and Automation (ICRA)**. **Proceedings** [...]. Montreal: IEEE, 2019. p. 9264–9270. Available at: <https://doi.org/10.1109/ICRA.2019.8793620>.

VAIDYA, B.; SHOMIN, M.; HOLLIS, R.; KANTOR, G. Operation of the ballbot on slopes and with center-of-mass offsets. *In*: **2015 IEEE International Conference on Robotics and Automation (ICRA)**. **Proceedings** [...]. Seattle: IEEE, 2015. p. 2383–2388. Available at: <https://doi.org/10.1109/ICRA.2015.7139516>.

XIAO, C. **Personal unique rolling experience: Design, modeling, and control of a riding ballbot**. 138 p. Thesis (PhD Thesis) — University of Illinois at Urbana-Champaign, 2022.

FOLHA DE REGISTRO DO DOCUMENTO

1. CLASSIFICAÇÃO/TIPO	2. DATA	3. DOCUMENTO Nº	4. Nº DE PÁGINAS
DM	22 de janeiro de 2026	DCTA/ITA/DM-102/2025	116

5. TÍTULO E SUBTÍTULO:

Design and Experimental Validation of a Modular Ballbot Platform for Balance Control Studies

6. AUTOR(ES):

Leandro Ventricci

7. INSTITUIÇÃO(ÕES)/ÓRGÃO(S) INTERNO(S)/DIVISÃO(ÕES):

Instituto Tecnológico de Aeronáutica – ITA

8. PALAVRAS-CHAVE SUGERIDAS PELO AUTOR:

Control Systems; Robotics; Embedded Systems.

9. PALAVRAS-CHAVE RESULTANTES DE INDEXAÇÃO:

Controle automático; Robôs; Sistemas de controle; Sistemas embarcados; Sensores; Robótica; Controle; Engenharia eletrônica.

10. APRESENTAÇÃO: ( ) **Nacional** (**X**) **Internacional**

ITA, São José dos Campos. Curso de Mestrado. Programa de Pós-Graduação em Engenharia Eletrônica e Computação. Área de Sistemas e Controle. Orientador: Prof. Dr. Cairo Lúcio Nascimento Júnior. Defesa em 09/12/2025. Publicada em 2025.

11. RESUMO:

This work presents the design, implementation, and experimental validation of a modular Ballbot framework developed for balance control studies. The system employs a layered architecture in which a high-frequency control loop runs on an ESP32 microcontroller, while communication and monitoring tasks are handled by a Raspberry Pi 4 through the Robot Operating System (ROS) framework. This architecture ensures that the control loops execute reliably at the required frequency for real-time operation, enables continuous monitoring of the robot's complete state, and preserves overall system flexibility. The modular design further enables easy hardware integration, allowing sensors to be quickly swapped and tested, facilitating the evaluation of different pose estimation methods, such as the Perspective-n-Point (PnP) using external cameras, and Laser Scan Matching using Light Detection and Ranging (LiDAR) sensors. To regulate the robot's motion, both a Linear Quadratic Regulator (LQR) and a Proportional-Derivative (PD) controller were implemented and tested. The LQR was employed to control the robot's balance and position in the XZ and YZ planes, while the PD controller regulated the robot's heading. Experimental results demonstrate that the controller maintains the ballbot balanced around the upright position within a region of $\pm 5^\circ$ in both roll and pitch angles. Additionally, the controller successfully regulates the robot's position, achieving deviations below 20 cm for both evaluated sensors, despite differences in update frequency and noise characteristics, with the LiDAR introducing additional vibrations that affected the Inertial Measurement Unit (IMU) readings. Overall, the proposed architecture provided a scalable and adaptable platform for continued research and education. It enables rapid experimentation with new sensors and control strategies, while offering valuable insights into the practical challenges of implementing balance control on dynamically unstable robots. The complete source code, structure design files, and videos of the experimental results are available in the project's GitHub repository: `https://github.com/Intelligent-Machines-Lab/Ballbot_LMI_V2`.

12. GRAU DE SIGILO:

(**X**) **OSTENSIVO** ( ) **RESERVADO** ( ) **SECRETO**